

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP

Diep.NK225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh

Ngành: Công nghệ thông tin

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP

Diep.NK225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh _____

Chữ kí GVHD

Khoa: Công nghệ thông tin Việt-Nhật

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

LỜI CẢM ƠN

Để hoàn thành đồ án này, em xin gửi lời cảm ơn chân thành đến quý thầy cô đã tận tâm hướng dẫn và trang bị kiến thức chuyên môn quý báu. Con xin cảm ơn gia đình vì luôn là chỗ dựa tinh thần vô giá. Cảm ơn những người bạn đồng hành và đặc biệt là người yêu đã luôn thấu hiểu, san sẻ áp lực và động viên em trong suốt thời gian qua. Và cuối cùng, tôi muốn gửi lời cảm ơn đến chính bản thân mình vì đã không bỏ cuộc, luôn chăm chỉ, kiên định và quyết tâm nỗ lực vượt qua mọi thử thách để hoàn thiện công việc với kết quả tốt nhất. Sự đồng hành của tất cả mọi người là động lực to lớn giúp em đạt được cột mốc này. Xin trân trọng cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh chuyển đổi số giáo dục, nhu cầu ứng dụng trí tuệ nhân tạo để hỗ trợ dạy và học ngày càng rõ rệt. Với môn Tin học THPT, học sinh cần một công cụ có thể trả lời câu hỏi theo sách giáo khoa, còn giáo viên cần hỗ trợ xây dựng dàn ý slide hoặc giáo án. Hiện nay, chatbot tổng quát có ưu điểm là phản hồi linh hoạt nhưng dễ sinh thông tin sai, trong khi các hệ thống tra cứu tài liệu bám nguồn tốt hơn nhưng kém tự nhiên. Vì vậy, cần có một hệ thống vừa bám sát sách giáo khoa, vừa đủ linh hoạt để hỗ trợ học tập.

Đồ án lựa chọn hướng tiếp cận kết hợp giữa truy xuất nội dung từ sách giáo khoa và sinh câu trả lời dựa trên nội dung truy xuất được. Hướng này giúp giảm nguy cơ trả lời sai nhưng vẫn giữ được khả năng phản hồi tự nhiên.

Giải pháp được xây dựng trên dữ liệu SGK Tin học lớp 10, 11 và 12 của hai bộ sách Cánh Diều và Kết Nối Tri Thức. Nội dung sách giáo khoa được tiền xử lý, chia thành các đoạn có cấu trúc và đưa vào kho tri thức để phục vụ truy xuất. Khi nhận yêu cầu, hệ thống tìm các đoạn nội dung phù hợp rồi mới tạo câu trả lời hoặc học liệu.

Đóng góp chính của đồ án là xây dựng được một trợ lý giáo dục theo hướng nghiên cứu và đánh giá tách biệt khỏi truy xuất với khối sinh câu trả lời. Với bộ đánh giá truy xuất gồm 600 câu hỏi bao phủ 185 bài học, hệ thống đạt Hit@1 bằng 0,8183 và Hit@10 bằng 0,9833. Với 246 mẫu đánh giá chất lượng sinh, hệ thống đạt Faithfulness 0,9757 và Context Recall 0,9593. Các kết quả này cho thấy hướng tiếp cận được lựa chọn là khả thi đối với bài toán xây dựng trợ lý học tập bám sát SGK Tin học THPT.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

In educational digital transformation, the demand for artificial intelligence in teaching and learning is growing rapidly, especially for high school Computer Science. Students need a tool that can answer questions according to official textbooks, while teachers need support for tasks such as preparing slide outlines and lesson plans. Existing solutions mainly follow two directions. General-purpose chatbots provide flexible responses, but they may generate incorrect or out-of-scope information. Traditional document retrieval systems are more source-grounded, but they are often less effective at producing natural and instructionally useful outputs. Therefore, the key problem is to build a system that remains faithful to textbook content while still being flexible enough to support learning tasks.

To address this problem, the thesis adopts an approach that combines textbook retrieval with response generation based on the retrieved content. This direction was chosen because it can reduce the risk of incorrect answers compared with general-purpose chatbots, while still keeping the interaction natural for users.

The proposed solution is built on Vietnamese high school Computer Science textbooks for grades 10 to 12 from the Canh Dieu and Ket Noi Tri Thuc series. The system includes one layer for user-request orchestration and another layer for content generation in tasks such as question answering, slide generation, and lesson-plan drafting. Textbook content is first preprocessed, segmented into structured units, and stored in a knowledge base for retrieval. When a request is received, the system first finds the most relevant textbook content and then uses a language model to generate the final response or instructional artifact. This design improves source grounding and makes output quality easier to control.

The main contribution of the thesis is an educational assistant developed in a research-oriented manner, where retrieval and generation are evaluated separately. On a retrieval benchmark of 600 questions covering all 185 lessons, the system achieves Hit@1 of 0.8183, Hit@10 of 0.9833, and MRR@10 of 0.8856. On 246 generation-evaluation samples, it achieves Faithfulness of 0.9757 and Context Recall of 0.9593. These results show that the proposed approach is feasible for building a textbook-grounded learning assistant for Vietnamese high school Computer Science.

Danh mục ký hiệu và chữ viết tắt

Ký hiệu	Ý nghĩa tiếng Anh	Ý nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
A2A	Agent-to-Agent	Giao tiếp giữa các tác tử
API	Application Programming Interface	Giao diện lập trình ứng dụng
BM25	Best Matching 25	Thuật toán xếp hạng truy xuất dựa trên từ khóa
CD		Bộ sách Cánh Diều
HITL	Human-in-the-Loop	Cơ chế có con người trong vòng lặp
IDF	Inverse Document Frequency	Tần suất nghịch đảo tài liệu
JSON	JavaScript Object Notation	Định dạng dữ liệu văn bản có cấu trúc
KNTT		Bộ sách Kết Nối Tri Thức
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MRR	Mean Reciprocal Rank	Trung bình nghịch đảo thứ hạng
nDCG	Normalized Discounted Cumulative Gain	Độ lợi tích lũy có chuẩn hóa
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
OCR	Optical Character Recognition	Nhận dạng ký tự quang học
PDF	Portable Document Format	Định dạng tài liệu PDF
PPTX	PowerPoint Open XML Presentation	Định dạng tệp trình chiếu PowerPoint
Q&A	Question and Answer	Hỏi đáp
RAG	Retrieval-Augmented Generation	Tạo sinh tăng cường bằng truy xuất
RRF	Reciprocal Rank Fusion	Hợp nhất thứ hạng nghịch đảo
SGK		Sách giáo khoa
TF	Term Frequency	Tần suất xuất hiện của từ
TF-IDF	Term Frequency–Inverse Document Frequency	Trọng số kết hợp tần suất từ và tần suất nghịch đảo tài liệu
UI	User Interface	Giao diện người dùng

Danh mục thuật ngữ chuyên ngành

Thuật ngữ	Giải thích ý nghĩa
Agent	Tác tử phần mềm có khả năng tiếp nhận mục tiêu, sử dụng ngữ cảnh và công cụ để thực hiện một nhiệm vụ xác định trong hệ thống.
Adaptive retrieval	Cơ chế lựa chọn chiến lược truy xuất theo đặc trưng truy vấn thay vì dùng một quy trình retrieval cố định cho mọi câu hỏi.
Artifact	Đầu ra trung gian hoặc đầu ra cuối của một bước xử lý, chẳng hạn dàn ý, nội dung chi tiết, câu hỏi đánh giá hoặc tệp trình chiếu.
Breadcrumb	Chuỗi thông tin phân cấp ngắn mô tả vị trí của một chunk trong cấu trúc SGK, thường theo dạng chủ đề, bài học và mục.
Chunk	Đơn vị văn bản được tách ra từ tài liệu gốc để phục vụ lập chỉ mục, truy xuất và dựng ngữ cảnh cho mô hình.
Chunking	Quá trình chia tài liệu thành các chunk có kích thước và ranh giới phù hợp với mục tiêu retrieval và generation.
Context window	Giới hạn số lượng token mà mô hình ngôn ngữ có thể tiếp nhận trong một lượt suy luận.
Cross-encoder reranker	Mô hình xếp hạng lại đánh giá trực tiếp cặp truy vấn–tài liệu để sắp xếp lại danh sách candidate sau bước truy xuất ban đầu.
Embedding	Biểu diễn vector số của văn bản trong không gian ngữ nghĩa, cho phép so sánh mức độ tương đồng giữa các câu hỏi và tài liệu.
Grounding	Cơ chế ràng buộc đầu ra của mô hình ngôn ngữ vào tập nguồn đã truy xuất nhằm giảm nguy cơ sinh thông tin không có căn cứ.
Hallucination	Hiện tượng mô hình ngôn ngữ sinh ra thông tin có vẻ hợp lý nhưng sai sự thật, sai ngữ cảnh hoặc không được hỗ trợ bởi nguồn dữ liệu.

Hybrid retrieval	Cơ chế truy xuất kết hợp <i>keyword search</i> và <i>vector search</i> để tận dụng đồng thời tín hiệu từ khóa và tín hiệu ngữ nghĩa.
Keyword search	Phương pháp truy xuất dựa trên mức độ trùng khớp từ khóa giữa truy vấn và tài liệu, điển hình là BM25.
Lesson key	Định danh logic của một bài học dùng để đối chiếu trong benchmark retrieval, thường được tạo từ các trường metadata như bộ sách, lớp, chủ đề và bài học.
Metadata	Tập thông tin mô tả đi kèm mỗi chunk, ví dụ bộ sách, lớp, chủ đề, bài học, cấp heading hoặc loại nội dung.
Multi-agent	Kiến trúc nhiều tác tử phối hợp với nhau để giải quyết một tác vụ phức tạp thông qua phân chia vai trò.
Multi-intent	Trường hợp một yêu cầu đầu vào đồng thời chứa nhiều ý định nghiệp vụ, chẳng hạn vừa hỏi đáp vừa yêu cầu sinh bài tập.
Multi-turn conversation	Hội thoại nhiều lượt, trong đó mỗi lượt mới có thể phụ thuộc vào lịch sử tương tác và trạng thái đã tích lũy trước đó.
Prompt	Chỉ thị đầu vào cung cấp cho mô hình ngôn ngữ hoặc mô hình thị giác-ngôn ngữ để điều khiển cách xử lý và định dạng đầu ra.
Query rewriting	Bước viết lại truy vấn nhằm làm rõ ngữ cảnh, chuẩn hóa phạm vi hoặc tăng khả năng truy xuất đúng tài liệu liên quan.
Reflection loop	Vòng phản tư trong đó đầu ra được hậu kiểm, sau đó hệ thống có thể yêu cầu sửa đổi một phần thay vì chạy lại toàn bộ pipeline.
Retriever	Thành phần thực hiện truy xuất candidate ban đầu từ kho tri thức trước khi sang bước reranking hoặc generation.
Session state	Trạng thái lưu theo phiên hội thoại, bao gồm lịch sử trao đổi, trạng thái quiz, trạng thái slide và các thông tin cần cho đa lượt tương tác.
Student profile	Hồ sơ trạng thái học tập của người học, lưu các tín hiệu như kết quả làm quiz, câu sai và tiến độ theo chủ đề để hỗ trợ theo dõi hoặc cá nhân hóa.

Supervisor– Specialist	Mô hình điều phối trong hệ đa tác tử, trong đó một supervisor phân công và hợp nhất kết quả từ các specialist agent chuyên trách từng nhiệm vụ con.
Token	Đơn vị biểu diễn nhỏ mà mô hình dùng để mã hóa và xử lý văn bản.
Vector database	Cơ sở dữ liệu chuyên dụng cho lưu trữ và truy vấn các vector embedding ở quy mô lớn.
Vector search	Phương pháp truy xuất dựa trên độ tương đồng giữa embedding của truy vấn và embedding của tài liệu trong không gian vector.
Workflow state	Trạng thái dùng chung trong một pipeline xử lý nhiều bước, đặc biệt quan trọng trong LangGraph và các workflow đa tác tử.

MỤC LỤC

Danh mục ký hiệu và chữ viết tắt.....	i
Danh mục thuật ngữ chuyên ngành	ii
DANH MỤC HÌNH VẼ	ix
DANH MỤC BẢNG BIỂU.....	x
DANH MỤC CÔNG THỨC.....	xi
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT, KHẢO SÁT VÀ ĐỀ XUẤT GIẢI PHÁP.....	5
2.1 Phạm vi nghiên cứu	5
2.2 Tổng quan về bài toán	6
2.2.1 Bài toán chatbot giáo dục.....	6
2.2.2 Hệ thống hỏi đáp dựa trên tài liệu	7
2.2.3 Thách thức đặc thù của dữ liệu SGK Tin học	7
2.3 Khảo sát các giải pháp liên quan	8
2.3.1 Các nhóm giải pháp hiện có	8
2.3.2 Bảng so sánh các nhóm giải pháp	9
2.3.3 Nhận xét và định vị giải pháp đồ án	10
2.4 Cơ sở lý thuyết và hướng tiếp cận.....	10
2.4.1 Mô hình ngôn ngữ lớn và hạn chế khi dùng trực tiếp	10
2.4.2 Embedding và tìm kiếm ngữ nghĩa.....	11

2.4.3 Retrieval-Augmented Generation.....	11
2.4.4 Adaptive retrieval, reranking và query rewriting.....	12
2.4.5 Multi-intent và điều phối yêu cầu học tập	12
2.4.6 Đánh giá hệ thống RAG.....	13
2.4.7 Khoảng trống và hướng tiếp cận của đề án	13
2.5 Kết chương.....	13
CHƯƠNG 3. THIẾT KẾ GIẢI PHÁP VÀ PHƯƠNG PHÁP ĐÁNH GIÁ.	15
3.1 Tổng quan kiến trúc hệ thống.....	15
3.1.1 Bài toán và yêu cầu	15
3.1.2 Kiến trúc tổng thể và luồng xử lý một yêu cầu	16
3.1.3 Công nghệ nền.....	17
3.2 Nền tảng tri thức: tiền xử lý, chunking và lập chỉ mục	17
3.2.1 Nguồn SGK, cấu trúc phân cấp và lược đồ metadata.....	19
3.2.2 Chunking theo tiêu đề.....	20
3.2.3 Biểu diễn vector tiếng Việt và lưu trữ Qdrant.....	22
3.3 Cơ chế truy hồi ngữ cảnh (RAG core).....	23
3.3.1 Truy hồi lai BM25 và dense, hợp nhất bằng Reciprocal Rank Fusion.....	23
3.3.2 Xếp hạng lại bằng cross-encoder	24
3.3.3 Truy hồi thích ứng (Adaptive RAG)	25
3.4 Tầng điều phối tác tử.....	27
3.4.1 Sự cần thiết của tầng điều phối tác tử	27
3.4.2 Định tuyến ý định bằng mô hình ngôn ngữ lớn.....	27
3.4.3 Làm giàu ngữ cảnh hội thoại và viết lại truy vấn	29
3.4.4 Quản lý trạng thái hội thoại và vòng đời session.....	29
3.4.5 Lập kế hoạch hành động và thực thi đa hành động.....	31

3.5 Tầng sinh nội dung đa tác tử	33
3.5.1 Mô hình Supervisor–Specialist và lý do lựa chọn	33
3.5.2 Giao thức giao tiếp và cơ chế điều phối giữa các tác tử	33
3.5.3 Các tác tử chuyên biệt trong pipeline sinh nội dung	35
3.5.4 Tổng hợp kết quả, kiểm soát chất lượng và con người trong vòng lặp	36
3.5.5 Tái sử dụng kiến trúc cho các tác vụ sinh nội dung	37
3.6 Phương pháp đánh giá	38
3.6.1 Đánh giá truy hồi	38
3.6.2 Đánh giá chất lượng sinh	39
3.7 Kết chương.....	40
CHƯƠNG 4. CÀI ĐẶT, KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM.....	42
4.1 Môi trường và cấu hình thực nghiệm	42
4.2 Cài đặt hệ thống và pipeline.....	44
4.3 Xây dựng benchmark cho retrieve node	46
4.4 Xây dựng dữ liệu cho generate node	49
4.5 Kiểm thử chức năng	50
4.6 Phương pháp tiến hành thí nghiệm	51
4.7 Kết quả đánh giá retrieve node	52
4.8 Kết quả đánh giá generate node theo RAGAS	54
4.9 Phân tích lỗi	56
4.10 Nhận xét tổng hợp kết quả thực nghiệm	56
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	58
5.1 Xây dựng nền tảng tri thức dựa trên chunk phân cấp và metadata	58
5.1.1 Vấn đề cần giải quyết.....	58
5.1.2 Giải pháp	58

5.1.3 Đóng góp đạt được	59
5.2 Kết hợp hybrid retrieval, reranking và adaptive retrieval	59
5.2.1 Vấn đề cần giải quyết.....	59
5.2.2 Giải pháp	60
5.2.3 Đóng góp đạt được	60
5.3 Thiết kế tầng điều phối cho multi-intent và multi-turn conversation.....	60
5.3.1 Vấn đề cần giải quyết.....	60
5.3.2 Giải pháp	61
5.3.3 Đóng góp đạt được	61
5.4 Triển khai kiến trúc supervisor–specialist trong sinh nội dung.....	61
5.4.1 Vấn đề cần giải quyết.....	61
5.4.2 Giải pháp	61
5.4.3 Đóng góp đạt được	62
5.5 Đóng góp thiết kế trong việc nâng cao tính kiểm soát, khả năng mở rộng và tính thực dụng của hệ thống	62
5.5.1 Vấn đề cần giải quyết.....	62
5.5.2 Giải pháp	62
5.5.3 Đóng góp đạt được	63
5.6 Kết chương.....	63
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	64
6.1 Kết luận.....	64
6.2 Hướng phát triển.....	65
TÀI LIỆU THAM KHẢO.....	69

DANH MỤC HÌNH VẼ

Hình 3.1	Kiến trúc tổng thể của phương pháp đề xuất theo hướng phân tầng, trong đó lõi truy xuất tri thức là nền tảng chung cho các nhánh xử lý	16
Hình 3.2	Quy trình từ văn bản SGK đến tập chunk được lập chỉ mục, với chunking theo tiêu đề là bước trung tâm	18
Hình 3.3	Cơ chế truy hồi thích ứng của hệ thống với bộ phân loại truy vấn và các chiến lược truy xuất tương ứng	26
Hình 3.4	Khối phân tích ngữ cảnh và định tuyến ý định trong tầng điều phối	28
Hình 3.5	Cơ chế lập kế hoạch hành động ở tầng điều phối	31
Hình 3.6	Vòng thực thi đa hành động trong tầng điều phối theo chu trình quan sát – quyết định – hành động	32
Hình 3.7	Kiến trúc đa tác tử cho bài toán sinh slide và giáo án	34
Hình 3.8	Hai nhánh sinh nội dung và cơ chế hậu kiểm chất lượng của hệ thống	37
Hình 4.1	Lược đồ một câu hỏi trong benchmark retrieval và các trường phục vụ truy vết nguồn gốc	48
Hình 4.2	Hit@ k theo k của hai cấu hình trên toàn bộ 600 câu hỏi	53
Hình 4.3	So sánh Hit@1 giữa hai cấu hình theo từng nhóm bộ sách và khối lớp	54

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các nhóm giải pháp cho bài toán chatbot giáo dục dựa trên SGK	9
Bảng 3.1	Phân bố số lượng chunk theo bộ sách và khối lớp	19
Bảng 3.2	Lược đồ metadata của một chunk	20
Bảng 3.3	Phân bố số lượng chunk theo loại nội dung sư phạm (type)	20
Bảng 3.4	Ví dụ minh họa cách chunking theo tiêu đề trên một tài liệu ba tầng heading	21
Bảng 3.5	Các độ đo dùng cho đánh giá retrieval trong đồ án	39
Bảng 3.6	Các độ đo dùng cho đánh giá chất lượng sinh trong đồ án	40
Bảng 4.1	Môi trường phần mềm và các thành phần cốt lõi của hệ thống thực nghiệm	43
Bảng 4.2	Các tham số chạy chính dùng cho thực nghiệm retrieval và generation	43
Bảng 4.3	Phân bố bộ benchmark retrieval theo bộ sách và khối lớp	48
Bảng 4.4	Phân bố bộ benchmark retrieval theo loại câu hỏi và độ khó	49
Bảng 4.5	Các ca kiểm thử chức năng chính của hệ thống trước khi chạy thực nghiệm	50
Bảng 4.6	So sánh kết quả retrieval tổng thể giữa cấu hình no-rerank và with-rerank trên 600 câu hỏi	52
Bảng 4.7	So sánh Hit@1 và MRR@10 theo từng nhóm bộ sách và khối lớp	53
Bảng 4.8	Bốn độ đo RAGAS dùng để đánh giá generate node	55
Bảng 4.9	Kết quả đánh giá generate node theo RAGAS trên 246 mẫu	55

DANH MỤC CÔNG THỨC

Công thức	Nội dung	Trang
(3.1)	Tần suất từ (<i>term frequency</i>)	23
(3.2)	Tần suất nghịch đảo tài liệu (<i>inverse document frequency</i>)	23
(3.3)	Hàm tính điểm BM25 cho truy vấn và chunk	23
(3.4)	Độ tương đồng cosine giữa hai vector embedding	24
(3.5)	Hàm hợp nhất thứ hạng Reciprocal Rank Fusion (RRF)	24
(3.6)	Hàm chấm điểm của cross-encoder reranker	25

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh cuộc cách mạng công nghiệp lần thứ tư đang diễn ra mạnh mẽ trên toàn cầu, chuyển đổi số đã trở thành một trong những ưu tiên hàng đầu của Việt Nam trong chiến lược phát triển quốc gia. Giáo dục được xác định là một trong những lĩnh vực trọng điểm trong quá trình chuyển đổi số, việc ứng dụng công nghệ thông tin và trí tuệ nhân tạo (AI) vào dạy và học không chỉ là xu hướng tất yếu mà còn là yêu cầu cấp thiết để nâng cao chất lượng giáo dục.

Môn Tin học tại trường trung học phổ thông (THPT) đóng vai trò đặc biệt quan trọng trong chương trình giáo dục Việt Nam. Với chương trình mới bao gồm hai bộ sách giáo khoa Cánh Diều và Kết Nối Tri Thức với Cuộc Sống cho các lớp 10, 11 và 12, nội dung bao phủ nhiều chủ đề chuyên môn từ khoa học máy tính, mạng máy tính đến an toàn thông tin. Tuy nhiên, việc giảng dạy và học tập môn học này vẫn đối mặt với nhiều thách thức đáng kể.

Về phía học sinh, các em thường thiếu một công cụ hỗ trợ tự học hiệu quả và bám sát nội dung sách giáo khoa. Các chatbot AI phổ quát hiện nay như ChatGPT, Claude hay Gemini tuy có khả năng trả lời câu hỏi nhưng không được tối ưu cho dữ liệu sách giáo khoa Việt Nam, dẫn đến tình trạng đưa ra thông tin không chính xác (hallucination). Điều này đặc biệt nguy hiểm trong bối cảnh giáo dục khi mà việc tiếp thu kiến thức sai lệch có thể ảnh hưởng nghiêm trọng đến quá trình học tập. Bên cạnh đó, học sinh cũng thiếu nguồn bài tập đa dạng và phản hồi tức thời được thiết kế chuyên biệt theo từng bài học.

Về phía giáo viên, việc biên soạn tài liệu giảng dạy như câu hỏi trắc nghiệm, bài tập hay cấu trúc slide bài giảng đòi hỏi rất nhiều thời gian và công sức. Trong khi đó, hầu hết các nền tảng e-learning hiện có tại Việt Nam chỉ đóng vai trò là kho lưu trữ và phân phối bài giảng số, chưa có khả năng tự động khởi tạo nội dung thông minh dựa trên yêu cầu của người dùng.

Từ những phân tích trên, nhu cầu xây dựng một hệ thống trợ lý ảo giáo dục ứng dụng công nghệ Tăng cường truy xuất dữ liệu (RAG) sử dụng dữ liệu nền tảng là sách giáo khoa trở nên cấp thiết. Hệ thống này cần đáp ứng đầy đủ các mục tiêu bao gồm tự động trả lời câu hỏi dựa trên nội dung SGK, sinh bài tập đa dạng định dạng, phát thảo cấu trúc slide bài giảng và hỗ trợ giáo viên trong công tác soạn giảng, qua đó tối ưu hóa công việc của cả người dạy lẫn người học.

1.2 Mục tiêu và phạm vi đề tài

Trong những năm gần đây, việc ứng dụng trí tuệ nhân tạo vào giáo dục đã thu hút sự quan tâm của nhiều nhà nghiên cứu và doanh nghiệp trên thế giới. Các mô hình ngôn ngữ lớn (LLM) như ChatGPT, Claude hay Gemini đã chứng minh khả năng ấn tượng trong việc tạo sinh văn bản và trả lời câu hỏi. Tuy nhiên, khi được sử dụng trong bối cảnh giáo dục cụ thể, các mô hình này bộc lộ những hạn chế đáng kể. Đầu tiên, chúng không được huấn luyện trên dữ liệu sách giáo khoa Việt Nam, dẫn đến thường xuyên đưa ra câu trả lời không chính xác hoặc không phù hợp với nội dung chương trình học. Thứ hai, hiện tượng hallucination - tức là sinh ra thông tin nhìn có vẻ đáng tin cậy nhưng hoàn toàn sai sự thật - là vấn đề nan giải mà các LLM hiện tại vẫn chưa thể khắc phục triệt để.

Đối với các nền tảng e-learning trong nước như VnEdu, HOC247, hay Moon, các hệ thống này chủ yếu cung cấp kho tài liệu số và video bài giảng được biên soạn sẵn. Một số nền tảng đã tích hợp tính năng quiz tự động nhưng chưa có khả năng sinh câu hỏi theo yêu cầu cụ thể của người dùng hoặc dựa trên nội dung bài học mới được cập nhật. Việc quản lý và cập nhật ngân hàng câu hỏi vẫn phải thực hiện thủ công, tốn kém nhiều thời gian và công sức.

Về mặt kỹ thuật, công nghệ Retrieval-Augmented Generation (RAG) đã được chứng minh hiệu quả trong việc cải thiện độ chính xác của LLM bằng cách kết hợp khả năng truy xuất tài liệu với khả năng sinh ngôn ngữ tự nhiên. Nhiều nghiên cứu đã áp dụng RAG vào các bài toán giáo dục và đạt được những kết quả khả quan. Tuy nhiên, phần lớn các ứng dụng này tập trung vào các ngữ cảnh sử dụng tiếng Anh và chưa có nhiều nghiên cứu cụ thể cho việc xây dựng hệ thống RAG dựa trên sách giáo khoa Việt Nam. Đặc biệt, việc kết hợp kiến trúc Multi-Agent với RAG để tạo ra hệ thống đa tác vụ (quiz + slide + scoring) vẫn còn là lĩnh vực tương đối mới và chưa được khai thác nhiều trong bối cảnh giáo dục Việt Nam.

So sánh giữa các giải pháp hiện có cho thấy rõ sự khác biệt về năng lực. Trong khi các chatbot AI phổ quát có khả năng đối thoại tự do nhưng không đảm bảo tính chính xác theo SGK, các nền tảng e-learning trong nước có nguồn dữ liệu chuẩn nhưng thiếu khả năng tự động sáng tạo nội dung. Đề tài này hướng tới việc kết hợp ưu điểm của cả hai hướng tiếp cận: đảm bảo tính bám sát sách giáo khoa đồng thời có khả năng sinh tự động các nội dung giáo dục đa dạng.

Dựa trên các phân tích và đánh giá ở trên, đề tài này xác định các hạn chế cần khắc phục bao gồm: thiếu công cụ AI tối ưu cho việc học Tin học THPT tại Việt Nam; chưa có giải pháp RAG nào tích hợp đầy đủ các tác vụ quiz, slide và scoring; và chưa có hệ thống nào kết hợp Multi-Agent với cơ chế Human-In-The-

Loop (HITL) để đảm bảo chất lượng đầu ra trong việc sinh slide bài giảng.

Mục tiêu tổng quát của đề tài là xây dựng hệ thống trợ lý ảo giáo dục hỗ trợ việc dạy và học môn Tin học tại trường trung học phổ thông, sử dụng nền tảng là sách giáo khoa Tin học lớp 10, 11 và 12 thuộc hai bộ sách Cánh Diều và Kết Nối Tri Thức.

Các mục tiêu cụ thể bao gồm: nghiên cứu và xây dựng hệ thống truy xuất kiến thức từ sách giáo khoa dựa trên kiến trúc RAG; thiết kế và phát triển module sinh câu hỏi và bài tập tự động với nhiều định dạng bao gồm trắc nghiệm, tự luận, điền khuyết và đúng/sai; xây dựng module sinh cấu trúc slide bài giảng với kiến trúc đa tác tử; phát triển module chấm điểm và phản hồi tự động dựa trên ngữ cảnh bài học; cuối cùng là đánh giá hiệu năng hệ thống thông qua các chỉ số đánh giá chuẩn.

1.3 Định hướng giải pháp

Để giải quyết các vấn đề đã xác định ở phần trên, đề tài này lựa chọn định hướng giải pháp dựa trên sự kết hợp giữa công nghệ Retrieval-Augmented Generation (RAG) và kiến trúc Multi-Agent. Đây là hai công nghệ tiên tiến trong lĩnh vực AI hiện nay, có khả năng bổ trợ lẫn nhau để tạo ra một hệ thống vừa đảm bảo tính chính xác vừa có khả năng xử lý đa tác vụ phức tạp.

Về nền tảng LLM, hệ thống sử dụng mô hình ngôn ngữ lớn của Google với ưu điểm là chi phí thấp, khả năng hỗ trợ tiếng Việt tốt và chế độ sinh output có cấu trúc ổn định. Mô hình này được đánh giá là phù hợp với mục tiêu của đề tài khi cần tối ưu hóa chi phí trong khi vẫn đảm bảo chất lượng đầu ra cần thiết.

Về thành phần truy xuất, thay vì phụ thuộc vào các thư viện đóng gói, đề tài tự xây dựng bộ máy truy xuất kết hợp giữa thuật toán BM25 và tìm kiếm ngữ nghĩa (Semantic Search). Hai luồng kết quả này được hợp nhất thông qua thuật toán Reciprocal Rank Fusion (RRF). Cách tiếp cận này mang lại sự chủ động trong việc tối ưu hóa thuật toán cho văn bản tiếng Việt và tránh phụ thuộc vào các thư viện có thể gây tốn kém tài nguyên không cần thiết.

Về kiến trúc điều phối, hệ thống thiết kế một Orchestrator tại lớp mức lệnh cơ sở thay vì sử dụng các framework có sẵn. Orchestrator này đóng vai trò là bộ xử lý mỏng, điều phối các thành phần xử lý bao gồm ContextAnalyzer, SessionManager, ActionPlanner và RAG Agent. Ưu điểm của cách thiết kế này là toàn bộ quy trình xử lý được tối ưu về tốc độ, đồng thời quá trình theo vết và debug trở nên dễ dàng hơn.

Về kiến trúc Multi-Agent, hệ thống áp dụng mô hình Supervisor với Supervisor Agent đóng vai trò điều phối, tiếp nhận yêu cầu và phân loại đến các sub-agents

chuyên biệt bao gồm: OutlineAgent cho việc tạo dàn ý bài giảng, ContentAgent cho việc sinh nội dung chi tiết, MediaAgent cho việc tìm kiếm hình ảnh minh họa, QuizAgent cho việc sinh câu hỏi luyện tập, và MergeAgent cho việc tổng hợp kết quả. Hệ thống còn tích hợp cơ chế Human-In-The-Loop (HITL) cho phép người dùng duyệt và chỉnh sửa outline trước khi tiến hành sinh nội dung chi tiết.

Về đóng góp chính, đề tài này tự xây dựng bộ máy truy xuất từ mức cơ sở, thiết kế kiến trúc Adaptive RAG với nhiều chiến lược truy xuất khác nhau, xây dựng hệ thống Multi-Agent cho sinh slide bài giảng với các sub-agents chuyên biệt, và tích hợp cơ chế HITL trong quy trình sinh slide.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày phạm vi nghiên cứu, tổng quan bài toán, khảo sát các giải pháp liên quan và cơ sở lý thuyết cần thiết cho đề tài. Chương này làm rõ vì sao bài toán hỏi đáp trên SGK Tin học THPT cần một hệ thống RAG có khả năng kiểm soát phạm vi dữ liệu, truy xuất đúng ngữ cảnh và đánh giá riêng các thành phần trong pipeline.

Chương 3 trình bày phương pháp đề xuất. Nội dung chương bao gồm kiến trúc tổng thể, nền tảng tri thức, cơ chế truy hồi ngữ cảnh, tầng điều phối tác tử, tầng sinh nội dung đa tác tử và phương pháp đánh giá. Đây là chương mô tả trực tiếp cách hệ thống xử lý dữ liệu SGK, thực hiện retrieval, reranking, generation và tổ chức các tác vụ sinh học liệu.

Chương 4 trình bày quá trình cài đặt, kiểm thử và kết quả thực nghiệm. Chương này mô tả môi trường thực nghiệm, dữ liệu đánh giá, các kịch bản kiểm thử, kết quả retrieval trước và sau rerank, kết quả đánh giá generation, độ trễ hệ thống và các phân tích lỗi quan trọng.

Chương 5 trình bày các giải pháp và đóng góp nổi bật của đồ án. Các đóng góp chính gồm tổ chức nền tảng tri thức từ SGK, thiết kế RAG core có truy hồi lai và adaptive retrieval, xây dựng tầng điều phối yêu cầu học tập, triển khai workflow multi-agent cho tác vụ sinh nội dung và tích hợp các cơ chế kiểm soát chất lượng đầu ra.

Cuối cùng, Chương 6 tổng kết kết quả đạt được, nêu các hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT, KHẢO SÁT VÀ ĐỀ XUẤT GIẢI PHÁP

Chương 1 đã trình bày bối cảnh, mục tiêu và định hướng tổng quát của đề tài. Trên cơ sở đó, chương này tập trung làm rõ phạm vi nghiên cứu, ngữ cảnh bài toán, các hướng giải pháp liên quan và những nền tảng kỹ thuật cần thiết để hiểu phương pháp được đề xuất ở chương sau. Trọng tâm của chương không phải là mô tả các chức năng phần mềm, mà là xây dựng lập luận nghiên cứu: vì sao bài toán hỏi đáp trên SGK Tin học THPT cần một hệ thống RAG có khả năng truy xuất theo phạm vi, xếp hạng lại ngữ cảnh và đánh giá riêng từng thành phần trong pipeline.

Nội dung chương được tổ chức thành bốn phần. Mục đầu tiên xác định phạm vi nghiên cứu và các giới hạn của đề án. Mục thứ hai trình bày tổng quan bài toán hỏi đáp giáo dục dựa trên tài liệu. Mục thứ ba khảo sát các nhóm giải pháp liên quan và chỉ ra khoảng trống còn tồn tại. Mục cuối cùng trình bày các kiến thức nền tảng và hướng tiếp cận mà đề án lựa chọn.

2.1 Phạm vi nghiên cứu

Đối tượng nghiên cứu của đề án là hệ thống hỏi đáp dựa trên tài liệu (document-grounded question answering) cho môn Tin học THPT. Hệ thống tiếp nhận câu hỏi tự nhiên của người dùng, truy xuất ngữ cảnh liên quan từ SGK, sau đó sử dụng LLM để sinh câu trả lời dựa trên phần ngữ cảnh đã được truy xuất. Vì vậy, bài toán không được xem là một chatbot hội thoại mở thông thường, mà là một hệ thống hỏi đáp có ràng buộc nguồn tri thức rõ ràng.

Phạm vi dữ liệu của đề tài gồm SGK Tin học lớp 10, 11 và 12 thuộc hai bộ sách Cánh Diều (CD) và Kết Nối Tri Thức (KNTT). Đây là nguồn dữ liệu có cấu trúc phân cấp theo bộ sách, lớp, chủ đề, bài học và mục nội dung. Cấu trúc này có ý nghĩa quan trọng đối với bài toán retrieval, vì nhiều khái niệm Tin học có thể xuất hiện ở nhiều lớp hoặc nhiều bộ sách với mức độ trình bày khác nhau. Nếu hệ thống chỉ tìm kiếm theo toàn bộ kho dữ liệu mà không xét phạm vi, câu trả lời có thể lấy nhầm nội dung thuộc lớp hoặc bộ sách khác.

Đối tượng sử dụng chính của hệ thống gồm học sinh, giáo viên và người tự học môn Tin học THPT. Với học sinh, hệ thống cần hỗ trợ đặt câu hỏi tự nhiên, giải thích khái niệm, ôn tập nội dung bài học và định hướng luyện tập. Với giáo viên, hệ thống có thể hỗ trợ chuẩn bị học liệu như dàn ý slide, giáo án hoặc câu hỏi kiểm tra. Tuy nhiên, các tác vụ sinh học liệu này đều phụ thuộc vào một nền tảng chung là khả năng truy xuất đúng ngữ cảnh từ SGK trước khi sinh nội dung.

Trọng tâm nghiên cứu của đề án nằm ở pipeline retrieval và generation trong

kiến trúc Retrieval-Augmented Generation (RAG). Thành phần retrieval được đánh giá định lượng thông qua benchmark truy hồi, trong đó một câu hỏi kiểm thử gắn với một nguồn đúng kỳ vọng. Thành phần generation được đánh giá theo hướng chất lượng đầu ra và mức độ bám nguồn, ưu tiên rubric thủ công hoặc kết quả đánh giá đã có sẵn thay vì mở rộng chi phí đánh giá tự động bằng LLM. Các thành phần như giao diện, API, quản lý phiên, triển khai hạ tầng và đóng gói phần mềm được trình bày ở mức cần thiết để chứng minh hệ thống có thể vận hành và kiểm thử, nhưng không phải trọng tâm nghiên cứu chính.

Trong phạm vi đề án, các tác vụ phụ như sinh slide, soạn giáo án hoặc tạo câu hỏi luyện tập được xem là các luồng ứng dụng trên cùng nền tảng RAG và multi-agent. Chúng có vai trò chứng minh khả năng mở rộng của kiến trúc, nhưng phần đánh giá định lượng sâu vẫn tập trung vào truy xuất ngữ cảnh và sinh câu trả lời bám tài liệu. Các hướng mở rộng như graph retrieval hoặc tối ưu DevOps không được xem là đóng góp chính nếu chưa có thực nghiệm đầy đủ.

Từ phạm vi trên, câu hỏi nghiên cứu chính của đề án có thể phát biểu như sau: làm thế nào để truy xuất đúng ngữ cảnh từ SGK Tin học THPT và sinh câu trả lời bám tài liệu, đồng thời đánh giá được chất lượng của từng bước trong pipeline RAG. Câu hỏi này dẫn trực tiếp đến các quyết định thiết kế ở Chương 3, đặc biệt là chunking theo cấu trúc SGK, truy hồi lai giữa keyword search và vector search, reranking, adaptive retrieval và benchmark retrieval theo single-gold.

2.2 Tổng quan về bài toán

2.2.1 Bài toán chatbot giáo dục

Việc ứng dụng AI trong giáo dục tạo ra nhu cầu về các hệ thống có khả năng tương tác tự nhiên với người học. Thay vì chỉ tra cứu tài liệu theo từ khóa, người học có xu hướng đặt câu hỏi bằng ngôn ngữ tự nhiên, yêu cầu giải thích lại khái niệm, đưa ví dụ hoặc hỗ trợ ôn tập theo một bài học cụ thể. Với môn Tin học THPT, nhu cầu này càng rõ ràng vì nội dung học bao gồm cả lý thuyết, thao tác thực hành, thuật ngữ chuyên ngành, quy trình xử lý dữ liệu và các khái niệm liên quan đến lập trình.

Một chatbot giáo dục dùng cho SGK cần đáp ứng đồng thời yêu cầu về ngôn ngữ tự nhiên và yêu cầu về tính chuẩn xác theo tài liệu. Nếu chỉ ưu tiên khả năng hội thoại, hệ thống có thể tạo câu trả lời mạch lạc nhưng không đúng phạm vi bài học. Ngược lại, nếu chỉ dựa vào tìm kiếm từ khóa, hệ thống khó xử lý các câu hỏi được diễn đạt lại hoặc có ngữ cảnh hội thoại. Do đó, bài toán đặt ra không chỉ là tạo một giao diện hỏi đáp, mà là thiết kế một pipeline có khả năng truy xuất nguồn phù hợp và sinh câu trả lời có căn cứ.

Dữ liệu SGK Tin học THPT có một số đặc điểm khác với tài liệu văn bản thông thường. Nội dung được tổ chức theo lớp, bộ sách, chủ đề, bài học và mục nhỏ. Một số chủ đề như dữ liệu, thuật toán, mạng máy tính hoặc đạo đức số có thể xuất hiện ở nhiều lớp nhưng với độ sâu khác nhau. Ngoài ra, mỗi bài học thường kết hợp nhiều loại nội dung như khái niệm, ví dụ, hoạt động, bài tập, ghi nhớ hoặc câu hỏi vận dụng. Những đặc điểm này khiến hệ thống cần nhận biết vai trò của từng đoạn tài liệu thay vì xem toàn bộ văn bản như một tập đoạn rời rạc không có cấu trúc.

2.2.2 Hệ thống hỏi đáp dựa trên tài liệu

Hệ thống hỏi đáp có thể được chia thành hai nhóm lớn: closed-book QA và open-book QA. Trong closed-book QA, mô hình trả lời chủ yếu dựa trên tri thức đã được học trong quá trình huấn luyện. Cách tiếp cận này phù hợp với tri thức phổ quát, nhưng không bảo đảm chính xác khi câu hỏi phụ thuộc vào một tài liệu cụ thể. Trong open-book QA hoặc document-grounded QA, hệ thống được cung cấp thêm tài liệu nguồn và câu trả lời phải dựa trên tài liệu đó.

Đồ án này thuộc nhóm document-grounded QA. Khi người dùng hỏi một khái niệm trong SGK Tin học, hệ thống không nên chỉ dựa vào tri thức tham số của LLM, vì tri thức này có thể không trùng với chương trình giáo dục Việt Nam hoặc có thể vượt khỏi phạm vi lớp học. Thay vào đó, hệ thống cần tìm các chunk liên quan trong SGK, đưa các chunk này vào context, rồi yêu cầu LLM sinh câu trả lời dựa trên context được cung cấp.

Luồng xử lý tổng quát của một hệ thống hỏi đáp dựa trên tài liệu gồm tiếp nhận câu hỏi, phân tích ý định hoặc phạm vi, truy xuất tài liệu liên quan, xây dựng context, sinh câu trả lời và trả kết quả cho người dùng. Trong luồng này, retrieval là bước có ảnh hưởng trực tiếp đến chất lượng đầu ra. Nếu context bị thiếu hoặc sai nguồn, LLM có thể sinh câu trả lời không đầy đủ hoặc không đúng dù prompt được thiết kế tốt. Vì vậy, đánh giá riêng retrieval là yêu cầu quan trọng khi nghiên cứu một hệ thống RAG.

2.2.3 Thách thức đặc thù của dữ liệu SGK Tin học

Thách thức đầu tiên đến từ cấu trúc phân cấp của SGK. Cùng một thuật ngữ có thể xuất hiện trong nhiều bài hoặc nhiều lớp, nhưng ý nghĩa sư phạm và mức độ chi tiết không giống nhau. Ví dụ, một khái niệm liên quan đến dữ liệu có thể được trình bày ở lớp 10 như kiến thức nền, nhưng ở lớp 11 hoặc 12 lại gắn với bài toán xử lý dữ liệu hoặc cơ sở dữ liệu. Vì vậy, hệ thống cần lưu và sử dụng metadata về bộ sách, lớp, chủ đề và bài học trong quá trình retrieval.

Thách thức thứ hai là cách học sinh đặt câu hỏi thường không trùng với câu chữ trong SGK. Người học có thể hỏi bằng ngôn ngữ đời thường, dùng từ đồng nghĩa,

hỏi tiếp theo một lượt hội thoại trước đó hoặc bỏ qua thông tin về bài học đang xét. Điều này làm cho keyword search đơn thuần chưa đủ. Hệ thống cần kết hợp vector search, query rewriting hoặc cơ chế phân tích ngữ cảnh để tăng khả năng tìm đúng tài liệu.

Thách thức thứ ba là tính đa dạng của nội dung Tin học. Các câu hỏi về định nghĩa, quy trình, so sánh, ví dụ và vận dụng cần các loại context khác nhau. Câu hỏi định nghĩa thường cần đoạn mô tả khái niệm trực tiếp, trong khi câu hỏi quy trình có thể cần nhiều bước nằm trong cùng một mục. Nếu chunking làm mất mạch nội dung hoặc retrieval trả về các đoạn rời rạc, câu trả lời cuối cùng có thể thiếu tính sự phạm.

Những thách thức trên cho thấy yêu cầu cốt lõi của bài toán không chỉ là tích hợp LLM vào hệ thống học tập. Hệ thống cần một nền tảng tri thức được chuẩn hóa, cơ chế retrieval có xét phạm vi, bước reranking để cải thiện thứ tự candidate, và quy trình đánh giá có thể xác định lỗi đến từ retrieval hay generation.

2.3 Khảo sát các giải pháp liên quan

2.3.1 Các nhóm giải pháp hiện có

Các giải pháp liên quan đến bài toán trợ lý học tập có thể được chia thành một số nhóm chính. Nhóm thứ nhất là các chatbot rule-based hoặc hệ thống intent classification truyền thống. Các hệ thống này sử dụng luật, kịch bản hoặc bộ intent được định nghĩa trước để tạo phản hồi. Ưu điểm của nhóm này là dễ kiểm soát trong phạm vi nhỏ và ít xảy ra hallucination nếu câu hỏi nằm đúng kịch bản. Tuy nhiên, chúng kém linh hoạt khi người dùng diễn đạt khác với mẫu dự kiến, đồng thời chi phí mở rộng luật tăng nhanh khi số bài học và số dạng câu hỏi tăng lên.

Nhóm thứ hai là các hệ thống FAQ hoặc keyword search. Cách tiếp cận này phù hợp với các kho câu hỏi cố định hoặc tài liệu có từ khóa rõ ràng. Trong bối cảnh SGK, keyword search có thể xử lý tốt các câu hỏi chứa thuật ngữ chính xác, tên khái niệm hoặc cụm từ gần giống tài liệu. Hạn chế của nhóm này là khả năng hiểu câu hỏi tự nhiên còn thấp, đặc biệt khi học sinh diễn đạt lại bằng ngôn ngữ đời thường hoặc hỏi theo ngữ cảnh hội thoại.

Nhóm thứ ba là semantic search dựa trên embedding. Thay vì chỉ so khớp từ khóa, semantic search biểu diễn câu hỏi và đoạn tài liệu thành vector để tìm các đoạn gần nhau về mặt ngữ nghĩa. Cách tiếp cận này phù hợp với câu hỏi được diễn đạt khác với văn bản gốc. Tuy nhiên, nếu chỉ dùng vector search đơn lẻ, hệ thống vẫn có thể bỏ sót các thuật ngữ hiếm, tên lệnh hoặc ký hiệu đặc thù của Tin học. Ngoài ra, semantic search không tự giải quyết vấn đề sinh câu trả lời cuối cùng.

Nhóm thứ tư là các chatbot dùng LLM trực tiếp, chẳng hạn các nền tảng AI đa mục đích như ChatGPT, Gemini hoặc Claude. Ưu điểm của nhóm này là khả năng hội thoại tự nhiên, diễn giải linh hoạt và xử lý tốt nhiều yêu cầu phức tạp. Tuy nhiên, khi áp dụng trực tiếp vào SGK Tin học Việt Nam, các mô hình này thiếu ràng buộc về nguồn tài liệu, dễ trả lời vượt phạm vi chương trình hoặc sinh thông tin không có căn cứ trong SGK.

Nhóm thứ năm là các hệ thống RAG chatbot. RAG kết hợp retrieval với generation, cho phép hệ thống lấy tài liệu liên quan trước khi gọi LLM. Đây là hướng phù hợp hơn với dữ liệu SGK vì tri thức có thể được cập nhật bằng cách thay đổi kho tài liệu thay vì huấn luyện lại mô hình. Dù vậy, hiệu quả của RAG phụ thuộc mạnh vào chất lượng chunking, embedding, retrieval, reranking và cách xây dựng context.

Nhóm thứ sáu là hybrid hoặc adaptive RAG. Nhóm này mở rộng RAG cơ bản bằng cách kết hợp nhiều chiến lược tìm kiếm, lọc metadata theo phạm vi, viết lại truy vấn, truy xuất nhiều truy vấn và reranking. Cách tiếp cận này phù hợp với dữ liệu SGK có cấu trúc phân cấp, nhưng đi kèm độ phức tạp cao hơn về thiết kế, log và đánh giá.

2.3.2 Bảng so sánh các nhóm giải pháp

Để làm rõ mức độ phù hợp của từng nhóm giải pháp đối với bài toán chatbot giáo dục dựa trên SGK, Bảng 2.1 đối chiếu các nhóm giải pháp theo năm tiêu chí chính.

Nhóm giải pháp	Hiểu câu hỏi tự nhiên	Dùng dữ liệu riêng	Giảm hallucination	Phù hợp với SGK
Rule-based	Thấp	Trung bình	Cao nếu luật đủ	Thấp
FAQ/keyword search	Thấp–Trung bình	Có	Trung bình	Trung bình
Semantic search	Trung bình	Có	Phụ thuộc tài liệu trả về	Trung bình–Cao
LLM trực tiếp	Cao	Thấp nếu không nạp context	Thấp	Thấp–Trung bình
RAG	Cao	Cao	Trung bình–Cao	Cao
Hybrid/adaptive RAG	Cao	Cao	Cao hơn nếu retrieval tốt	Cao

Bảng 2.1: So sánh các nhóm giải pháp cho bài toán chatbot giáo dục dựa trên SGK

Bảng 2.1 cho thấy các nhóm giải pháp đơn giản như rule-based hoặc FAQ có ưu

điểm về khả năng kiểm soát trong phạm vi hẹp, nhưng khó mở rộng khi số lượng bài học và cách đặt câu hỏi tăng lên. LLM trực tiếp có khả năng xử lý ngôn ngữ tốt hơn, nhưng thiếu cơ chế gắn câu trả lời với dữ liệu SGK. RAG và hybrid/adaptive RAG phù hợp hơn với bài toán của đề án vì vừa tận dụng được năng lực sinh ngôn ngữ của LLM, vừa có thể kiểm soát nguồn tri thức thông qua retrieval.

2.3.3 Nhận xét và định vị giải pháp đề án

Từ khảo sát trên, có thể rút ra ba nhận xét chính. Thứ nhất, các hệ thống dựa trên luật hoặc FAQ chưa đáp ứng tốt nhu cầu hỏi đáp tự nhiên của học sinh. Người học thường không đặt câu hỏi theo đúng mẫu cố định, do đó hệ thống cần khả năng hiểu ngữ nghĩa và xử lý nhiều cách diễn đạt khác nhau. Thứ hai, LLM trực tiếp tuy mạnh về ngôn ngữ nhưng không bảo đảm câu trả lời bám SGK. Với giáo dục phổ thông, một câu trả lời nghe hợp lý nhưng vượt phạm vi hoặc sai nguồn có thể gây tác động tiêu cực đến quá trình học. Thứ ba, RAG là hướng tiếp cận phù hợp, nhưng cần được thiết kế và đánh giá cẩn thận vì lỗi retrieval có thể lan truyền sang bước generation.

Khoảng trống mà đề án hướng tới nằm ở giao điểm giữa ba yêu cầu: dữ liệu SGK tiếng Việt có cấu trúc phân cấp, câu hỏi tự nhiên từ người dùng, và nhu cầu đánh giá riêng từng node trong pipeline. Nhiều hệ thống hiện có chỉ đánh giá câu trả lời cuối cùng hoặc tập trung vào trải nghiệm sản phẩm, trong khi chưa phân tích rõ chất lượng của retrieval, reranking và generation. Đề án vì vậy định vị giải pháp theo hướng hybrid/adaptive RAG, trong đó hệ thống có thể truy xuất theo phạm vi sách, lớp, chủ đề, xếp hạng lại candidate và xây dựng benchmark riêng để đo chất lượng retrieval.

Đối với nhóm giải pháp thương mại đa mục đích, điểm khác biệt của đề án không nằm ở việc xây dựng một LLM mới, mà nằm ở cách tổ chức tri thức SGK và kiểm soát context đưa vào LLM. Đối với nhóm chatbot dịch vụ trong nước, điểm khác biệt nằm ở mục tiêu học thuật: hệ thống cần giải thích, truy xuất bài học, sinh học liệu và giữ ngữ cảnh sư phạm. Đối với các hệ thống nghiên cứu hoặc đề án đi trước, điểm khác biệt nằm ở việc tách retrieval và generation thành các thành phần có thể quan sát, kiểm thử và đánh giá riêng.

2.4 Cơ sở lý thuyết và hướng tiếp cận

2.4.1 Mô hình ngôn ngữ lớn và hạn chế khi dùng trực tiếp

LLM là mô hình học máy có khả năng sinh văn bản tự nhiên dựa trên ngữ cảnh đầu vào. Trong bài toán giáo dục, LLM có thể diễn giải khái niệm, tóm tắt nội dung, chuyển đổi yêu cầu thành câu trả lời có cấu trúc và hỗ trợ sinh học liệu. Tuy nhiên, năng lực ngôn ngữ của LLM không đồng nghĩa với việc mô hình luôn có

tri thức đúng theo SGK đang xét. Khi không được cung cấp context từ tài liệu, mô hình có thể trả lời dựa trên tri thức tham số đã học, trong đó có thể chứa thông tin ngoài chương trình, thông tin không cập nhật hoặc thông tin không phù hợp với cách diễn đạt của SGK Việt Nam.

Hạn chế này đặc biệt quan trọng trong môi trường giáo dục. Một câu trả lời sai hoặc vượt cấp không chỉ là lỗi kỹ thuật, mà còn có thể làm người học hiểu sai kiến thức. Ngoài ra, context window của LLM có giới hạn, nên không thể đưa toàn bộ SGK vào prompt cho mọi lượt hỏi. Vì vậy, đề án không dùng LLM như nguồn tri thức duy nhất, mà đặt LLM sau lớp retrieval để mô hình sinh câu trả lời dựa trên các chunk đã được truy xuất.

2.4.2 Embedding và tìm kiếm ngữ nghĩa

Embedding là kỹ thuật biểu diễn văn bản thành vector số trong không gian nhiều chiều. Khi hai đoạn văn có nội dung gần nhau, vector của chúng thường có độ tương đồng cao. Trong hệ thống hỏi đáp dựa trên SGK, embedding cho phép đưa câu hỏi của người dùng và các chunk tài liệu vào cùng một không gian biểu diễn, từ đó thực hiện vector search để tìm các đoạn có ý nghĩa gần với câu hỏi.

Tìm kiếm ngữ nghĩa có ưu điểm ở các truy vấn diễn đạt lại, dùng từ đồng nghĩa hoặc không trùng bề mặt với tài liệu. Tuy nhiên, vector search cũng có hạn chế khi gặp thuật ngữ hiếm, tên hàm, ký hiệu hoặc cụm từ kỹ thuật cần khớp chính xác. Vì lý do đó, hướng tiếp cận của đề án không chỉ dựa vào embedding đơn lẻ, mà kết hợp thêm keyword search, metadata filtering và reranking. Các công thức cụ thể cho BM25, cosine similarity và RRF được trình bày trong Chương 3 khi mô tả cơ chế truy hồi ngữ cảnh.

2.4.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation là kiến trúc kết hợp giữa retrieval và generation. Trong pha chuẩn bị dữ liệu, tài liệu nguồn được làm sạch, chia thành chunk, tạo embedding và lưu vào chỉ mục kèm metadata. Trong pha truy vấn, hệ thống nhận câu hỏi, truy xuất các chunk liên quan, có thể xếp hạng lại candidate, sau đó đưa context vào prompt để LLM sinh câu trả lời.

RAG phù hợp với bài toán SGK vì tri thức của hệ thống được quản lý thông qua kho tài liệu thay vì phụ thuộc hoàn toàn vào mô hình. Khi bổ sung hoặc cập nhật tài liệu, hệ thống có thể tái lập chỉ mục mà không cần huấn luyện lại LLM. Ngoài ra, các chunk được truy xuất có thể được ghi log, hiển thị hoặc dùng cho phân tích lỗi, giúp quá trình đánh giá minh bạch hơn so với việc chỉ quan sát câu trả lời cuối cùng.

Tuy nhiên, RAG không tự bảo đảm chất lượng đầu ra. Nếu tài liệu được chunking không hợp lý, metadata sai, retriever lấy nhầm bài hoặc context đưa vào prompt chứa nhiều nhiễu, LLM vẫn có thể sinh câu trả lời không đạt yêu cầu. Vì vậy, phương pháp của đồ án xem retrieval là thành phần cần đánh giá định lượng riêng, còn generation được đánh giá theo mức độ đúng, đủ, bám context và rõ ràng.

2.4.4 Adaptive retrieval, reranking và query rewriting

Một truy vấn người dùng có thể thiếu thông tin về phạm vi, phụ thuộc vào hội thoại trước đó hoặc dùng cách diễn đạt không trùng với tài liệu. Query rewriting giúp chuẩn hóa hoặc làm rõ truy vấn trước khi retrieval. Trong khi đó, adaptive retrieval chọn chiến lược truy xuất tùy theo loại câu hỏi, scope và tín hiệu ngữ cảnh, thay vì dùng một cấu hình cố định cho mọi trường hợp.

Reranking là bước đánh giá lại danh sách candidate sau truy xuất ban đầu. Retriever thường ưu tiên tốc độ và độ phủ, nên top-k ban đầu có thể chứa các chunk chưa thật sự phù hợp. Reranker, thường được triển khai bằng cross-encoder, nhận cặp query-document và ước lượng lại mức độ liên quan. Cách làm này có thể cải thiện thứ tự context đưa vào LLM, nhưng cũng làm tăng latency và chi phí tính toán. Vì vậy, trong Chương 4, kết quả thực nghiệm cần so sánh trước và sau rerank để xác định lợi ích thực tế của bước này.

Trong đồ án, adaptive retrieval, query rewriting và reranking được xem là các cơ chế hỗ trợ cho RAG core. Chúng không thay thế mục tiêu chính là truy xuất đúng context, mà giúp hệ thống xử lý tốt hơn các câu hỏi mơ hồ, câu hỏi có scope cụ thể và câu hỏi được diễn đạt khác với văn bản SGK.

2.4.5 Multi-intent và điều phối yêu cầu học tập

Người dùng của hệ thống không chỉ đặt câu hỏi kiến thức, mà còn có thể yêu cầu tạo quiz, sinh slide, soạn giáo án hoặc kết hợp nhiều yêu cầu trong cùng một lượt hỏi. Vì vậy, hệ thống cần một lớp điều phối để phân tích intent, xác định task type, lập kế hoạch hành động và gọi đúng service. Lớp điều phối này giúp biến yêu cầu tự nhiên thành các hành động có cấu trúc, nhưng không phải trọng tâm đánh giá định lượng của đồ án.

Vai trò của multi-intent routing trong nghiên cứu này là bảo đảm request được đưa vào đúng pipeline. Với câu hỏi kiến thức, hệ thống cần ưu tiên retrieval và answer generation. Với yêu cầu sinh học liệu, hệ thống cần sử dụng context từ SGK rồi chuyển sang workflow multi-agent để xây dựng dàn ý, nội dung, media hoặc câu hỏi đánh giá. Cách tổ chức này cho phép các tác vụ khác nhau dùng chung nền tảng tri thức, đồng thời giữ cho retrieval vẫn là thành phần trung tâm.

2.4.6 Đánh giá hệ thống RAG

Đánh giá hệ thống RAG cần tách hai câu hỏi: hệ thống có truy xuất đúng context không, và LLM có sinh câu trả lời tốt từ context đó không. Nếu chỉ đánh giá câu trả lời cuối cùng, rất khó xác định lỗi đến từ retriever hay generator. Một câu trả lời sai có thể do context sai, context thiếu, prompt chưa tốt hoặc LLM suy diễn quá mức. Ngược lại, một câu trả lời đúng không nhất thiết chứng minh retrieval tốt, vì LLM có thể dựa vào tri thức sẵn có để trả lời.

Đối với retrieval, đề án sử dụng benchmark có expected source rõ ràng để tính các chỉ số như Hit@k, Recall@k, Precision@k, MRR và nDCG@k. Các chỉ số này cho phép đo khả năng đưa đúng nguồn vào top-k, thứ hạng của nguồn đúng và mức độ tập trung của kết quả truy xuất. Đối với generation, hướng đánh giá phù hợp hơn là dùng rubric về độ đúng nội dung, độ đầy đủ, mức độ bám context, độ rõ ràng và khả năng tránh hallucination. Cách tách đánh giá này giúp quá trình phân tích lỗi có cơ sở hơn và phù hợp với mục tiêu nghiên cứu của đề án.

2.4.7 Khoảng trống và hướng tiếp cận của đề án

Từ các phân tích trên, có thể thấy khoảng trống chính của bài toán nằm ở việc xây dựng một hệ thống RAG cho dữ liệu SGK tiếng Việt có khả năng kiểm soát scope và đánh giá được từng thành phần. Các giải pháp hiện có hoặc quá cứng nhắc khi dựa vào luật và từ khóa, hoặc quá mở khi dùng LLM trực tiếp, hoặc chưa đánh giá rõ chất lượng retrieval trước khi kết luận về chất lượng câu trả lời.

Hướng tiếp cận của đề án là thiết kế một pipeline RAG cho SGK Tin học THPT, trong đó tài liệu được chunking theo cấu trúc bài học, mỗi chunk được gắn metadata phục vụ lọc scope, retrieval kết hợp keyword search và vector search, candidate được rerank trước khi đưa vào generation, và các bước xử lý được ghi log để phục vụ đánh giá. Trên nền tảng đó, hệ thống mở rộng sang các tác vụ sinh học liệu bằng lớp điều phối và workflow multi-agent, nhưng vẫn giữ nguyên nguyên tắc grounding trên nội dung SGK.

Phần tiếp theo của báo cáo sẽ trình bày chi tiết phương pháp đề xuất. Cụ thể, Chương 3 mô tả kiến trúc tổng thể, nền tảng tri thức, RAG core, tầng điều phối tác tử, tầng sinh nội dung đa tác tử và phương pháp đánh giá. Những nội dung nền tảng trong chương này là cơ sở để giải thích các quyết định thiết kế và thực nghiệm ở các chương sau.

2.5 Kết chương

Chương này đã xác định phạm vi nghiên cứu của đề án là hệ thống hỏi đáp dựa trên SGK Tin học THPT theo hướng document-grounded QA. Chương cũng đã

phân tích đặc thù của dữ liệu SGK, làm rõ các thách thức về scope, ngôn ngữ tự nhiên, thuật ngữ Tin học và vai trò của retrieval trong chất lượng câu trả lời.

Thông qua khảo sát các nhóm giải pháp liên quan, chương đã chỉ ra rằng RAG và hybrid/adaptive RAG là hướng tiếp cận phù hợp hơn so với rule-based, FAQ hoặc LLM trực tiếp khi yêu cầu câu trả lời phải bám sát SGK. Các kiến thức nền tảng về LLM, embedding, RAG, query rewriting, reranking, multi-intent và đánh giá hệ thống RAG tạo cơ sở cho Chương 3, nơi phương pháp đề xuất được trình bày chi tiết hơn về kiến trúc, các node xử lý và phương pháp đánh giá.

CHƯƠNG 3. THIẾT KẾ GIẢI PHÁP VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

Chương 2 đã trình bày cơ sở lý thuyết, khảo sát các hướng tiếp cận và xác định Retrieval-Augmented Generation là hướng tiếp cận phù hợp cho bài toán hỏi đáp trên SGK Tin học THPT. Trên cơ sở đó, chương này mô tả phương pháp được đề xuất, bao gồm kiến trúc tổng thể, các thành phần xử lý chính và cách tổ chức đánh giá cho các node trọng tâm trong pipeline.

Nội dung chương được trình bày theo hướng nghiên cứu, tập trung vào dữ liệu, retrieval, reranking, generation và evaluation, không đi sâu vào chi tiết cài đặt phần mềm.

3.1 Tổng quan kiến trúc hệ thống

3.1.1 Bài toán và yêu cầu

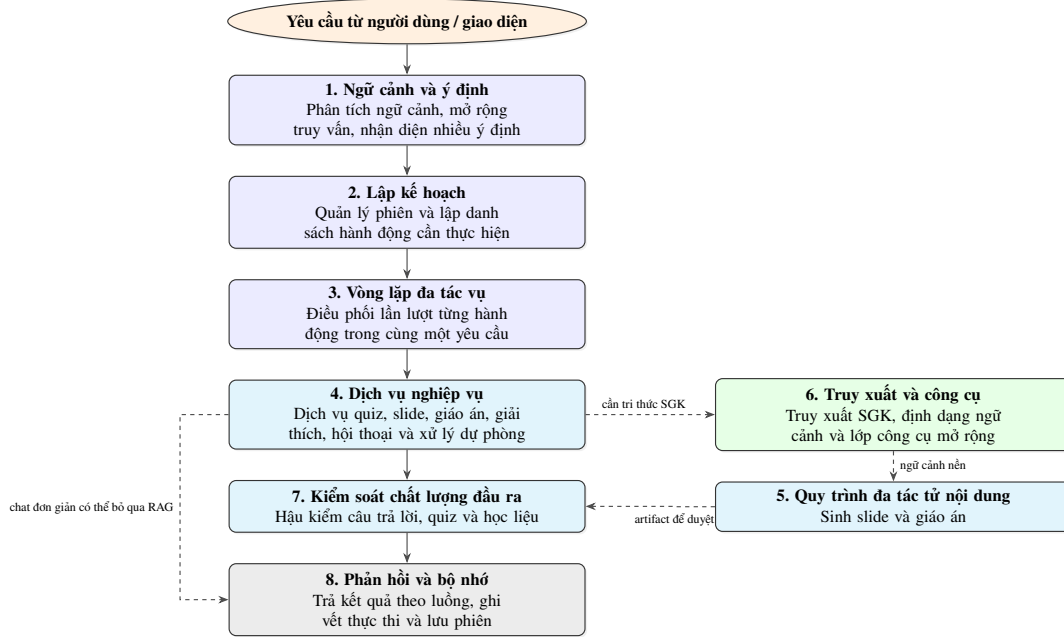
Bài toán của đề án là xây dựng một trợ lý giáo dục cho môn Tin học THPT, hỗ trợ người học và giáo viên khai thác nội dung từ sách giáo khoa trong môi trường tương tác số. Phạm vi dữ liệu của hệ thống được giới hạn vào sách giáo khoa Tin học lớp 10, 11 và 12 của hai bộ sách Cánh Diều và Kết Nối Tri Thức. Trên nền dữ liệu này, hệ thống không chỉ thực hiện tác vụ hỏi đáp kiến thức mà còn hỗ trợ các tác vụ sinh học liệu có cấu trúc, đòi hỏi khả năng tổng hợp và tổ chức lại nội dung.

Ba nhóm tác vụ chính của hệ thống gồm hỏi đáp dựa trên SGK, sinh slide bài giảng và soạn giáo án. Với tác vụ hỏi đáp, hệ thống cần truy xuất các đoạn ngữ cảnh liên quan thông qua *Retriever*, sau đó sử dụng *LLM* để sinh câu trả lời bám sát tài liệu nguồn. Với tác vụ sinh slide và giáo án, hệ thống cần bao phủ nội dung bài học ở mức tổng thể, đồng thời tổ chức lại tri thức theo trình tự sư phạm thay vì chỉ trả về các đoạn văn bản rời rạc. Do đó, bài toán không dừng ở một pipeline *Retrieval-Augmented Generation* (RAG) đơn giản, mà mở rộng thành bài toán điều phối nhiều bước xử lý với các yêu cầu khác nhau về truy xuất, tổng hợp và kiểm soát chất lượng.

Trên cơ sở đó, phương pháp đề xuất cần thỏa mãn một số yêu cầu chính. Thứ nhất, hệ thống phải khai thác tri thức từ SGK theo đúng phạm vi lớp, bộ sách và chủ đề nhằm hạn chế sai lệch ngữ cảnh. Thứ hai, cơ chế truy xuất phải xử lý được cả câu hỏi cụ thể lẫn các yêu cầu tổng hợp nội dung ở mức bài học. Thứ ba, phần sinh nội dung phải có cơ chế kiểm soát chất lượng đầu ra đối với các sản phẩm dài như slide và giáo án. Cuối cùng, kiến trúc hệ thống phải hỗ trợ đánh giá riêng chất lượng truy hồi của *Retriever/Reranker* và chất lượng sinh của *LLM*.

3.1.2 Kiến trúc tổng thể và luồng xử lý một yêu cầu

Kiến trúc của hệ thống được tổ chức theo hai tầng *Agent* đặt trên nền *Retrieval-Augmented Generation*. Tầng điều phối tác tử đảm nhiệm việc phân tích yêu cầu, lập kế hoạch và lựa chọn nhánh xử lý. Tầng sinh nội dung *Multi-Agent* đảm nhiệm việc tạo các sản phẩm dài như slide và giáo án. Hai tầng này cùng sử dụng một lõi truy xuất tri thức chung để bảo đảm đầu ra được grounding trên nội dung SGK.



Hình 3.1: Kiến trúc tổng thể của phương pháp đề xuất theo hướng phân tầng, trong đó lõi truy xuất tri thức là nền dùng chung cho các nhánh xử lý

Trong Hình 3.1, trục chính của vòng đời xử lý đi theo chuỗi từ yêu cầu đầu vào, phân tích ngữ cảnh và ý định, lập kế hoạch, điều phối đa tác vụ, thực thi dịch vụ nghiệp vụ, kiểm soát chất lượng đầu ra, rồi đến phản hồi và lưu trạng thái phiên. Cách biểu diễn này cho thấy bộ điều phối của hệ thống hoạt động theo một luồng xử lý chính, trong khi các năng lực truy xuất và sinh học liệu dài được gọi vào theo từng loại yêu cầu, thay vì xuất hiện như một bước tuần tự bắt buộc cho mọi truy vấn.

Ở bên phải của trục chính, khối “Truy xuất và công cụ” được đặt như một nền dùng chung cho các nhánh cần tri thức SGK. Từ khối “Dịch vụ nghiệp vụ”, hệ thống gọi xuống lõi truy xuất để lấy ngữ cảnh thông qua *Retriever*; nếu tác vụ là sinh slide hoặc giáo án thì ngữ cảnh này tiếp tục được đưa vào “Quy trình đa tác tử nội dung”. Bố cục này phản ánh quan hệ phụ thuộc giữa các thành phần: truy xuất tri thức diễn ra trước khi tầng *Multi-Agent* tổng hợp sản phẩm.

Phần cuối của sơ đồ cho thấy mọi đầu ra đều đi qua khối “Kiểm soát chất lượng đầu ra” trước khi trả về cho người dùng. Với câu trả lời ngắn, lớp này kiểm tra mức

độ phù hợp và bám sát ngữ cảnh; với quiz, slide và giáo án, lớp này thực hiện hậu kiểm trước khi công bố kết quả. Hai mũi tên nét đứt cũng thể hiện rằng chat đơn giản có thể bỏ qua lỗi RAG, trong khi các tác vụ cần grounding trên SGK sẽ kết nối đến nền truy xuất chung.

3.1.3 Công nghệ nền

Về mặt công nghệ, phương pháp được hiện thực hóa trên một số thành phần nền tảng tương ứng với từng vai trò trong kiến trúc. Mô hình ngôn ngữ lớn Gemini được sử dụng như *LLM* cho các bước cần suy luận ngôn ngữ như nhận diện ý định, viết lại truy vấn và sinh nội dung. LangGraph được sử dụng cho bộ điều phối nội dung ở tầng sinh học liệu, nơi quy trình có nhiều bước trung gian và có thể tạm dừng để con người tham gia duyệt. Ở tầng truy xuất, Qdrant được sử dụng như một *Vector Database* để lưu trữ và tìm kiếm ngữ nghĩa trên tập chunk đã được lập chỉ mục.

Đối với biểu diễn tri thức, hệ thống sử dụng mô hình *Embedding* tiếng Việt để chuyển mỗi khối nội dung SGK thành vector ngữ nghĩa, qua đó bảo toàn tốt hơn đặc trưng ngôn ngữ của dữ liệu trong không gian biểu diễn. Song song với đó, một thành phần tìm kiếm từ khóa kiểu BM25 được dùng để bổ sung tín hiệu lexical, sau đó kết quả được hợp nhất bằng *Reciprocal Rank Fusion*. Ở giai đoạn sau truy xuất, mô hình *Reranker* dạng cross-encoder được dùng để sắp xếp lại các ứng viên theo mức độ liên quan đối với truy vấn.

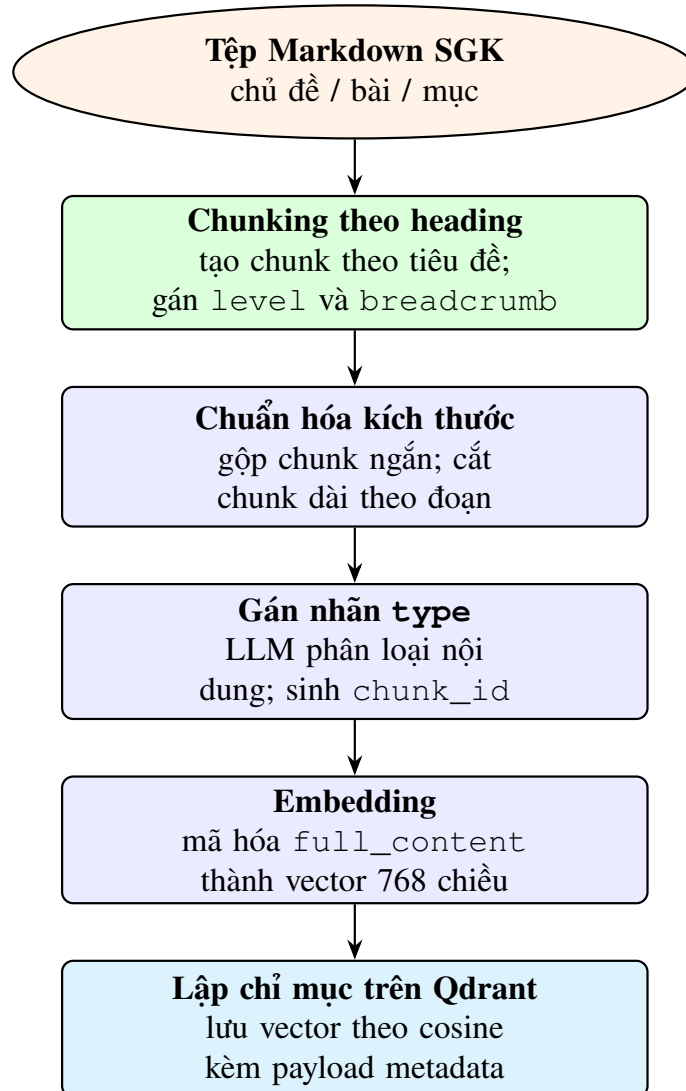
Trong phạm vi chương này, các công nghệ trên chỉ được trình bày ở mức cần thiết để làm rõ vai trò trong phương pháp đề xuất. Thông số cài đặt cụ thể của từng công nghệ sẽ được trình bày ở chương tiếp theo.

3.2 Nền tảng tri thức: tiền xử lý, chunking và lập chỉ mục

Mọi đầu ra của hệ thống đều được grounding trên nội dung sách giáo khoa, do đó chất lượng của tầng tri thức nền ảnh hưởng trực tiếp đến hiệu quả truy hồi và sinh nội dung. Phần này mô tả quy trình biến văn bản SGK thô thành một tập chunk có cấu trúc, được embedding và lập chỉ mục để phục vụ truy hồi. Trong toàn bộ pipeline này, bước chunking theo tiêu đề (heading-based chunking) giữ vai trò trung tâm vì nó quyết định ranh giới của đơn vị tri thức được đưa vào các bước xử lý sau.

Về bản chất, tầng tri thức của hệ thống không chỉ là một kho lưu trữ dữ liệu tĩnh mà là kết quả của một pipeline tiền xử lý nhiều bước. Mỗi bước trong pipeline này tác động trực tiếp đến chất lượng truy hồi ở giai đoạn sau. Nếu ranh giới chunk không phù hợp, mô hình embedding sẽ biểu diễn sai đơn vị kiến thức; nếu metadata không đầy đủ, tầng truy hồi sẽ khó giới hạn đúng phạm vi lớp, bộ sách và bài học; nếu chỉ mục vector không đồng bộ với dữ liệu nguồn, kết quả truy xuất có thể mất

tính ổn định giữa các lần chạy thực nghiệm. Vì vậy, việc mô tả rõ quy trình xây dựng tầng tri thức là cần thiết trước khi phân tích cơ chế retrieval ở mục kế tiếp.



Hình 3.2: Quy trình từ văn bản SGK đến tập chunk được lập chỉ mục, với chunking theo tiêu đề là bước trung tâm

Hình 3.2 mô tả quy trình một chiều biến tập bài học SGK thành các chunk sẵn sàng cho truy hồi, gồm năm bước nối tiếp. Đầu vào là tập Markdown của một bài học, trong đó cấu trúc phân cấp (chủ đề, tiêu đề bài, tiêu đề mục) đã được thể hiện qua các mốc tiêu đề. Bước trung tâm là *chunking theo heading*: một máy trạng thái quét tuần tự văn bản và chốt một chunk mới tại mỗi mốc tiêu đề, đồng thời gán trường `level` (độ sâu tiêu đề) và dựng chuỗi breadcrumb cho chunk đó. Tiếp theo, bước chuẩn hóa kích thước gộp các chunk quá ngắn và cắt các chunk quá dài để đưa độ dài về khoảng hợp lý mà không phá vỡ ranh giới ngữ nghĩa. Bước thứ tư phân loại vai trò sư phạm (`type`) của mỗi chunk bằng một mô hình ngôn ngữ nhẹ và sinh `chunk_id` bất định. Cuối cùng, mỗi chunk được embedding cùng

breadcrumb thành vector 768 chiều và được lập chỉ mục trên Qdrant kèm toàn bộ payload metadata. Trong các bước này, chunking theo tiêu đề giữ vai trò quyết định vì nó định hình ranh giới và tính mạch lạc của mọi chunk ở các bước sau; ba mục tiếp theo lần lượt trình bày chi tiết nguồn dữ liệu và metadata, cơ chế chunking, và khâu vector hóa – lưu trữ.

3.2.1 Nguồn SGK, cấu trúc phân cấp và lược đồ metadata

Dữ liệu nền của hệ thống là sách giáo khoa Tin học THPT của hai bộ Cánh Diều và Kết Nối Tri Thức, trải đều ba khối lớp 10, 11 và 12. Toàn bộ nội dung được số hóa về định dạng Markdown và tổ chức theo đúng cấu trúc phân cấp tự nhiên của sách: mỗi bộ sách (*book*) chia theo khối lớp (*grade*), mỗi lớp gồm nhiều chủ đề (*topic*), mỗi chủ đề gồm nhiều bài học (*lesson*), và mỗi bài học lại được chia thành các mục theo cấp độ tiêu đề. Cấu trúc thư mục phản ánh trực tiếp phân cấp này, trong đó mỗi tệp Markdown tương ứng với một bài học và tệp `README.md` ở mỗi thư mục chủ đề lưu mã cùng tên chủ đề.

Sau bước xử lý, tập tri thức thu được gồm 2.348 chunk, bao phủ 36 chủ đề và 186 bài học khác nhau. Bảng 3.1 trình bày phân bố số lượng chunk theo bộ sách và khối lớp.

Bộ sách	Lớp 10	Lớp 11	Lớp 12	Tổng
Cánh Diều (CD)	401	439	364	1.204
Kết Nối Tri Thức (KNTT)	401	350	393	1.144
Tổng	802	789	757	2.348

Bảng 3.1: Phân bố số lượng chunk theo bộ sách và khối lớp

Số liệu trong Bảng 3.1 cho thấy dữ liệu tương đối cân bằng giữa hai bộ sách, với 1.204 chunk thuộc bộ Cánh Diều và 1.144 chunk thuộc bộ Kết Nối Tri Thức. Phân bố theo khối lớp cũng không có chênh lệch lớn, lần lượt là 802 chunk ở lớp 10, 789 chunk ở lớp 11 và 757 chunk ở lớp 12. Phân bố này giúp giảm nguy cơ tăng truy hồi thiên lệch về một bộ sách hay một khối lớp cụ thể.

Mỗi chunk được gắn một lược đồ metadata thống nhất nhằm phục vụ lọc phạm vi khi truy hồi và truy vết nguồn khi sinh nội dung. Bảng 3.2 mô tả các trường chính của lược đồ này.

Bảng 3.2 cho thấy mỗi chunk không chỉ lưu nội dung văn bản mà còn mang theo các trường metadata về phạm vi học liệu, vị trí phân cấp và nguồn gốc dữ liệu. Tập trường này là cơ sở để tăng truy hồi thực hiện lọc theo bộ sách, lớp, chủ đề và bài học, đồng thời hỗ trợ truy vết nguồn ở các bước sinh nội dung và đánh giá.

Trường `type` phân loại mỗi chunk theo vai trò sử dụng thay vì chỉ theo hình

Trường	Kiểu	Ý nghĩa
chunk_id	uuid	Định danh duy nhất, sinh tất định
book	str	Bộ sách (CD / KNTT)
grade	str	Khối lớp (10 / 11 / 12)
topic, topic_name	str	Mã và tên chủ đề
lesson, lesson_name	str	Mã và tên bài học
section_title	str	Tiêu đề mục chứa chunk
level	int	Độ sâu tiêu đề (1–6)
type	str	Loại nội dung sư phạm
breadcrumb	str	Đường dẫn ngữ cảnh: chủ đề > bài > mục
content	str	Nội dung văn bản của chunk
full_content	str	Ghép breadcrumb và content để vector hóa
source_path	str	Đường dẫn tệp nguồn

Bảng 3.2: Lược đồ metadata của một chunk

thức văn bản, gồm sáu nhãn: *theory* (lý thuyết, khái niệm, ví dụ), *exercise* (luyện tập, bài tập), *objective* (mục tiêu bài học), *application* (vận dụng), *summary* (tóm tắt) và *note* (lưu ý). Bảng 3.3 trình bày phân bố số lượng chunk theo từng nhãn.

Loại (type)	Vai trò	Số lượng	Tỷ lệ
<i>theory</i>	Lý thuyết, khái niệm, ví dụ	1.682	71,6%
<i>exercise</i>	Luyện tập, bài tập	250	10,6%
<i>objective</i>	Mục tiêu bài học	193	8,2%
<i>application</i>	Vận dụng	122	5,2%
<i>summary</i>	Tóm tắt	85	3,6%
<i>note</i>	Lưu ý	16	0,7%
Tổng		2.348	100%

Bảng 3.3: Phân bố số lượng chunk theo loại nội dung sư phạm (type)

Số liệu trong Bảng 3.3 cho thấy phần lớn nội dung thuộc nhãn *theory*, chiếm khoảng 71,6% tổng số chunk. Phân bố này phản ánh đặc thù của SGK, trong đó phần trình bày khái niệm, giải thích và ví dụ chiếm ưu thế so với các thành phần như bài tập, mục tiêu hay tóm tắt. Nhãn loại nội dung này tạo cơ sở cho việc khai thác chọn lọc ở các tác vụ phía sau, chẳng hạn ưu tiên chunk *objective* và *summary* khi dựng dàn ý bài giảng, hoặc chunk *exercise* khi sinh câu hỏi luyện tập.

3.2.2 Chunking theo tiêu đề

Khác với cách chia chunk theo cửa sổ ký tự cố định, chiến lược **chunking theo tiêu đề (heading-based chunking)** được sử dụng trong đồ án xác định ranh giới chunk trực tiếp từ cấu trúc tiêu đề Markdown của SGK. Theo đó, mỗi chunk tương ứng với một đơn vị nội dung mạch lạc về mặt sư phạm. Cụ thể, bộ chunking hoạt động như một máy trạng thái quét tuần tự từng dòng của tệp bài học; mỗi khi gặp một mốc phân cấp — dòng chủ đề (“CHỦ ĐỀ ...”), tiêu đề bài (“# Bài X: ...”) hay

tiêu đề mục (“## ...”, “### ...”) — hệ thống chốt chunk đang tích lũy và mở một chunk mới gắn với mốc đó. Cách chia này bảo đảm mỗi chunk nằm trọn trong một mục của một bài, không bị cắt ngang giữa câu hay giữa các ý.

Để làm rõ cơ chế này, có thể xét một tài liệu giả lập gồm ba tầng tiêu đề như sau:

CHU DE 2. Mạng máy tính và Internet

Bai 3. Giao tiep tren mang

Doan mo dau cua bai hoc.

1. Dia chi IP

Dia chi IP dung de dinh danh thiet bi tren mang.

Vi du

May tinh A co dia chi 192.168.1.10.

2. Ten mien

Ten mien giup nguoi dung de nho dia chi tai nguyn hon.

Với đầu vào trên, hệ thống không tạo một chunk duy nhất cho toàn bộ bài học mà chốt chunk tại từng mốc tiêu đề. Bảng 3.4 minh họa kết quả phân đoạn tương ứng.

Chunk giả lập	Level	Breadcrumb	Nội dung chính
c_1	1	Chủ đề 2 > Bài 3	Đoạn mở đầu của bài học
c_2	2	Chủ đề 2 > Bài 3 > 1. Địa chỉ IP	Khái niệm địa chỉ IP
c_3	3	Chủ đề 2 > Bài 3 > 1. Địa chỉ IP > Ví dụ	Ví dụ về địa chỉ IP cụ thể
c_4	2	Chủ đề 2 > Bài 3 > 2. Tên miền	Khái niệm tên miền

Bảng 3.4: Ví dụ minh họa cách chunking theo tiêu đề trên một tài liệu ba tầng heading

Bảng 3.4 cho thấy ranh giới chunk được quyết định bởi tiêu đề gần nhất đang có hiệu lực. Đoạn mở đầu ngay sau tiêu đề bài được gán vào chunk mức 1. Khi gặp mục “1. Địa chỉ IP”, hệ thống tạo một chunk mới ở mức 2. Phần “Ví dụ” bên trong mục này tiếp tục được tách thành một chunk mức 3, do nó được mở bởi một tiêu đề con riêng. Tương tự, khi gặp mục “2. Tên miền”, hệ thống chốt chunk trước đó và mở ra một chunk mới. Nhờ cách chia này, mỗi chunk giữ được cả nội dung trực tiếp lẫn vị trí phân cấp trong bài học.

Mỗi chunk được gán trường `level` bằng đúng độ sâu tiêu đề trong Mark-

down: tiêu đề bài tương ứng `level = 1`, các mục chính `level = 2`, các mục con `level ≥ 3`. Phân bố thực tế cho thấy nội dung tập trung chủ yếu ở `level 2` (1.229 chunk) và `level 3` (921 chunk). Song song, mỗi chunk mang một chuỗi *breadcrumb* dạng “tên chủ đề > tên bài > tiêu đề mục” làm ngữ cảnh phân cấp gọn nhẹ; breadcrumb được ghép vào trước nội dung khi embedding và khi xếp hạng lại bằng *Reranker*, qua đó hỗ trợ phân biệt các chunk có câu chữ tương tự nhưng thuộc bài khác nhau.

Vì ranh giới chunk bám theo tiêu đề nên kích thước chunk phụ thuộc độ dài tự nhiên của từng mục. Để xử lý hai trường hợp cực đoan, một bước chuẩn hóa kích thước nhẹ được áp dụng sau khi chia: các chunk quá ngắn (dưới 100 ký tự) được gộp vào chunk liền trước nếu cùng bài và cùng loại nội dung, còn các chunk quá dài (trên 2.000 ký tự) được cắt theo ranh giới đoạn văn. Các ngưỡng này chỉ tác động tới một phần nhỏ tập chunk và không làm thay đổi bản chất chia theo tiêu đề; độ dài chunk thực tế dao động quanh trung bình 651 ký tự. Cuối cùng, nhãn `type` được tinh chỉnh bằng một mô hình ngôn ngữ nhẹ (Gemini 2.5 Flash-Lite) trên cơ sở nhãn heuristic ban đầu, có cơ chế đệm (cache) để bảo đảm tính tái lập; đồng thời mỗi chunk nhận một `chunk_id` sinh tất định bằng hàm băm trên (bộ sách, lớp, chủ đề, bài, breadcrumb, thứ tự, dấu vân nội dung), giúp việc tái dựng tập chunk có tính idempotent.

3.2.3 Biểu diễn vector tiếng Việt và lưu trữ Qdrant

Để hỗ trợ truy hồi ngữ nghĩa, mỗi chunk được chuyển thành một vector số thực bằng mô hình embedding tiếng Việt `dangvantuan/vietnamese-document-embedding`, sinh vector 768 chiều. Văn bản đưa vào mô hình là trường `full_content` (breadcrumb ghép với nội dung) thay vì chỉ riêng nội dung, để ngữ cảnh phân cấp được phản ánh trong không gian vector. Sau khi mã hóa, các vector được chuẩn hóa L_2 nhằm bảo đảm độ đo cosine quy về tích vô hướng; quá trình này có kiểm tra và xử lý các vector suy biến (chuẩn gần không hoặc giá trị không hữu hạn) để duy trì tính ổn định của tập chỉ mục.

Tập vector cùng metadata được lưu vào Qdrant dưới một collection tên `educational_chatbot`, cấu hình không gian 768 chiều với độ đo khoảng cách cosine. Mỗi điểm (point) trong Qdrant nhận `chunk_id` làm định danh, mang vector ngữ nghĩa và toàn bộ payload metadata ở Bảng 3.2; nhờ đó hệ thống có thể vừa tìm kiếm theo độ tương đồng vừa lọc cứng theo bộ sách, lớp, chủ đề trong cùng một truy vấn. Quá trình nạp dữ liệu được thực hiện theo lô và có kiểm tra tính nhất quán giữa số chunk và số vector. Ngoài ra, một dấu vân (fingerprint) trên nguồn embedding được lưu kèm để phát hiện khi dữ liệu chunk thay đổi và chỉ tính lại vector khi

cần. Thiết kế lưu trữ này cho phép tăng truy hồi nạp dữ liệu từ kho Qdrant hoặc từ tệp vector cục bộ trong các kịch bản triển khai và thực nghiệm.

3.3 Cơ chế truy hồi ngữ cảnh (RAG core)

3.3.1 Truy hồi lai BM25 và dense, hợp nhất bằng Reciprocal Rank Fusion

Lỗi truy hồi của hệ thống được xây dựng theo hướng truy hồi lai, kết hợp giữa tìm kiếm từ khóa (*keyword search*) và tìm kiếm ngữ nghĩa theo vector (*vector search*). Trong đó, *keyword search* phù hợp với các truy vấn có mức trùng khớp bề mặt cao với SGK, còn *vector search* phù hợp với các truy vấn diễn đạt lại hoặc dùng từ đồng nghĩa.

Với *keyword search*, hệ thống sử dụng BM25. Xuất phát từ TF-IDF, trong đó tần suất từ được ký hiệu bởi Công thức 3.1:

$$TF(t, d) = f(t, d), \quad (3.1)$$

và tần suất nghịch đảo tài liệu được tính theo Công thức 3.2:

$$IDF(t) = \log \frac{N - n(t) + 0.5}{n(t) + 0.5}, \quad (3.2)$$

Trong đó:

- t là một term trong truy vấn.
- N là số tài liệu trong tập chỉ mục.
- $n(t)$ là số tài liệu chứa từ t .

Trên cơ sở đó, điểm BM25 của truy vấn q đối với chunk d được tính như sau:

$$BM25(q, d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)}. \quad (3.3)$$

Trong đó:

- q là truy vấn.
- t là một term thuộc truy vấn q .
- $f(t, d)$ là số lần xuất hiện của từ t trong chunk d .
- $|d|$ là độ dài của chunk d .
- $avgdl$ là độ dài chunk trung bình của toàn bộ tập chỉ mục.

- k_1 và b là các siêu tham số điều chỉnh ảnh hưởng của tần suất từ và độ dài tài liệu.

Với *vector search*, mỗi truy vấn q và mỗi chunk d được ánh xạ thành embedding $\mathbf{e}_q$ và $\mathbf{e}_d$. Độ tương đồng được tính bằng cosine similarity theo Công thức 3.4:

$$\text{cos_sim}(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_d\|_2}. \quad (3.4)$$

Trong đó:

- $\mathbf{e}_q$ là embedding của truy vấn.
- $\mathbf{e}_d$ là embedding của chunk.
- $\mathbf{e}_q \cdot \mathbf{e}_d$ là tích vô hướng giữa hai vector.
- $\|\cdot\|_2$ là chuẩn Euclid.

Do các vector được chuẩn hóa L_2 trước khi lập chỉ mục, độ đo này quy về so sánh hướng trong không gian vector và được dùng trực tiếp trong truy vấn Qdrant.

Hai danh sách kết quả từ BM25 và *vector search* được hợp nhất bằng *Reciprocal Rank Fusion* (RRF), thay vì trộn trực tiếp điểm số. Với một chunk d , điểm RRF được xác định theo Công thức 3.5:

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)}, \quad (3.5)$$

Trong đó:

- $\mathcal{R}$ là tập các danh sách xếp hạng thành phần.
- $\text{rank}_r(d)$ là thứ hạng của chunk d trong danh sách r .
- k là hằng số làm tròn.

Cách hợp nhất này tránh phụ thuộc vào khác biệt thang điểm giữa BM25 và cosine similarity, đồng thời giữ lại các chunk có thứ hạng cao ở ít nhất một kênh truy hồi. Kết quả của bước này là một tập candidate ban đầu để chuyển sang bước reranking.

3.3.2 Xếp hạng lại bằng cross-encoder

Sau bước truy hồi lai, hệ thống áp dụng một bước *reranking* bằng cross-encoder để tái xếp hạng tập candidate. Mục tiêu của bước này là thay thế thứ tự dựa trên mức tương đồng sơ cấp bằng thứ tự gần hơn với mức độ liên quan thực sự giữa truy vấn và chunk.

Reranker nhận đầu vào là cặp (*query*, *context*) và sinh ra một điểm số liên quan

$$s_{\text{rerank}} = f_{\theta}(q, d), \quad (3.6)$$

Trong đó:

- q là truy vấn.
- d là chunk ứng viên.
- θ là tập tham số của mô hình.
- f_{θ} là hàm chấm điểm của cross-encoder.

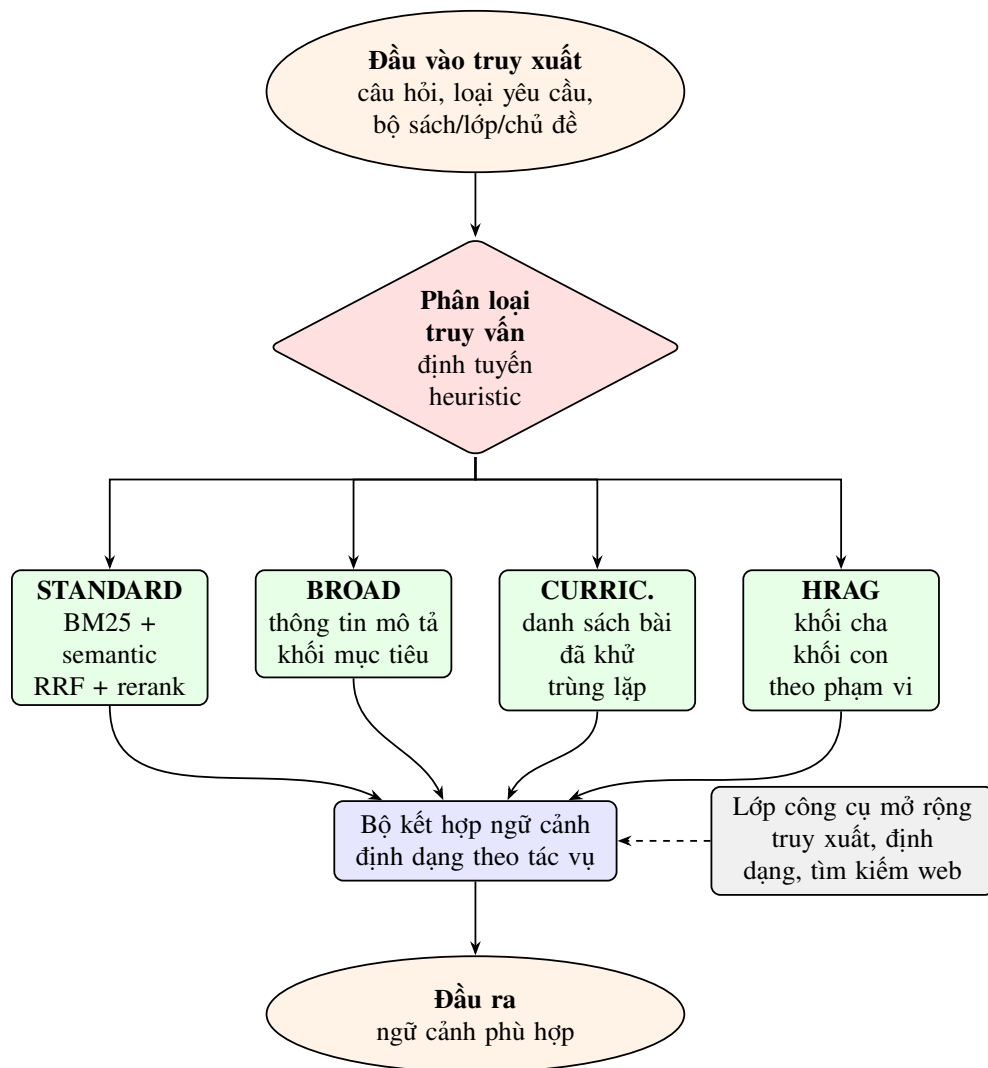
Khác với *vector search*, mô hình này mã hóa đồng thời cả truy vấn và chunk, do đó có thể khai thác tương tác token-level giữa hai vế. Trong triển khai hiện tại, phần văn bản đưa vào *Reranker* không chỉ gồm *content* mà còn ghép thêm *breadcrumb*, nhằm cung cấp thêm ngữ cảnh phân cấp của chunk trong SGK.

Sau khi tính lại điểm số, hệ thống chọn top- N chunk để xây dựng ngữ cảnh cuối. Giá trị N được đặt theo loại tác vụ: với hỏi đáp ngắn, N nhỏ hơn để hạn chế nhiễu; với slide hoặc giáo án, N lớn hơn để tăng độ bao phủ. Vì vậy, reranking vừa là bước cải thiện thứ hạng, vừa là bước chọn lọc ngữ cảnh đầu vào cho tầng generation.

3.3.3 Truy hồi thích ứng (Adaptive RAG)

Không phải mọi truy vấn trong hệ thống đều cần cùng một cách truy hồi. Câu hỏi định nghĩa ngắn, yêu cầu liệt kê bài học, truy vấn tổng quan theo chủ đề và yêu cầu sinh slide từ một bài học có phạm vi ngữ cảnh khác nhau. Vì vậy, hệ thống sử dụng cơ chế *Adaptive RAG* để chọn strategy truy hồi phù hợp theo loại truy vấn và thông tin phạm vi hiện có.

Về mặt lý thuyết, cơ chế này có hai thành phần chính. Thành phần thứ nhất là bước phân loại truy vấn, nhằm xác định loại nhu cầu truy hồi trước khi chọn strategy. Thành phần thứ hai là bước ràng buộc phạm vi (*scope filtering*) theo bộ sách, lớp hoặc chủ đề nếu các metadata này đã được nhận diện ở tầng điều phối. Nhờ đó, không gian tìm kiếm được thu hẹp trước khi thực hiện BM25 hoặc *vector search*. Trong trường hợp strategy đang dùng không trả về đủ ngữ cảnh, hệ thống có thể fallback về cấu hình mặc định để bảo đảm độ phủ.



Hình 3.3: Cơ chế truy hồi thích ứng của hệ thống với bộ phân loại truy vấn và các chiến lược truy xuất tương ứng

Trong Hình 3.3, bộ phân loại truy vấn được đặt ở trung tâm vì đây là thành phần quyết định strategy truy hồi sẽ được áp dụng trước. Bốn strategy ở tầng dưới tương ứng với bốn kiểu nhu cầu truy hồi khác nhau, trong khi khối “Bộ kết hợp ngữ cảnh” chuẩn hóa đầu ra của từng strategy về một cấu trúc chung cho tầng trên sử dụng. Trong cấu trúc này, *STANDARD* là strategy mặc định gồm BM25, *vector search*, RRF và reranking. *BROAD* ưu tiên các chunk mô tả ở mức khái quát. *CURRICULUM* phục vụ các truy vấn liệt kê bài học hoặc cấu trúc chương trình, trong đó đầu ra cần ở mức danh sách bài thay vì đoạn nội dung chi tiết. *HRAG* kết hợp các chunk mức cao và mức chi tiết trong cùng phạm vi bài học. Sau khi truy hồi, khối “Bộ kết hợp ngữ cảnh” thực hiện chuẩn hóa đầu ra để các strategy khác nhau có thể cung cấp ngữ cảnh theo cùng một cấu trúc cho tầng trên sử dụng.

3.4 Tầng điều phối tác tử

3.4.1 Sự cần thiết của tầng điều phối tác tử

Nếu hệ thống chỉ phục vụ một tác vụ hỏi đáp đơn giản, một pipeline tuyến tính gồm query analysis, retrieval và answer generation là đủ. Tuy nhiên, bài toán của đồ án bao gồm đồng thời nhiều nhóm yêu cầu khác nhau, từ hỏi đáp khái niệm, giải thích bài học, sinh slide đến soạn giáo án. Các yêu cầu này khác nhau không chỉ ở output format mà còn ở retrieval scope, mức độ phụ thuộc vào multi-turn context và số bước xử lý trung gian. Do đó, việc dùng một pipeline cố định cho mọi truy vấn làm giảm khả năng thích ứng của hệ thống.

Tầng điều phối tác tử được đưa vào để tách riêng ba trách nhiệm: intent understanding, execution routing và action execution. Trong kiến trúc này, bộ điều phối không được thiết kế như một *Agent* tự trị bao trùm toàn bộ hệ thống. Thay vào đó, nó là một lớp orchestration mỏng, giữ *request state*, chỉ gọi *LLM* ở các vị trí cần suy luận ngôn ngữ như intent routing hoặc query rewriting, và dùng deterministic logic cho planning, dispatch và session management. Cách tổ chức này giúp giảm số lượt gọi mô hình, hạn chế latency và làm rõ ranh giới giữa reasoning dựa trên ngôn ngữ với quyết định có thể mô hình hóa bằng rule.

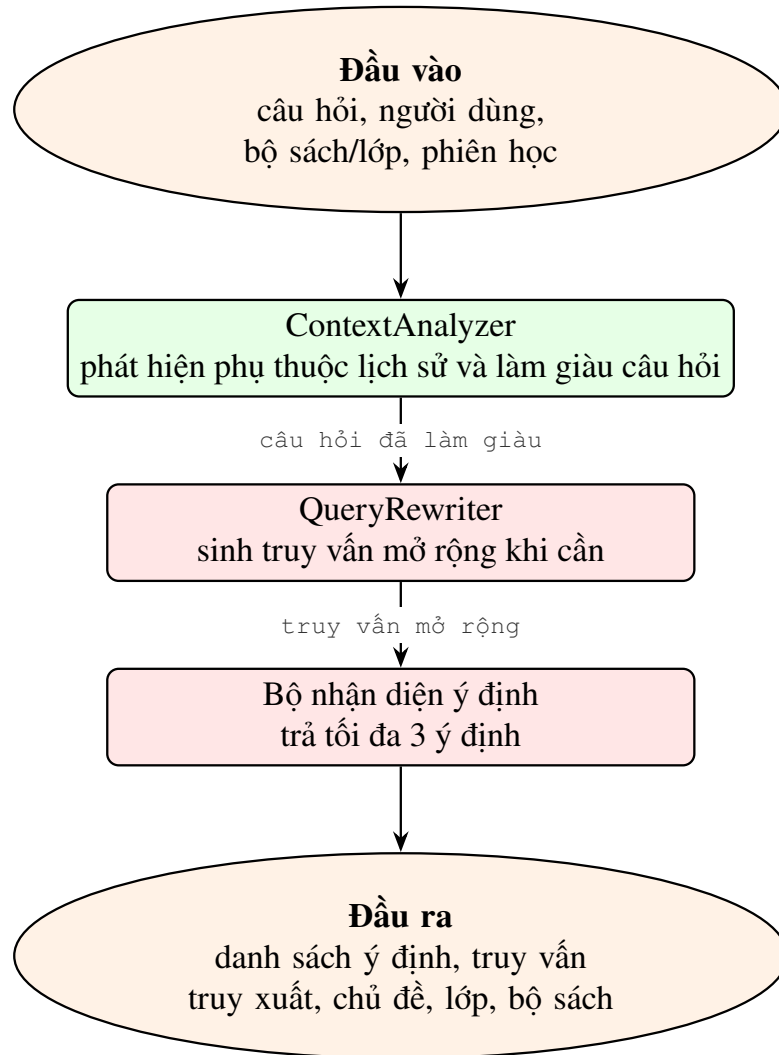
Vai trò thứ hai của tầng điều phối là hỗ trợ multi-intent request. Trong thực tế sử dụng, một câu hỏi có thể đồng thời chứa nhiều action, chẳng hạn vừa yêu cầu giải thích khái niệm, vừa yêu cầu tạo bài tập hoặc sinh dàn ý bài giảng. Khi đó, hệ thống cần phân rã đầu vào thành một *action list* có thứ tự, tránh xử lý trùng lặp và duy trì được *session state* xuyên suốt phiên làm việc. Đây là lý do phần điều phối được mô hình hóa thành một tầng riêng, thay vì gắn trực tiếp vào từng dịch vụ nghiệp vụ.

3.4.2 Định tuyến ý định bằng mô hình ngôn ngữ lớn

Điểm vào đầu tiên của tầng điều phối là bài toán intent routing. Với cùng một surface query, hệ thống cần xác định truy vấn đang hướng tới hỏi đáp kiến thức, sinh học liệu dài, hay một thao tác phụ trợ khác như thống kê tiến độ hoặc ôn câu sai. Ngoài intent type, hệ thống còn cần trích xuất các scope field như bộ sách, lớp, chủ đề và bài học khi thông tin này xuất hiện trong câu hỏi hoặc có thể suy ra từ session context.

Trong phương pháp đề xuất, bước intent routing được giao cho *LLM* thay vì heuristic thuần túy. Lý do là cùng một intent có thể được diễn đạt theo nhiều cách khác nhau, đặc biệt trong ngôn ngữ học sinh. Router cần xử lý được indirect query, composite query và cả các trường hợp câu hỏi ngắn nhưng phụ thuộc mạnh vào turn history. Đầu ra của bước này không phải một nhãn duy nhất, mà là một danh

sách tối đa ba intent có thứ tự ưu tiên, đi kèm các scope field phục vụ các tầng sau.



Hình 3.4: Khối phân tích ngữ cảnh và định tuyến ý định trong tầng điều phối

Hình 3.4 mô tả processing chain từ câu hỏi gốc đến đầu ra intent routing. Đầu vào ban đầu gồm câu hỏi hiện tại, thông tin người dùng, bộ sách hoặc lớp được chọn trên giao diện và *session state*. Thành phần *ContextAnalyzer* đứng trước cùng vì đây là bước quyết định câu hỏi hiện tại có phụ thuộc vào turn history hay không. Nếu phụ thuộc, hệ thống lấy các lượt hội thoại gần và liên quan để mở rộng câu hỏi hiện tại trước khi chuyển sang bước sau.

Sau bước phân tích ngữ cảnh, hệ thống chỉ gọi *QueryRewriter* khi truy vấn cần được reformulate để hỗ trợ *retrieval*. Thành phần này không thay thế câu hỏi gốc, mà tạo ra một hoặc nhiều expanded query nhằm tăng recall của tầng truy hồi trong các trường hợp câu hỏi quá ngắn, thiếu chủ ngữ hoặc dùng diễn đạt không trùng với SGK. Cuối cùng, bộ định tuyến ý định dùng *LLM* để phân tích yêu cầu đã được làm giàu và trả về danh sách intent, retrieval query và các scope field như chủ đề, lớp và bộ sách. Đầu ra này là input trực tiếp cho tầng planning.

3.4.3 Làm giàu ngữ cảnh hội thoại và viết lại truy vấn

Context enrichment là bước cần thiết vì nhiều truy vấn trong thực tế không tự mang đủ thông tin để tăng truy hồi hoạt động chính xác. Các câu như “giải thích kỹ hơn”, “bài này khác gì bài trước” hoặc “làm cho em vài câu hỏi” chỉ có thể được hiểu đúng nếu hệ thống nối chúng với turn trước đó. Do vậy, trước khi query rewriting, hệ thống trước hết phải xác định liệu câu hỏi hiện tại có phụ thuộc vào history hay không.

Trong hệ thống, *ContextAnalyzer* được hiện thực bằng deterministic logic. Thành phần này quan sát câu hỏi hiện tại, *session state* và một số surface cue như đại từ hồi chỉ, câu rút gọn hoặc việc người dùng tiếp tục một chủ đề đã mở trước đó. Khi phát hiện context dependency, bộ phân tích trích xuất một cửa sổ hội thoại ngắn, ưu tiên các turn gần nhất và các turn có cùng lesson scope. Kết quả của bước này là một phiên bản câu hỏi đã được disambiguate, đủ để tăng dưới xử lý mà không cần mang theo toàn bộ history.

QueryRewriter đóng vai trò chuyển câu hỏi đã làm giàu thành dạng phù hợp hơn cho *retrieval*. Với truy vấn hỏi đáp, bộ viết lại có thể sinh thêm các paraphrase gần nghĩa để tăng recall của *keyword search* và *vector search*. Với truy vấn sinh học liệu dài, bộ viết lại ưu tiên làm rõ lesson scope, topic scope và output type. Việc tách bước này khỏi *IntentRouter* giúp hệ thống kiểm soát hai bài toán khác nhau: một bài toán là intent understanding, bài toán còn lại là retrieval-oriented reformulation.

Trong thiết kế hiện tại, conversation context sau khi được phân tích không được đưa thẳng vào toàn bộ pipeline ở mọi bước. Thay vào đó, nó được nén thành một structured representation gồm câu hỏi đã làm rõ, expanded query, scope field và cờ đánh dấu mức độ phụ thuộc history. Cách chuẩn hóa này giúp các thành phần planning và retrieval dùng lại cùng một input schema, thay vì mỗi thành phần tự diễn giải lịch sử theo cách riêng.

3.4.4 Quản lý trạng thái hội thoại và vòng đời session

State management là nền của tầng điều phối, không phải một feature tách rời. Sau bước intent routing, hệ thống phải quyết định câu hỏi hiện tại có thuộc cùng một phiên làm việc với các turn trước đó hay không, rồi mới có thể lập kế hoạch hành động chính xác. Vì vậy, trong phương pháp đề xuất, *SessionManager* được đặt đúng giữa intent routing và action planning, đóng vai trò duy trì continuity của multi-turn context, đồng thời ngăn việc trộn lẫn các workflow khác bản chất trong cùng một session.

Mô hình trạng thái được tổ chức theo ba lớp. Ở lớp chung nhất là *Session*, lưu định danh phiên, người dùng, topic hiện tại, intent chính, book scope, timestamp và lịch sử message. Trên nền đó, hệ thống chỉ kích hoạt một trong hai intent-specific state là *QuizState* hoặc *SlideState*, tương ứng với hai nhóm workflow có cấu trúc trạng thái khác nhau. *QuizState* phục vụ các vòng sinh câu hỏi, chấm đáp án và ôn tập; *SlideState* phục vụ các workflow sinh slide, giáo án và các điểm dừng *human-in-the-loop*. Thiết kế này giúp session vừa giữ được conversation history dùng chung, vừa cô lập được state nghiệp vụ đặc thù của từng task type.

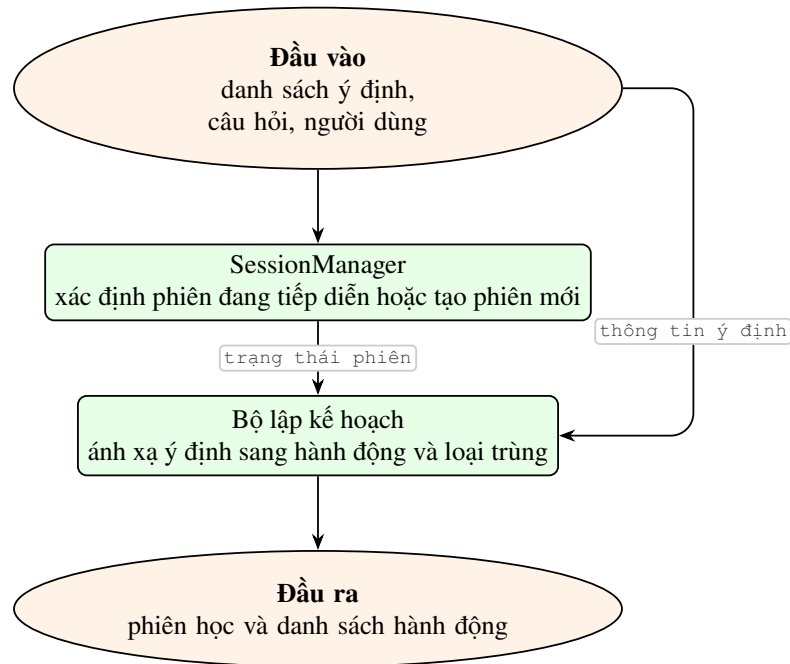
Luật vòng đời session được thực hiện bởi *SessionManager.resolve_session()*. Nếu người dùng chưa có session hiện hành, hệ thống tạo session mới. Nếu *IntentRouter* phát hiện đổi topic rõ ràng, session cũ được lưu lại và phiên mới được khởi tạo để tránh kéo theo context không còn liên quan. Với các yêu cầu sinh nội dung, hệ thống còn xét thêm *task type transition*. Chẳng hạn, một session đang ở nhánh quiz có thể tiếp tục cho nhiều round cùng loại, nhưng khi chuyển từ quiz sang slide hoặc từ slide sang quiz thì hệ thống tách phiên mới, vì hai workflow này có state object, artifact và logic điều phối khác nhau. Khi không có tín hiệu đổi topic hoặc đổi task type, session hiện tại được reuse để bảo toàn ngữ cảnh hội thoại.

Vai trò quan trọng thứ hai của session management là cung cấp history theo đúng mức chi tiết cho các bước suy luận ngôn ngữ. Trong thiết kế hiện tại, *Session.get_context_messages()* trả về một cửa sổ message gần nhất để inject vào intent detection, giúp router phân biệt các truy vấn mơ hồ như “làm tiếp”, “chấm câu này” hay “giải thích ý trên”. Ở nhánh contextualization, *ContextAnalyzer* và *QueryRewriter* không dùng toàn bộ session thô, mà chỉ lấy phần history liên quan để tạo *context snippet* rồi viết lại truy vấn phục vụ *retrieval*. Cách dùng này giữ cho session trở thành nguồn trạng thái thống nhất của tầng điều phối, nhưng vẫn kiểm soát được độ dài context và ranh giới thông tin được truyền sang từng node.

Việc tách state management thành một cơ chế thiết kế riêng trong tầng điều phối mang lại hai lợi ích. Thứ nhất, action planner luôn nhận được một *session state* đã được chuẩn hóa về topic, task type và history context, nhờ đó quyết định tiếp theo không phải suy diễn lại trạng thái từ đầu. Thứ hai, các workflow multi-turn như “sinh quiz rồi chấm đáp án”, “sinh dàn ý rồi duyệt outline”, hoặc “hỏi tiếp trong cùng bài học” có thể vận hành liên tục mà không làm rò rỉ ngữ cảnh sang một tác vụ khác. Chi tiết cài đặt của lớp lưu phiên, cơ chế auto-save và đồng bộ truy cập sẽ được trình bày ở Chương 4, vì chúng thuộc phạm vi hiện thực hóa hơn là phạm vi thiết kế phương pháp.

3.4.5 Lập kế hoạch hành động và thực thi đa hành động

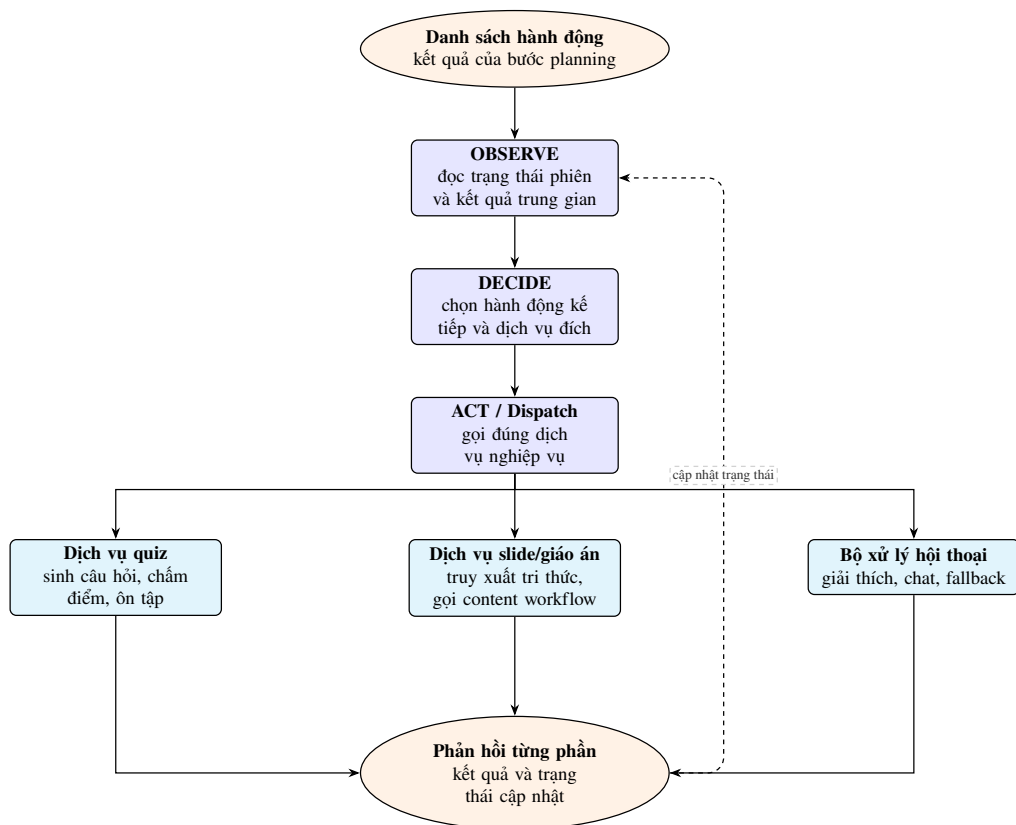
Sau khi có danh sách intent, hệ thống chuyển sang bước action planning. Mục tiêu của bước này là ánh xạ từng intent sang một business action cụ thể, đồng thời quyết định execution order khi trong cùng một yêu cầu xuất hiện nhiều intent. Khác với bước intent routing, bài toán planning ở đây được giải quyết bằng deterministic logic thay vì *LLM*. Lý do là action space của hệ thống là hữu hạn và đã biết trước, bao gồm hỏi đáp, giải thích khái niệm, sinh slide, sinh giáo án, sinh quiz, chấm đáp án, ôn câu sai và một số thao tác hỗ trợ khác.



Hình 3.5: Cơ chế lập kế hoạch hành động ở tầng điều phối

Hình 3.5 cho thấy bước planning gồm hai khối chính. Trước hết, *SessionManager* xác định session hiện tại của người dùng, từ đó quyết định nên tiếp tục workflow đang có hay khởi tạo phiên mới. Tiếp theo, planner nhận đồng thời thông tin intent và *session state* để sinh ra một *action list* đã được deduplicate. Chẳng hạn, nếu nhiều intent liên tiếp cùng dẫn tới một hành động hỏi đáp trong cùng một lesson scope, hệ thống không tạo nhiều bước xử lý trùng nhau mà gộp về một action duy nhất.

Sau bước planning, *action list* được thực thi theo một execution loop đa hành động. Về bản chất, đây là một chu trình *OBSERVE–DECIDE–ACT*: hệ thống đọc *session state* và intermediate result, chọn action kế tiếp cần chạy, rồi dispatch sang đúng business service tương ứng. Cấu trúc này cho phép hệ thống stream dần kết quả của từng action, đồng thời cập nhật intermediate state mà không cần chờ toàn bộ composite request kết thúc mới sinh phản hồi.



Hình 3.6: Vòng thực thi đa hành động trong tầng điều phối theo chu trình quan sát – quyết định – hành động

Hình 3.6 làm rõ execution loop sau bước planning. Khối *OBSERVE* đọc *session state* và các intermediate result đã có. Khối *DECIDE* chọn action kế tiếp cần chạy dựa trên *action list* còn lại. Khối *ACT* tương ứng với execution dispatcher, chịu trách nhiệm route sang đúng business service. Với các action gắn với quiz, luồng được chuyển sang dịch vụ quiz. Với các action cần sinh học liệu dài, luồng được chuyển sang dịch vụ slide/giáo án để tiếp tục gọi nhánh *Multi-Agent*. Với các action giải thích hoặc hội thoại ngắn, hệ thống dùng chat handler chuyên biệt. Kết quả sau mỗi vòng được ghi trở lại vào *session state* để phục vụ iteration kế tiếp.

Thiết kế này cho phép bộ điều phối giữ vai trò trung tâm của phương pháp đề xuất mà không làm nó trở thành một monolithic reasoning block. *LLM* chỉ xuất hiện ở những bước cần language understanding hoặc query reformulation; còn state management, planning và action dispatch vẫn được thực hiện bằng deterministic mechanism. Nhờ đó, hệ thống duy trì được tính giải thích được của processing flow, đồng thời vẫn đủ linh hoạt để hỗ trợ composite request trong cùng một session.

3.5 Tầng sinh nội dung đa tác tử

3.5.1 Mô hình Supervisor–Specialist và lý do lựa chọn

Đối với các tác vụ sinh slide và soạn giáo án, hệ thống không chỉ cần tạo văn bản đầu ra mà còn phải bảo đảm đồng thời nhiều ràng buộc như cấu trúc sư phạm, độ bám nguồn, khả năng gắn học liệu minh họa, mức độ đầy đủ của phần luyện tập và tính nhất quán của toàn bộ artifact cuối. Nếu toàn bộ quá trình này được giao cho một *single-agent* duy nhất, prompt sẽ phải bao hàm cùng lúc yêu cầu lập dàn ý, viết nội dung, đề xuất media, sinh đánh giá và tự kiểm tra chất lượng. Cách tiếp cận đó làm tăng tải nhận thức của mô hình, khiến ranh giới trách nhiệm mờ đi và khó truy vết khi đầu ra không đạt yêu cầu.

Vì vậy, đồ án lựa chọn kiến trúc *Supervisor–Specialist*. Trong kiến trúc này, một *Content Supervisor* giữ vai trò điều phối workflow, còn từng *specialist agent* chỉ xử lý một lớp nhiệm vụ hẹp đã được xác định trước. Supervisor không trực tiếp sinh nội dung bài học mà thực hiện ba chức năng chính: (i) phân rã yêu cầu thành các bước con, (ii) ràng buộc thứ tự thực thi giữa các bước, và (iii) thu thập kết quả trung gian để quyết định bước tiếp theo. Cách phân tách này giúp workflow sinh nội dung gắn với một quy trình xử lý có trạng thái (*stateful workflow*) hơn là một lần gọi mô hình đơn lẻ.

Một ưu điểm quan trọng khác của mô hình này là khả năng kiểm soát *grounding*. Trong pipeline hiện tại, phần outline được sinh trước để xác lập khung sư phạm, sau đó agent viết nội dung mới được phép khai thác các chunk đã truy hồi để điền nội dung chi tiết cho từng slide hoặc từng mục giáo án. Việc tách outline và drafting thành hai bước độc lập giúp giảm nguy cơ mô hình vừa tự quyết cấu trúc vừa tự mở rộng nội dung ngoài phạm vi nguồn. Đồng thời, các bước tùy chọn như gợi ý media hoặc sinh đánh giá nhúng cũng có thể được thêm vào hoặc bỏ qua mà không làm thay đổi hợp đồng dữ liệu của các bước cốt lõi.

Xét về mặt hiện thực hệ thống, kiến trúc *Supervisor–Specialist* còn phù hợp với LangGraph, nơi toàn bộ workflow được mô hình hóa thành đồ thị có trạng thái, hỗ trợ *interrupt–resume* cho *human-in-the-loop* và hỗ trợ vòng phản tư (*reflection loop*) khi hậu kiểm chất lượng yêu cầu sửa đổi. Do đó, đây là lựa chọn phù hợp hơn so với *single-agent* nếu mục tiêu không chỉ là “sinh được nội dung”, mà còn là quản lý được quá trình sinh nội dung một cách có thể giải thích và có thể kiểm chứng.

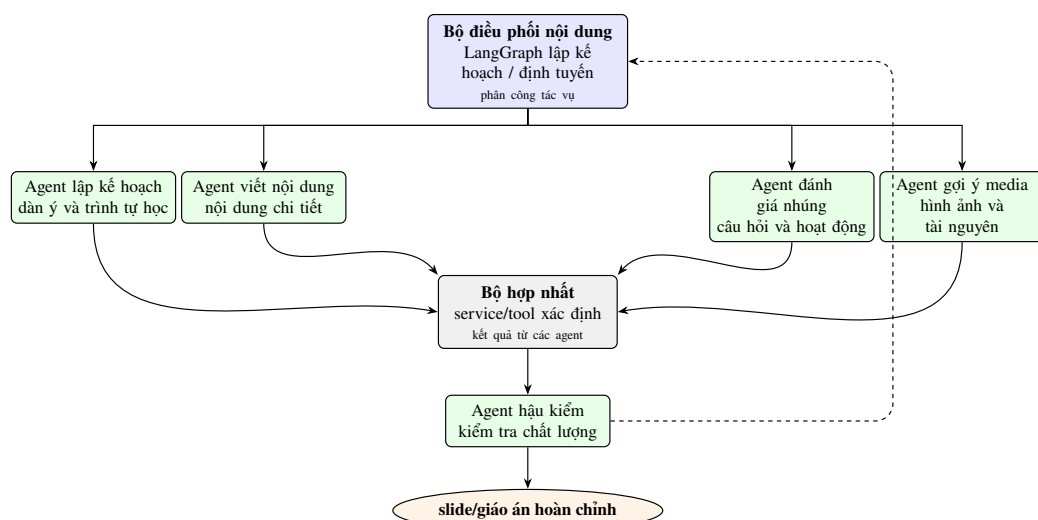
3.5.2 Giao thức giao tiếp và cơ chế điều phối giữa các tác tử

Để các tác tử chuyên biệt có thể phối hợp mà không phụ thuộc chặt vào cài đặt nội bộ của nhau, hệ thống sử dụng một giao thức trao đổi thống nhất dựa trên

hai thực thể *AgentTask* và *AgentTaskResult*. *AgentTask* là gói lệnh mà supervisor gửi cho một specialist agent, bao gồm định danh tác vụ, loại tác vụ, mục tiêu, đầu vào, tập ràng buộc, tham chiếu ngữ cảnh và loại artifact kỳ vọng. Chiều ngược lại, *AgentTaskResult* là gói phản hồi chuẩn hóa, chứa trạng thái thực thi, artifact đầu ra, điểm tin cậy (*confidence*), danh sách cảnh báo, thời gian xử lý và thông tin lỗi nếu có. Việc chuẩn hóa hai thực thể này giúp supervisor có thể điều phối nhiều agent bằng cùng một cơ chế, thay vì phải xây dựng một giao diện riêng cho từng agent.

Toàn bộ workflow được duy trì trên một *workflow state* dùng chung. Trạng thái này chứa không chỉ các trường đầu vào như `task_type`, `query`, `topic`, `grade`, `book` và tập `rag_chunks`, mà còn lưu các artifact trung gian như `outline_payload`, `content_payload`, `media_payload`, `quiz_payload`, `quality_review` và một *artifact registry* để supervisor có thể tra cứu kết quả theo `artifact_type`. Cấu trúc trạng thái này đóng vai trò là lớp bộ nhớ thao tác của riêng pipeline sinh nội dung, tách biệt với *session state* ở tầng điều phối đã trình bày trước đó.

Trong triển khai hiện tại, cơ chế điều phối được hiện thực bằng các *tool adapter* gắn với supervisor. Chúng vừa đóng vai trò cầu nối để chuyển từ quyết định của supervisor sang một *AgentTask* cụ thể, vừa đảm bảo rằng dữ liệu đầu vào cho từng specialist luôn tuân theo hợp đồng tương ứng. Chẳng hạn, bước `generate_content` sẽ bị chặn nếu chưa có `outline_payload`; bước `merge_results` chỉ được phép chạy sau khi ít nhất outline và content đã sẵn sàng; còn bước `check_quality` chỉ được kích hoạt sau khi artifact cuối đã được hợp nhất. Nhờ vậy, phần suy luận bằng mô hình ngôn ngữ được giữ trong phạm vi “quyết định dùng tác tử nào”, còn điều kiện hợp lệ của workflow vẫn được bảo vệ bằng logic xác định.



Hình 3.7: Kiến trúc đa tác tử cho bài toán sinh slide và giáo án

Hình 3.7 mô tả quan hệ giữa supervisor, các specialist agent và bộ hợp nhất kết quả. Ở đỉnh sơ đồ là bộ điều phối nội dung, chịu trách nhiệm quyết định bước nào sẽ được gọi kế tiếp. Ở tầng dưới là các specialist agent tương ứng với các nhiệm vụ con: lập dàn ý, viết nội dung, gợi ý media và sinh đánh giá nhúng. Điểm cần nhấn mạnh là các agent này không giao tiếp trực tiếp với nhau; mọi yêu cầu đều đi qua supervisor. Cách tổ chức này giữ cho luồng điều khiển tập trung tại một điểm, nhờ đó dễ áp dụng các ràng buộc thứ tự như “phải có outline trước khi viết nội dung” hoặc “chỉ được hậu kiểm sau khi đã hợp nhất kết quả”.

3.5.3 Các tác tử chuyên biệt trong pipeline sinh nội dung

Trong pipeline hiện tại, *PedagogyPlannerAgent* là tác tử được kích hoạt đầu tiên. Đầu vào của tác tử này là `context_map` đã được tổng hợp từ các chunk truy hồi, cùng với metadata về chủ đề, lớp, bộ sách và `task_type`. Nhiệm vụ của nó là xây dựng *outline payload* gồm tiêu đề bài học và danh sách các slide hoặc section dự kiến, mỗi phần tử có `slide_type`, tiêu đề, các ý chính và tham chiếu đến các chunk nguồn liên quan. Về bản chất, đây là bước chuyển từ ngữ cảnh truy hồi sang cấu trúc sơ phạm ở mức khung, tạo tiền đề cho các bước sinh nội dung sau đó.

ContentDraftingAgent là tác tử cốt lõi của pipeline vì nó trực tiếp biến outline thành nội dung giảng dạy. Tác tử này nhận `outline_payload` đã được duyệt cùng với `chunk_map`, sau đó sinh `content_payload` ở mức từng slide hoặc từng mục giáo án. Trong ràng buộc gửi kèm *AgentTask*, hệ thống đặt `must_use_source_chunks` do đó, nội dung sinh ra phải bám vào tập chunk đã truy hồi thay vì mở rộng tùy ý theo tri thức tham số của mô hình. Đây là cơ chế quan trọng để duy trì *grounding* cho các artifact có độ dài lớn hơn hỏi đáp ngắn.

MediaResearchAgent giữ vai trò hỗ trợ. Tác tử này không làm thay đổi luồng kiến thức chính của bài giảng mà chỉ sinh các gợi ý media như ảnh minh họa hoặc tài nguyên trực quan phù hợp với loại slide. Kết quả của bước này được lưu dưới dạng `media_payload`, gồm media cho trang mở đầu và media gắn với các slide nội dung. Trong trường hợp bước này thất bại, workflow vẫn có thể tiếp tục vì media được xem là thành phần tăng cường chứ không phải điều kiện bắt buộc để hoàn thành artifact.

ContentAssessmentAgent chịu trách nhiệm sinh phần đánh giá nhúng (*embedded assessment*) thay vì tạo một bài kiểm tra độc lập. Đầu vào của tác tử này là chủ đề và ngữ cảnh bài học; đầu ra là `quiz_payload` chứa tập câu hỏi có thể gắn vào slide luyện tập hoặc vào phần cuối giáo án. Về mặt thiết kế, đây không phải là một năng lực tách biệt hoàn toàn khỏi nhánh quiz độc lập, mà là cơ chế tái sử dụng năng lực sinh câu hỏi đã có của hệ thống trong một ngữ cảnh artifact dài hơn. Khác biệt

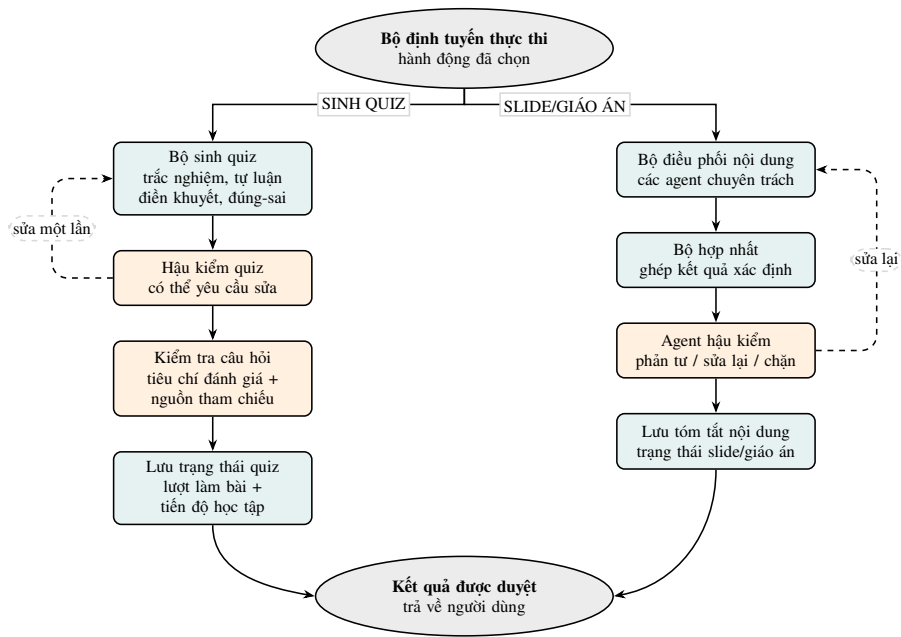
chủ yếu nằm ở mục tiêu sử dụng và dạng đóng gói đầu ra: ở nhánh quiz độc lập, kết quả được giữ như một workflow riêng để phục vụ làm bài, chấm đáp án và theo dõi tiến độ; còn trong nhánh slide hoặc giáo án, cùng loại đầu ra câu hỏi được rút gọn và nhúng vào artifact như một thành phần sơ phạm hỗ trợ. Cách tổ chức này phù hợp với mục tiêu sử dụng thực tế của slide bài giảng, nơi thành phần đánh giá thường chỉ là một phần nhỏ trong artifact tổng thể. Tương tự media, nếu bước này không thành công thì supervisor vẫn có thể hoàn tất workflow bằng cách bỏ qua hoặc giảm nhẹ phần đánh giá.

3.5.4 Tổng hợp kết quả, kiểm soát chất lượng và con người trong vòng lặp

Sau khi các specialist agent hoàn thành phần việc của mình, hệ thống không giao cho một mô hình ngôn ngữ khác nhiệm vụ “viết lại toàn bộ” artifact cuối. Thay vào đó, bước hợp nhất được thực hiện bởi một dịch vụ xác định (*deterministic service*) là *SlideMerger*. Dịch vụ này lấy `outline_payload` làm khung chuẩn, sau đó lần lượt ghép nội dung chi tiết, media và câu hỏi tương ứng vào từng slide theo `slide_id`. Cách hợp nhất này giúp bảo toàn cấu trúc đã được lập kế hoạch từ trước, đồng thời hạn chế nguy cơ một lượt gọi mô hình cuối cùng làm thay đổi ngoài ý muốn thứ tự hoặc nội dung của các phần đã được specialist tạo ra.

Về mặt chất lượng, hệ thống áp dụng hai lớp kiểm soát bổ sung nhau. Lớp thứ nhất là các kiểm tra cấu trúc mang tính xác định, chẳng hạn kiểm tra sự hiện diện của các loại slide tối thiểu, kiểm tra việc thiếu tiêu đề hoặc thiếu `source_chunk_ids` ở các slide cần grounding. Lớp thứ hai là *QualityReviewerAgent*, một tác tử hậu kiểm dùng mô hình ngôn ngữ để đánh giá artifact đầu ra theo rubric tổng quát và trả về `quality_review`. Kết quả hậu kiểm không chỉ gồm điểm số và các vấn đề phát hiện được, mà còn chỉ ra *reflection action* tương ứng, ví dụ chấp nhận, yêu cầu sửa outline, yêu cầu sửa content, yêu cầu sửa phần đánh giá hoặc chặn kết quả để cần người duyệt.

Ngoài hậu kiểm tự động, pipeline còn tích hợp *human-in-the-loop* tại bước outline. Sau khi *PedagogyPlannerAgent* sinh dàn ý ban đầu, workflow có thể tạm dừng ở trạng thái *interrupt*. Người dùng được phép duyệt nguyên trạng hoặc chỉnh sửa outline trước khi supervisor tiếp tục gọi bước viết nội dung. Điểm chèn con người tại đây là hợp lý vì outline là nơi quyết định cấu trúc sơ phạm tổng thể; một chỉnh sửa sớm ở giai đoạn này thường hiệu quả hơn so với can thiệp sau khi toàn bộ slide hoặc giáo án đã được sinh xong.



Hình 3.8: Hai nhánh sinh nội dung và cơ chế hậu kiểm chất lượng của hệ thống

Hình 3.8 mô tả mối quan hệ giữa nhánh sinh nội dung và cơ chế hậu kiểm. Trong nhánh slide/giáo án, bộ điều phối nội dung gọi lần lượt các specialist, chuyển kết quả sang khối hợp nhất, rồi mới đưa artifact cuối qua lớp hậu kiểm chất lượng. Mũi tên nét đứt quay ngược từ khối hậu kiểm về bộ điều phối biểu diễn *reflection loop*: nếu reviewer xác định đầu ra cần chỉnh sửa, supervisor sẽ phát sinh một *revision_instruction* và kích hoạt lại đúng bước liên quan thay vì chạy lại toàn bộ pipeline từ đầu. Giới hạn số vòng phản tư được đặt nhỏ để kiểm soát chi phí và độ trễ.

3.5.5 Tái sử dụng kiến trúc cho các tác vụ sinh nội dung

Một mục tiêu thiết kế quan trọng của tầng sinh nội dung là không xây dựng hai pipeline tách rời cho slide và giáo án. Thay vào đó, hệ thống dùng chung một kiến trúc đa tác tử, trong đó khác biệt giữa hai tác vụ được điều khiển chủ yếu bởi biến `task_type`. Giá trị này được truyền xuyên suốt từ đầu vào của *ContentPipelineInput*, vào *workflow state*, rồi đến từng *AgentTask*. Nhờ đó, cùng một supervisor và cùng một tập specialist có thể thích nghi đầu ra cho hai loại artifact mà không cần nhân đôi logic orchestration.

Việc tái sử dụng này thể hiện ở nhiều mức. Thứ nhất, cả hai tác vụ đều dùng chung tầng truy hồi, cùng cơ chế tổng hợp ngữ cảnh và cùng giao thức *AgentTask/AgentTaskResult*. Thứ hai, các specialist dùng chung hầu hết prompt và contract dữ liệu, chỉ thay đổi một số ràng buộc theo `task_type`, chẳng hạn slide thường yêu cầu ngắn gọn hơn và ưu tiên phần luyện tập nhúng, trong khi giáo án có thể cần cấu trúc section dài hơn. Thứ ba, năng lực sinh quiz không bị tách rời khỏi

nhánh sinh nội dung mà được tái sử dụng dưới dạng *embedded assessment*, cho phép cùng một nền sinh câu hỏi phục vụ cả workflow quiz độc lập lẫn workflow slide/giáo án. Thứ tư, bước hậu kiểm và phản tư vẫn giữ nguyên logic, chỉ thay reviewer hoặc rubric tương ứng với loại artifact cần đánh giá.

Khác biệt chủ yếu nằm ở giai đoạn đóng gói đầu ra. Với `task_type=slide`, artifact sau khi hợp nhất có thể được chuyển tiếp sang *SlideExportService* để xuất tệp PPTX. Với `task_type=lesson_plan`, pipeline vẫn dùng chung các bước sinh nội dung và hậu kiểm, nhưng không cần qua lớp xuất bản slide. Cách tổ chức này cho phép tái sử dụng phần lớn hạ tầng sinh nội dung, đồng thời vẫn giữ đủ linh hoạt để mở rộng thêm các loại artifact mới trong tương lai mà không phải thiết kế lại toàn bộ workflow.

3.6 Phương pháp đánh giá

3.6.1 Đánh giá truy hồi

Trong đề án này, retrieval là thành phần được đánh giá định lượng chính vì đây là bước quyết định chất lượng ngữ cảnh đầu vào cho toàn bộ pipeline RAG. Phương pháp đánh giá được tổ chức dưới dạng một benchmark hỏi đáp gắn với dữ liệu SGK, trong đó mỗi câu hỏi được liên kết với một bài học đích thông qua *lesson key*. Cách tổ chức này phù hợp với đặc thù dữ liệu của đề án: một bài học thường được chia thành nhiều chunk, nên điều cần kiểm tra trước hết là hệ thống có truy xuất đúng bài học mục tiêu hay không, thay vì yêu cầu gán nhãn relevance chi tiết cho toàn bộ chunk trong kho tri thức. Bộ benchmark này được xây dựng từ chính kho tri thức SGK đã xử lý và được dùng thống nhất cho các cấu hình retrieval khác nhau.

Với mỗi truy vấn kiểm thử, một chunk được xem là đúng nếu metadata của nó khớp với bài học và nằm trong các trường phạm vi như bộ sách, lớp, chủ đề và bài học. Trên cơ sở đó, đề án sử dụng các độ đo $\text{Hit}@k$, $\text{Recall}@k$, $\text{Precision}@k$, $\text{MRR}@k$ và $\text{nDCG}@k$ để đánh giá chất lượng retrieval tại các mức k khác nhau. Các độ đo này được dùng nhằm phản ánh ba khía cạnh bổ sung nhau: khả năng tìm thấy bài học đúng trong top- k , vị trí xuất hiện của kết quả đúng trong danh sách xếp hạng, và mức độ tập trung của các kết quả liên quan ở các thứ hạng đầu.

Benchmark retrieval của đề án gồm tập câu hỏi kiểm thử được sinh bám theo nội dung SGK, kèm metadata phạm vi và bài học đích để đối chiếu. Bộ benchmark này bao phủ nhiều nhóm truy vấn như *definition*, *fact*, *list*, *compare*, *process* và *application*, qua đó cho phép quan sát chất lượng retrieval dưới nhiều kiểu diễn đạt khác nhau của học sinh. Đồng thời, benchmark được xây dựng theo hướng bao phủ toàn bộ các seed đã chọn từ kho tri thức, nên tập truy vấn kiểm thử không chỉ tập trung vào một vài bài đơn lẻ mà trải đều trên toàn bộ hai bộ sách, ba khối lớp và

các chủ đề khác nhau trong chương trình. Nhờ đó, kết quả đánh giá retrieval phản ánh được hành vi của hệ thống trên phạm vi dữ liệu đầy đủ của đề án, thay vì chỉ trên một tập con hẹp hoặc một vài bài học tiêu biểu.

Metric	Ý nghĩa	Mục đích sử dụng
Hit@ k	Kiểm tra xem có ít nhất một kết quả đúng xuất hiện trong top- k hay không	Đo khả năng truy xuất trúng bài học mục tiêu ở các mức cắt khác nhau
Recall@ k	Tỷ lệ kết quả liên quan được thu hồi trong top- k	Đánh giá độ bao phủ của retriever đối với các chunk thuộc bài học đúng
Precision@ k	Tỷ lệ kết quả đúng trong top- k	Đánh giá mức độ nhiễu của danh sách truy xuất
MRR@ k	Nghịch đảo thứ hạng của kết quả đúng đầu tiên	Đo mức độ hệ thống đưa kết quả đúng lên sớm đến đâu
nDCG@ k	Độ lợi tích lũy có chuẩn hóa theo thứ hạng	Đánh giá chất lượng sắp xếp tổng thể của danh sách retrieval

Bảng 3.5: Các độ đo dùng cho đánh giá retrieval trong đề án

Để tách riêng ảnh hưởng của retriever và reranker, quy trình đánh giá retrieval được chia thành hai cấu hình. Cấu hình *no-rerank* đo trực tiếp chất lượng của khối truy xuất ban đầu sau bước hợp nhất candidate. Cấu hình *with-rerank* dùng cùng tập candidate nhưng đánh giá lại sau khi áp dụng cross-encoder reranker. Cách tách này giúp xác định rõ reranker có thực sự cải thiện thứ hạng của các chunk liên quan hay không, đồng thời cho phép so sánh công bằng giữa hai giai đoạn trước và sau xếp hạng lại trên cùng một tập truy vấn kiểm thử. Các số liệu thực nghiệm tương ứng được trình bày ở Chương 4.

3.6.2 Đánh giá chất lượng sinh

Khác với retrieval, chất lượng generation không được đánh giá chủ yếu bằng một độ đo duy nhất ở mức chunk hay lesson, mà được xem xét theo mức độ đúng nội dung, mức độ bám nguồn và mức độ phù hợp với câu hỏi của người học. Trong đề án, phần đánh giá này được thực hiện theo hai hướng bổ trợ nhau. Hướng thứ nhất là sử dụng kết quả đánh giá tự động đã có sẵn bằng RAGAS để quan sát các tiêu chí như *faithfulness*, *answer relevancy*, *context precision* và *context recall*. Hướng thứ hai là đọc và phân tích đầu ra theo rubric thủ công khi cần xem xét sâu hơn các

trường hợp lỗi hoặc các mẫu câu trả lời đặc thù.

Benchmark generation được tổ chức ở mức cặp *câu hỏi – câu trả lời tham chiếu*, kèm theo ngữ cảnh truy xuất mà hệ thống đã sử dụng để sinh đáp án. Cách tổ chức này cho phép đánh giá không chỉ chất lượng đầu ra cuối cùng mà còn cả mối quan hệ giữa chất lượng câu trả lời và mức độ phù hợp của context được đưa vào mô hình.

Metric	Ý nghĩa	Mục đích sử dụng
Faithfulness	Mức độ câu trả lời bám vào ngữ cảnh đã truy xuất	Kiểm tra nguy cơ mô hình bịa thêm thông tin ngoài nguồn
Answer Relevancy	Mức độ câu trả lời đi đúng trọng tâm câu hỏi	Đánh giá tính phù hợp của đáp án với nhu cầu hỏi đáp của người học
Context Precision	Mức độ các chunk quan trọng được ưu tiên trong context	Đánh giá chất lượng ngữ cảnh từ góc nhìn hỗ trợ generation
Context Recall	Mức độ context bao phủ thông tin cần thiết cho đáp án	Kiểm tra xem retriever đã cung cấp đủ cơ sở để sinh câu trả lời hay chưa

Bảng 3.6: Các độ đo dùng cho đánh giá chất lượng sinh trong đồ án

Điểm cần nhấn mạnh là node *validate* hoặc *quality reviewer* xuất hiện trong pipeline runtime không được dùng như bộ chấm điểm chính cho generation trong thực nghiệm. Vai trò của chúng là kiểm soát đầu ra khi hệ thống vận hành, còn đánh giá trong luận văn vẫn phải dựa trên benchmark, tiêu chí đã xác định trước và kết quả kiểm chứng tách biệt với pipeline chạy thực tế. Nhờ đó, phần đánh giá generation giữ được ranh giới rõ giữa cơ chế kiểm soát runtime và bằng chứng thực nghiệm.

3.7 Kết chương

Chương này đã trình bày phương pháp đề xuất của đồ án theo ba lớp chính: nền tảng tri thức được xây dựng từ dữ liệu SGK đã chunking và lập chỉ mục; lõi truy hồi ngữ cảnh dựa trên hybrid retrieval, reranking và truy hồi thích ứng; cùng với tầng điều phối tác tử và tầng sinh nội dung đa tác tử cho các tác vụ phức tạp hơn. Bên cạnh đó, chương cũng xác định cách đánh giá hai thành phần trọng tâm của hệ thống là retrieval và generation, trong đó retrieval được xem là bằng chứng định lượng chính, còn generation được đánh giá theo mức độ bám nguồn và mức độ phù hợp với câu hỏi.

Trên cơ sở phương pháp đã mô tả, Chương 4 sẽ trình bày cách hiện thực hóa hệ thống, quy trình xây dựng benchmark, các kiểm thử chức năng và những kết quả thực nghiệm đã thu được. Đây là phần dùng để kiểm chứng mức độ hiệu quả của giải pháp trên dữ liệu SGK Tin học THPT trong điều kiện cài đặt cụ thể của đề án.

CHƯƠNG 4. CÀI ĐẶT, KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM

Chương 4 trình bày cách hiện thực hóa giải pháp đã đề xuất ở Chương 3, đồng thời báo cáo quy trình kiểm thử và các kết quả thực nghiệm đã thu được. Khác với Chương 3 vốn tập trung vào phương pháp, chương này chỉ giữ lại mức chi tiết cài đặt vừa đủ để chứng minh hệ thống đã được triển khai và vận hành trên dữ liệu thực. Trọng tâm của chương đặt vào hai node lõi của pipeline là *retrieve node* và *generate node*: cách hai node này được cài đặt, cách xây dựng hai bộ dữ liệu kiểm thử tương ứng, và kết quả đánh giá định lượng của từng node. Một phần được trình bày kỹ là quy trình tạo bộ dữ liệu kiểm thử, bởi tính tin cậy của mọi số liệu trong chương phụ thuộc trực tiếp vào việc bộ dữ liệu được sinh ra một cách có kiểm soát, truy vết được và không bị.

4.1 Môi trường và cấu hình thực nghiệm

Hệ thống được cài đặt chủ yếu bằng Python. Tầng điều phối nội dung cho tác vụ sinh slide và giáo án được xây dựng trên LangGraph, trong khi các thành phần còn lại như xử lý ngữ cảnh, định tuyến ý định, lập kế hoạch hành động, truy xuất tri thức và ghi vết thực thi được hiện thực bằng logic xác định trong mã nguồn. Cách tổ chức này giúp giữ chi phí suy luận ở mức hợp lý và tạo điều kiện thuận lợi cho việc phân tích lỗi theo từng bước của pipeline.

Về mô hình và hạ tầng tri thức, hệ thống sử dụng Google Gemini làm mô hình sinh ngôn ngữ, mô hình embedding tiếng Việt `dangvantuan/vietnamese-document-embedding` để biểu diễn chunk, mô hình reranker `AITeamVN/Vietnamese_Reranker` cho bước xếp hạng lại và Qdrant làm vector database. Bên cạnh *vector search*, hệ thống còn duy trì một kênh *keyword search* dựa trên BM25 tự cài đặt, sau đó hợp nhất hai danh sách kết quả bằng Reciprocal Rank Fusion. Các thành phần cài đặt chính của môi trường thực nghiệm được liệt kê trong Bảng 4.1.

Thành phần	Cấu hình sử dụng
Ngôn ngữ lập trình	Python
Mô hình sinh ngôn ngữ	Google Gemini <code>gemini-2.5-flash-lite</code>
Mô hình embedding	<code>dangvantuan/vietnamese-document-embedding</code>
Mô hình reranker	<code>AITeamVN/Vietnamese_Reranker</code>
Vector database	Qdrant
Pipeline đa tác tử	LangGraph
Retriever lexical	BM25 tự cài đặt
Retriever ngữ nghĩa	Dense retrieval theo cosine similarity

Bảng 4.1: Môi trường phần mềm và các thành phần cốt lõi của hệ thống thực nghiệm

Bảng 4.1 cho thấy môi trường thực nghiệm được tổ chức nhất quán với kiến trúc đã mô tả ở Chương 3: lớp biểu diễn tri thức dùng embedding tiếng Việt và Qdrant, lớp truy xuất dùng đồng thời BM25 và dense retrieval, còn lớp sinh nội dung dựa trên Gemini. Cấu hình này cho phép tách riêng vai trò của từng thành phần khi phân tích kết quả thực nghiệm ở các mục sau.

Để các thí nghiệm có thể tái lập, các tham số chạy chính được dùng thống nhất trong toàn bộ chương được trình bày trong Bảng 4.2.

Tham số	Giá trị
Kích thước search pool	50 candidate
Số candidate sau hợp nhất	25 candidate
Các mức k khi tính độ đo	$\{1, 3, 5, 10\}$
Temperature của mô hình sinh	0,1
Temperature của mô hình chấm RA-GAS	0

Bảng 4.2: Các tham số chạy chính dùng cho thực nghiệm retrieval và generation

Bảng 4.2 cho thấy bước truy xuất lấy một *search pool* gồm 50 candidate từ hai kênh lexical và dense rồi hợp nhất còn 25 candidate; các độ đo retrieval được tính ở các mức $k \in \{1, 3, 5, 10\}$. Ở bước sinh câu trả lời, *temperature* được đặt thấp để ưu tiên tính ổn định và bám nguồn. Ở bước đánh giá generation bằng mô hình chấm, *temperature* được đặt bằng 0 để kết quả chấm nhất quán giữa các lần chạy.

Nguồn dữ liệu của đồ án gồm SGK Tin học lớp 10, 11 và 12 thuộc hai bộ sách Cánh Diều và Kết Nối Tri Thức. Sau bước tiền xử lý và chunking, kho tri thức thu

được 2348 chunk, mỗi chunk mang metadata về bộ sách, lớp, chủ đề, bài học, loại nội dung và cấp phân cấp. Chính cấu trúc này cho phép hệ thống vừa truy xuất ở mức cục bộ cho câu hỏi ngắn, vừa hỗ trợ các tác vụ cần bao phủ một bài học hoàn chỉnh như sinh slide hoặc soạn giáo án.

4.2 Cài đặt hệ thống và pipeline

Tại mức cài đặt, hệ thống được chia thành bốn nhóm module chính. Nhóm thứ nhất là lớp tiếp nhận và điều phối yêu cầu, bao gồm các thành phần tạo *Request-Context*, phân tích ngữ cảnh hội thoại, định tuyến ý định, lập kế hoạch hành động và quản lý vòng đời session. Nhóm thứ hai là lớp dịch vụ nghiệp vụ, nơi triển khai các chức năng hỏi đáp, quiz, sinh slide và sinh giáo án. Nhóm thứ ba là lớp tri thức, phụ trách ingestion, chunking, embedding, indexing, retrieval, reranking và dựng ngữ cảnh đầu vào cho LLM. Nhóm cuối cùng là lớp thực nghiệm và đánh giá, gồm pipeline tạo benchmark retrieval, pipeline đánh giá generation theo RAGAS và các file raw output phục vụ phân tích lỗi.

Cách chia này nhằm giữ ranh giới rõ giữa phần điều phối hệ thống và phần nghiên cứu chính. Các module điều phối giúp hệ thống vận hành trên nhiều loại yêu cầu, nhưng phần đóng góp kỹ thuật được kiểm chứng trong chương này vẫn tập trung vào retrieve node và generate node. Do đó, Chương 4 không đi vào liệt kê toàn bộ package hay từng file cài đặt, mà chỉ trình bày đủ để người đọc hiểu dữ liệu đi qua những khối nào trước khi trở thành câu trả lời hoặc artifact cuối.

Pipeline xử lý tài liệu.

Pipeline xử lý tài liệu bắt đầu từ tập văn bản SGK đã được chuẩn hóa. Ở bước đầu, nội dung được làm sạch để loại bỏ nhiễu trình bày, sau đó được tách theo cấu trúc heading của từng bài học. Mỗi đoạn nội dung sau khi tách được gắn *breadcrumb*, *level* và metadata bài học tương ứng. Cách cài đặt này giúp chunk vẫn giữ được mối liên hệ với ngữ cảnh sư phạm của SGK, thay vì chỉ là các đoạn văn bản rời rạc.

Sau khi chunking, hệ thống sinh embedding trên trường `full_content` và nạp vector vào Qdrant. Song song với đó, các trường văn bản cần thiết cũng được lưu để phục vụ BM25 và để hiển thị nguồn khi phân tích kết quả retrieval. Việc lưu đồng thời cả metadata phân cấp lẫn vector embedding cho phép hệ thống lọc trước theo *scope* rồi mới thực hiện truy xuất, từ đó giảm rủi ro nhầm lẫn giữa các bài học có thuật ngữ gần nhau nhưng khác bộ sách hoặc khác khối lớp.

Điểm cần nhấn mạnh là chính kho chunk này về sau cũng là nguồn để sinh bộ benchmark retrieval. Mỗi câu hỏi kiểm thử được sinh từ một đoạn chunk cụ thể và

được gắn với bài học chứa đoạn đó, nên bộ câu hỏi đánh giá luôn nằm trong đúng phạm vi dữ liệu mà hệ thống thực sự lập chỉ mục. Quan hệ này được trình bày chi tiết ở Mục 4.3.

Pipeline hỏi đáp.

Khi người dùng gửi câu hỏi, hệ thống trước hết tạo một *RequestContext* để gom câu hỏi hiện tại, thông tin bộ sách hoặc lớp được chọn, lịch sử hội thoại gần và các trường debug cần thiết. Từ đây, *ContextAnalyzer* kiểm tra xem câu hỏi có phụ thuộc vào turn trước hay không; nếu có, hệ thống trích một phần ngữ cảnh hội thoại để làm giàu truy vấn. *IntentRouter* sau đó xác định loại yêu cầu, chẳng hạn hỏi đáp kiến thức, sinh quiz, sinh slide hoặc soạn giáo án.

Với các intent cần tri thức từ SGK, *ActionPlanner* tạo hành động tương ứng và chuyển cho *ExecutionDispatcher*. Ở nhánh hỏi đáp, dispatcher gọi *RAGService* để truy xuất ngữ cảnh (retrieve node), gửi ngữ cảnh này vào prompt sinh câu trả lời (generate node) và ghi lại trace của toàn bộ phiên xử lý. Với các tác vụ dài như slide hoặc giáo án, dispatcher không sinh đầu ra trực tiếp mà chuyển tiếp cho pipeline đa tác tử đã mô tả ở Chương 3. Nhờ vậy, phần thực thi vẫn thống nhất ở tầng orchestration, trong khi logic chuyên biệt của từng tác vụ được cô lập ở các service tương ứng.

Điểm quan trọng trong cài đặt hiện tại là hệ thống không xem mọi yêu cầu như một lời gọi LLM đơn lẻ. Thay vào đó, truy vấn được biến đổi có kiểm soát qua các bước phân tích ngữ cảnh, định tuyến, retrieval, rồi mới tới generation. Cách tổ chức này làm rõ hơn quan hệ nhân quả giữa lỗi retrieval và lỗi answer, vốn là điều cần thiết cho các thí nghiệm báo cáo ở phần sau của chương.

Retrieve node và reranking.

Trong cấu hình retrieval dùng cho thực nghiệm, retrieve node lấy một *search pool* gồm 50 candidate từ hai kênh lexical và dense, sau đó hợp nhất bằng RRF để thu được danh sách 25 candidate. Đây là đầu vào cho cấu hình đánh giá *no-rerank*. Ở cấu hình *with-rerank*, cùng tập candidate này được chuyển qua cross-encoder để chấm lại mức độ liên quan trước khi chọn ra top-*N* cuối cùng. Cơ chế lọc theo *scope* được áp dụng trước truy xuất khi câu hỏi hoặc session đã xác định được *book*, *grade* hoặc *topic*.

Một kết quả retrieval trả về cho mỗi candidate gồm phần nội dung chunk, điểm số liên quan, metadata bài học và thông tin nguồn. Bộ thông tin này vừa được dùng làm ngữ cảnh cho generate node, vừa được lưu lại trong raw output để phục vụ phân tích lỗi. Việc tách riêng bước retrieval và bước rerank trong cài đặt cho phép

đo lường đóng góp của reranker một cách độc lập, thay vì gộp chung vào một điểm số duy nhất.

Generate node.

Ở generate node, prompt đầu vào luôn gồm hai thành phần: câu hỏi của người dùng và tập ngữ cảnh đã truy xuất. Các chunk được nối lại bằng dấu phân tách rõ ràng trước khi đưa vào prompt, kèm chỉ dẫn để mô hình trả lời theo ngôn ngữ phù hợp với học sinh THPT và ưu tiên bám sát thông tin xuất hiện trong context hơn là suy luận mở rộng từ tri thức tham số. Khi ngữ cảnh không đủ hoặc không truy xuất được tài liệu phù hợp, mô hình được yêu cầu nêu rõ giới hạn thay vì tiếp tục sinh nội dung không có căn cứ.

Việc tách riêng retrieve node và generate node trong cài đặt là tiền đề để hai node được đánh giá bằng hai phương pháp khác nhau ở phần sau: retrieve node được đánh giá định lượng bằng benchmark *single-gold*, còn generate node được đánh giá bằng bộ độ đo RAGAS. Nhờ vậy, mỗi lớp bằng chứng phản ánh đúng khía cạnh mà nó đo, thay vì bị trộn lẫn trong một con số tổng hợp.

4.3 Xây dựng benchmark cho retrieve node

Phần này trình bày cách xây dựng benchmark retrieval dùng để đánh giá retrieve node. Vì độ tin cậy của toàn bộ kết quả thực nghiệm phụ thuộc trực tiếp vào chất lượng của bộ benchmark này, phần trình bày tập trung vào quy trình sinh, cơ chế kiểm soát chất lượng và khả năng truy vết nguồn gốc.

Bối cảnh và lý do xây dựng lại. Phiên bản benchmark đầu tiên được sinh bằng một bộ khuôn câu hỏi tắt định: hệ thống nhét cụm từ khóa vào một số mẫu câu cố định mà không dùng mô hình ngôn ngữ. Khi đánh giá, khoảng 95% câu hỏi rơi vào dạng template rỗng nghĩa, kéo Hit@1 xuống chỉ còn khoảng 0,43. Tuy nhiên, nhóm nhỏ các câu không thuộc template lại đạt Hit@10 bằng 1,00, cho thấy vấn đề nằm ở chất lượng câu hỏi chứ không phải ở khả năng truy xuất. Phát hiện này dẫn tới quyết định loại bỏ hoàn toàn bộ sinh template và xây dựng lại benchmark bằng một mô hình ngôn ngữ thật, với ràng buộc rằng mỗi câu hỏi phải *phân biệt được bài học* mà nó hỏi tới.

Quy trình sinh có đặc tả từng bước. Toàn bộ quy trình tạo benchmark được đặc tả thành một tài liệu chỉ dẫn từng bước trước khi cài đặt, bao gồm: nguyên tắc tái sử dụng phần chọn seed, hợp đồng đầu ra dạng JSON, tập điều kiện ràng buộc câu hỏi, các luật kiểm tra tự động, lược đồ bản ghi và tiêu chí nghiệm thu. Việc có đặc tả rõ ràng giúp quy trình lặp lại được và không phụ thuộc vào thao tác thủ công ngẫu hứng. Quy trình gồm ba bước, trong đó phân định rõ phần được thực

hiện bằng mã nguồn xác định và phần do mô hình ngôn ngữ đảm nhận.

Bước 1 — sinh câu hỏi bằng mô hình ngôn ngữ, có kiểm soát. Trước hết, một thủ tục chọn seed bằng mã nguồn xác định lựa chọn các đoạn chunk đại diện sao cho phủ đủ các bài học và cân bằng giữa các nhóm bộ sách và khối lớp. Với mỗi seed, mô hình Gemini đọc trực tiếp nội dung của đoạn và sinh hai câu hỏi, trả về dưới dạng JSON theo hợp đồng định sẵn. Mỗi câu hỏi bắt buộc tuân thủ các điều kiện: trả lời được chỉ bằng nội dung đoạn và không bịa thông tin ngoài đoạn; chứa ít nhất một thuật ngữ, cú pháp hoặc tên riêng đặc trưng của bài để không bài khác trả lời được; tự chứa về ngữ nghĩa, không tham chiếu mơ hồ như “đoạn trên” hay “bài này”; diễn đạt tự nhiên và không lặp lại các khuôn template đã bị loại; đa dạng về loại câu hỏi. Ngay sau khi mô hình trả về, một bước kiểm tra tự động xác định sẽ loại câu nếu không kết thúc bằng dấu hỏi, quá ngắn, khớp khuôn template, chứa từ mơ hồ, có từ khóa đặc trưng không xuất hiện trong nội dung đoạn, hoặc phần đáp án kỳ vọng trùng lặp với nội dung đoạn dưới ngưỡng quy định. Kết quả của mô hình được lưu cache theo định danh đoạn và phiên bản prompt, nên chạy lại cho kết quả như cũ mà không tốn thêm lời gọi mô hình.

Bước 2 — lọc bằng mã nguồn xác định. Tập câu hỏi thô được đưa qua bước lọc gồm khử trùng lặp và phát hiện rò rỉ tiêu đề. Phép kiểm tra rò rỉ tiêu đề loại bỏ những câu lặp lại tên bài hoặc breadcrumb ở mức n-gram, nhằm tránh trường hợp câu hỏi “ăn gian” bằng cách nhắc lại đúng tên bài học.

Bước 3 — kiểm tra cuối và lấy mẫu phân tầng bằng mã nguồn xác định. Bước cuối kiểm tra lại từng câu theo các tiêu chí: tự chứa về ngữ nghĩa, từ khóa đặc trưng nằm trong nội dung đoạn gốc, và phần đáp án kỳ vọng có độ trùng từ với nội dung đoạn không thấp hơn ngưỡng. Các câu đạt sau đó được lấy mẫu phân tầng theo nhóm bộ sách, bài học và loại câu hỏi để giữ độ phủ. Cần nói rõ rằng bước này hoàn toàn dựa trên luật xác định và không gọi mô hình ngôn ngữ; do đó yêu cầu “phân biệt được bài học” được bảo đảm ngay từ Bước 1 thông qua ràng buộc khi sinh và bước kiểm tra đi kèm, chứ không phải bằng một lớp mô hình kiểm tra lại ở Bước 3.

Khả năng truy vết và chứng thực nguồn gốc. Mỗi câu hỏi trong benchmark được gắn các trường cho phép truy ngược về nguồn: định danh đoạn chunk gốc đã sinh ra câu hỏi, khóa bài học vàng gồm bộ sách, lớp, chủ đề, bài và tên bài, cùng thông tin về bộ sinh và phiên bản prompt. Trong bản ghi gốc còn lưu toàn văn nội dung đoạn và một mã băm nội dung của đoạn đó. Bộ ba gồm định danh đoạn, mã băm nội dung và thông tin bộ sinh cho phép kiểm chứng độc lập: người đánh giá có thể mở đúng đoạn chunk, đối chiếu mã băm để xác nhận nội dung không bị sửa,

rồi đọc đoạn để xác nhận câu hỏi thực sự trả lời được bằng đoạn đó. Đây là cơ sở để khẳng định bộ dữ liệu được sinh từ chính kho tri thức của hệ thống, có thể kiểm chứng, chứ không phải được bịa ra. Lược đồ một câu hỏi trong benchmark được minh họa ở Hình 4.1.

```
{
  "query_id": "V3A-0001",
  "query": "Sau khi mo IDLE (Python 3.9 64-bit), cua so Shell
           cho phép người học thực hiện thao tác gì ...?",
  "question_type": "fact",           // fact|definition|process|
                                     list|application|compare
  "difficulty": "easy",              // easy|medium|hard
  "anchor": "IDLE (Python 3.9 64-bit)", // cum tu khoa, phai co
                                     trong doan goc
  "seed_chunk_id": "59e9899d-...",  // dinh danh doan SGK
                                     goc
  "gold_lesson_key": {               // nhan vang muc bai
    hoc
    "book": "CD", "grade": "10",
    "topic_name": "...", "lesson": "Bai 1", "lesson_name": "..."
  }
}
```

Hình 4.1: Lược đồ một câu hỏi trong benchmark retrieval và các trường phục vụ truy vết nguồn gốc

Quy mô và phân bố. Quá trình sinh bắt đầu từ 300 seed, cho ra 600 câu hỏi và bao phủ toàn bộ 185 bài học trong tập dữ liệu. Bộ câu hỏi được cân bằng tuyệt đối theo nhóm bộ sách và khối lớp, mỗi nhóm đúng 100 câu. Bước lọc xác định ở Bước 2 không loại bỏ thêm mẫu nào, tức toàn bộ 600 câu đều đạt yêu cầu tối thiểu nhờ chất lượng đã được kiểm soát từ Bước 1. Phân bố theo nhóm và phân bố theo loại câu hỏi, độ khó được trình bày lần lượt trong Bảng 4.3 và Bảng 4.4.

Nhóm dữ liệu	Số lượng câu hỏi	Số bài được bao phủ
CD-10	100	185
CD-11	100	
CD-12	100	
KNTT-10	100	
KNTT-11	100	
KNTT-12	100	
Tổng cộng	600	185

Bảng 4.3: Phân bố bộ benchmark retrieval theo bộ sách và khối lớp

Bảng 4.3 cho thấy bộ benchmark được cân bằng tuyệt đối theo sáu nhóm bộ sách – khối lớp, mỗi nhóm gồm đúng 100 câu hỏi. Đồng thời, 600 câu hỏi này bao phủ được 185 bài học, cho thấy tập kiểm thử không chỉ tập trung vào một số bài tiêu biểu mà trải rộng gần như toàn bộ kho tri thức dùng trong thực nghiệm.

Loại câu hỏi	Số lượng	Độ khó	Số lượng
Định nghĩa (definition)	153	Dễ (easy)	326
Sự kiện (fact)	150	Trung bình (medium)	274
Vận dụng (application)	141		
So sánh (compare)	81		
Liệt kê (list)	42		
Quy trình (process)	33		

Bảng 4.4: Phân bố bộ benchmark retrieval theo loại câu hỏi và độ khó

Bảng 4.4 cho thấy benchmark nghiêng về các dạng câu hỏi định nghĩa, sự kiện và vận dụng, tức là những dạng gần với hành vi hỏi đáp thực tế của học sinh. Về độ khó, tập dữ liệu chỉ gồm hai mức dễ và trung bình, phản ánh chủ đích ưu tiên các câu hỏi có thể trả lời trực tiếp từ nội dung SGK thay vì các câu suy luận nhiều bước ngoài phạm vi tài liệu.

Định nghĩa độ liên quan. Bộ benchmark dùng chuẩn *single-gold* ở mức bài học: mỗi câu hỏi gắn với đúng một bài học vàng. Một chunk được tính là đúng nếu có cùng bộ sách, lớp, chủ đề và tên bài với khóa bài học vàng. Việc dùng khóa bài học làm chuẩn so khớp, thay vì gán nhãn relevance chi tiết cho từng chunk, cho phép mở rộng benchmark trên toàn bộ kho SGK mà vẫn giữ được tính nhất quán. Hệ quả của lựa chọn này là các độ đo dạng phủ như Recall và nDCG sẽ thấp về giá trị tuyệt đối, vì trong cùng một bài có nhiều chunk đều được tính đúng; do đó các độ đo phản ánh đúng nhất chất lượng của retrieve node là $\text{Hit}@k$ và MRR.

4.4 Xây dựng dữ liệu cho generate node

Để đánh giá generate node, đề án sử dụng một tập mẫu hỏi đáp gồm 246 mẫu. Mỗi mẫu gồm bốn thành phần: câu hỏi của người dùng, tập ngữ cảnh mà retrieve node thực sự trả về, câu trả lời do generate node sinh ra, và một đáp án tham chiếu đi kèm câu hỏi. Quy trình tạo bộ dữ liệu này gồm các bước nối tiếp: với mỗi câu hỏi, hệ thống thực hiện truy xuất và rerank để lấy ngữ cảnh, đưa ngữ cảnh và câu hỏi vào generate node để sinh câu trả lời, rồi ghi lại đồng thời câu hỏi, ngữ cảnh, câu trả lời, đáp án tham chiếu và thời gian xử lý từng bước. Tập mẫu thu được sau

đó là đầu vào trực tiếp cho bước tính độ đo RAGAS ở Mục 4.10.

Cần phân biệt rõ bộ dữ liệu này với benchmark retrieval ở mục trước. Benchmark retrieval đo khả năng xếp đúng bài học vào top đầu và dùng khóa bài học làm chuẩn, trong khi tập mẫu hỏi đáp đo chất lượng của câu trả lời sinh ra và dùng đáp án tham chiếu làm cơ sở cho các độ đo dựa trên tham chiếu. Hai bộ phục vụ hai mục tiêu khác nhau và được báo cáo độc lập.

4.5 Kiểm thử chức năng

Ngoài đánh giá định lượng, hệ thống còn được kiểm thử ở mức chức năng để bảo đảm các bước cài đặt cơ sở vận hành đúng trước khi chạy benchmark. Các ca kiểm thử được chọn theo các điểm nút quan trọng của pipeline, từ ingestion dữ liệu, chunking, indexing đến scope filtering, retrieval, generation và trace logging. Các ca kiểm thử chính được liệt kê trong Bảng 4.5.

Mã	Chức năng	Kết quả mong đợi	Kết quả
TC01	Đọc dữ liệu SGK	Tài liệu được parse và làm sạch đúng định dạng đầu vào	Đạt
TC02	Chunking theo heading	Sinh chunk có metadata bài học và level nhất quán	Đạt
TC03	Embedding và indexing	Vector được tạo và nạp vào kho chỉ mục không lỗi	Đạt
TC04	Scope filtering	Truy xuất được giới hạn đúng theo sách hoặc lớp đã chọn	Đạt
TC05	Retrieval	Top- k chứa ngữ cảnh liên quan cho câu hỏi kiến thức	Đạt
TC06	Reranking	Chunk đúng được đưa lên thứ hạng cao hơn sau rerank	Đạt
TC07	Generation	Câu trả lời bám tài liệu và có thể ghi lại context debug	Đạt
TC08	Out-of-scope handling	Hệ thống ưu tiên báo thiếu thông tin thay vì bịa nội dung	Đạt
TC09	Multi-intent routing	Yêu cầu phức hợp được tách thành nhiều action hợp lệ	Đạt
TC10	Logging và trace	Mỗi request có debug step phục vụ phân tích lỗi	Đạt

Bảng 4.5: Các ca kiểm thử chức năng chính của hệ thống trước khi chạy thực nghiệm

Kết quả ở Bảng 4.5 cho thấy các khối nền tảng của hệ thống đã vận hành ổn định trước khi bước vào thực nghiệm. Đây là điều kiện cần để các số liệu retrieval và generation ở các mục tiếp theo có thể được diễn giải như kết quả của phương pháp, thay vì chỉ phản ánh lỗi cài đặt cơ bản của phần mềm.

4.6 Phương pháp tiến hành thí nghiệm

Phần này trình bày cách tổ chức thí nghiệm trước khi đi vào kết quả, gồm các cấu hình được so sánh, lý do lựa chọn, số lần chạy và căn cứ chọn tham số.

Các cấu hình so sánh và lý do chọn. Đồ án tiến hành ba thí nghiệm, tập trung vào hai node lõi. Thí nghiệm thứ nhất đánh giá retrieve node ở cấu hình cơ sở *no-rerank*, trong đó hệ thống chỉ dùng truy xuất lai (BM25 kết hợp dense, hợp nhất bằng RRF) mà không qua bước xếp hạng lại. Thí nghiệm thứ hai đánh giá retrieve node ở cấu hình *with-rerank*, bổ sung cross-encoder để xếp hạng lại cùng tập candidate. Thí nghiệm thứ ba đánh giá generate node bằng RAGAS. Cấu hình *no-rerank* được chọn làm *baseline* một cách có chủ đích: vì hai cấu hình retrieval dùng chung tập candidate đầu vào và chỉ khác nhau ở bước rerank, nên chênh lệch kết quả giữa chúng cô lập được đúng phần đóng góp của reranker. Đây là thiết kế so sánh kiểu *ablation*, phù hợp để trả lời câu hỏi nghiên cứu về giá trị thực tế của reranker, thay vì so với một hệ thống ngoài vốn khác biệt ở quá nhiều yếu tố và không thể quy kết nguyên nhân.

Số lần chạy. Mỗi cấu hình retrieval được chạy một lần trên toàn bộ 600 câu hỏi của benchmark, và thí nghiệm generation được chạy một lần trên 246 mẫu. Việc chạy một lần là hợp lý vì pipeline ở các thí nghiệm này về cơ bản mang tính tất định: bước truy xuất và xếp hạng lại cho kết quả ổn định với cùng đầu vào, còn mô hình chấm RAGAS được đặt ở nhiệt độ bằng 0 để loại bỏ phần lớn dao động ngẫu nhiên giữa các lần chấm. Do đó, lặp lại nhiều lần để lấy trung bình không làm thay đổi đáng kể kết luận, trong khi vẫn giữ được chi phí thực nghiệm ở mức hợp lý.

Căn cứ chọn tham số. Các tham số ở Bảng 4.2 được chọn theo các căn cứ sau. Các mức $k \in \{1, 3, 5, 10\}$ được dùng để vừa quan sát chất lượng ở thứ hạng cao nhất (Hit@1) vừa quan sát độ phủ trong một cửa sổ kết quả thực tế mà người dùng có thể xem (top-10). Kích thước *search pool* 50 candidate đủ rộng để bước rerank có không gian lựa chọn nhưng không làm chi phí cross-encoder tăng quá mức. Nhiệt độ sinh câu trả lời được đặt thấp ở mức 0,1 để ưu tiên tính ổn định và bám nguồn, phù hợp với ngữ cảnh giáo dục cần độ chính xác. Nhiệt độ mô hình chấm được đặt bằng 0 để kết quả đánh giá nhất quán giữa các lần chạy. Chuẩn so khớp *single-gold* ở mức bài học được chọn vì cho phép mở rộng benchmark trên toàn bộ kho SGK với chi phí gán nhãn thấp, như đã phân tích ở Mục 4.7.

Cách xử lý dữ liệu và môi trường. Toàn bộ thí nghiệm chạy trên cùng kho tri thức 2348 chunk và cùng cấu hình mô hình ở Bảng 4.1. Dữ liệu kiểm thử của hai node được xây dựng theo quy trình ở Mục 4.7, và kết quả raw của mỗi thí nghiệm được lưu lại theo từng truy vấn để phục vụ phân tích lỗi ở Mục 4.11.

4.7 Kết quả đánh giá retrieve node

Thực nghiệm retrieve node được chạy đầy đủ trên 600 câu hỏi của benchmark *single-gold* ở cả hai cấu hình: không có reranker (*no-rerank*) và có reranker (*with-rerank*). Mỗi câu hỏi được chấm tại các mức $k \in \{1, 3, 5, 10\}$ với các độ đo $\text{Hit}@k$, $\text{Recall}@k$, $\text{Precision}@k$, $\text{MRR}@k$ và $\text{nDCG}@k$. Vì benchmark dùng khóa bài học làm chuẩn đúng, các độ đo này phản ánh trực tiếp khả năng của retrieve node trong việc đưa đúng bài học mục tiêu lên các vị trí đầu của danh sách kết quả.

So sánh tổng thể giữa hai cấu hình. Kết quả tổng thể của hai cấu hình được trình bày trong Bảng 4.6.

Chỉ số	No-rerank	With-rerank	Δ
Hit@1	0,7617	0,8183	+0,0566
Hit@3	0,9367	0,9500	+0,0133
Hit@5	0,9683	0,9750	+0,0067
Hit@10	0,9817	0,9833	+0,0016
MRR@10	0,8484	0,8856	+0,0372
nDCG@10	0,4267	0,4169	-0,0098
Precision@10	0,3372	0,3133	-0,0239
Thời gian (600 câu, giây)	59,7	1895,5	$\times 31,8$

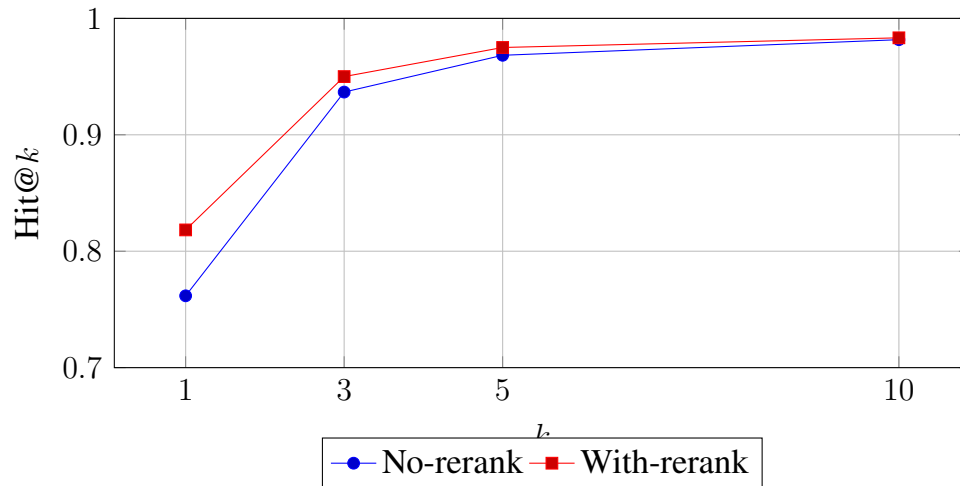
Bảng 4.6: So sánh kết quả retrieval tổng thể giữa cấu hình no-rerank và with-rerank trên 600 câu hỏi

Bảng 4.6 cho thấy reranker cải thiện rõ rệt ở nhóm thứ hạng đầu: Hit@1 tăng từ 0,7617 lên 0,8183 và MRR@10 tăng từ 0,8484 lên 0,8856. Mức cải thiện thu hẹp dần khi k tăng, và tới Hit@10 hai cấu hình gần như tương đương (0,9817 so với 0,9833). Điều này cho thấy reranker chủ yếu *sắp xếp lại* thứ hạng để đưa bài học đúng lên cao hơn, chứ không mở rộng độ phủ, bởi bài học đúng vốn đã nằm trong top-10 của retriever.

Hai độ đo nDCG@10 và Precision@10 giảm nhẹ khi bật reranker. Hiện tượng này là hệ quả trực tiếp của chuẩn *single-gold* ở mức bài học. Vì mọi chunk cùng bài đều được tính đúng, một retriever thô thường đưa nhiều chunk của cùng một bài vào top-10, khiến Precision@10 và nDCG@10 cao một cách hình thức. Khi reranker đưa chunk phù hợp nhất lên vị trí đầu nhưng đẩy bớt các chunk cùng bài xuống dưới, hai độ đo theo độ phủ này giảm, dù chất lượng ở thứ hạng đầu lại tăng. Vì vậy, với thiết kế benchmark hiện tại, Hit@1 và MRR là các độ đo phản ánh đúng

nhất hiệu quả của reranker, còn $nDCG@10$ và $Precision@10$ không phải là độ đo phù hợp để kết luận.

Hình 4.2 biểu diễn $Hit@k$ của hai cấu hình khi k tăng. Khoảng cách giữa hai đường rõ nhất tại $k = 1$ rồi thu hẹp dần và gần như khép lại tại $k = 10$, minh họa trực quan cho nhận định rằng reranker cải thiện thứ hạng đầu chứ không mở rộng độ phủ.



Hình 4.2: $Hit@k$ theo k của hai cấu hình trên toàn bộ 600 câu hỏi

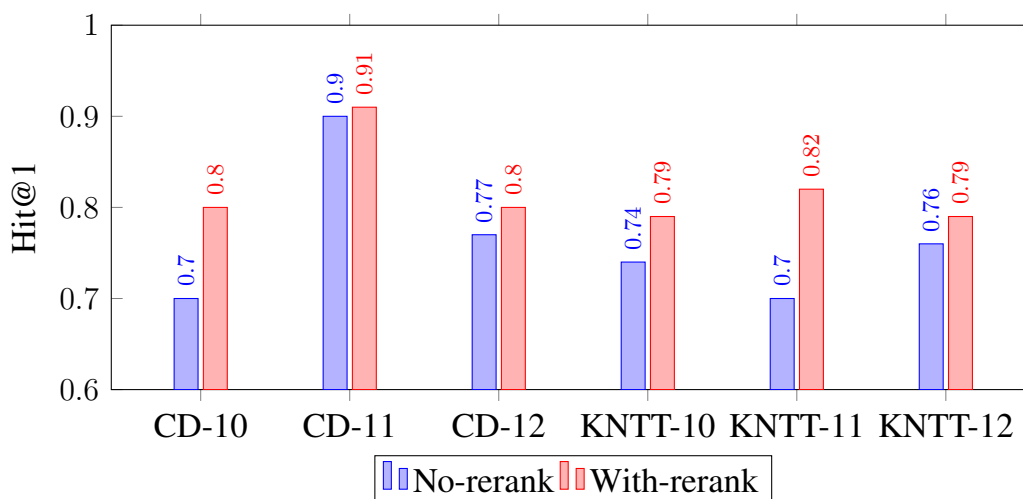
Kết quả theo từng nhóm bộ sách và khối lớp. Kết quả theo từng cặp bộ sách — khối lớp được trình bày trong Bảng 4.7.

Nhóm	Hit@1		MRR@10	
	No-rerank	With-rerank	No-rerank	With-rerank
CD-10	0,7000	0,8000	0,8086	0,8817
CD-11	0,9000	0,9100	0,9367	0,9464
CD-12	0,7700	0,8000	0,8427	0,8598
KNTT-10	0,7400	0,7900	0,8417	0,8690
KNTT-11	0,7000	0,8200	0,8132	0,8821
KNTT-12	0,7600	0,7900	0,8479	0,8745

Bảng 4.7: So sánh $Hit@1$ và $MRR@10$ theo từng nhóm bộ sách và khối lớp

Bảng 4.7 cho thấy reranker mang lại lợi ích lớn nhất ở chính những nhóm vốn yếu ở thứ hạng đầu: KNTT-11 tăng $Hit@1$ từ 0,70 lên 0,82 và CD-10 tăng từ 0,70 lên 0,80. Ngược lại, nhóm vốn đã mạnh như CD-11 gần như đã bão hòa nên mức tăng nhỏ. Xu hướng này cho thấy reranker đặc biệt hữu ích trong việc bù đắp cho các trường hợp mà retriever thô chưa đưa được bài học đúng lên đầu. Hình 4.3 trực

quan hóa sự chênh lệch này theo từng nhóm.



Hình 4.3: So sánh Hit@1 giữa hai cấu hình theo từng nhóm bộ sách và khối lớp

Chi phí tính toán. Lợi ích về độ chính xác ở thứ hạng đầu đi kèm một chi phí đáng kể về thời gian. Toàn bộ 600 câu hỏi được đánh giá trong khoảng 59,7 giây ở cấu hình no-rerank, tương đương xấp xỉ 0,10 giây mỗi truy vấn, nhưng tăng lên khoảng 1895,5 giây ở cấu hình with-rerank, tức gấp khoảng 31,8 lần. Phần lớn chi phí tăng thêm đến từ việc cross-encoder phải chấm lại từng cặp truy vấn — candidate. Đây là một đánh đổi cần cân nhắc khi triển khai thực tế: reranker đáng dùng cho mục tiêu nâng độ chính xác top-1, nhưng nên được áp dụng có chọn lọc, chẳng hạn xử lý theo lô, lưu đệm kết quả, hoặc chỉ kích hoạt khi cơ chế truy hồi thích ứng xác định truy vấn thực sự cần.

4.8 Kết quả đánh giá generate node theo RAGAS

Generate node được đánh giá định lượng bằng RAGAS, một khung đánh giá tự động theo hướng dùng mô hình ngôn ngữ làm giám khảo (*LLM-as-judge*) phổ biến cho các hệ thống RAG. Trong cấu hình của đề án, mô hình giám khảo là Google Gemini chạy ở nhiệt độ bằng 0 để bảo đảm tính nhất quán, kết hợp với mô hình embedding tiếng Việt cho các độ đo cần đối chiếu ngữ nghĩa. Bốn độ đo được sử dụng, mỗi độ đo nhắm vào một khía cạnh khác nhau của chất lượng câu trả lời, được liệt kê trong Bảng 4.8.

Độ đo	Ý nghĩa
Faithfulness	Mức độ câu trả lời bám vào ngữ cảnh được cung cấp, dùng để phát hiện thông tin bịa ngoài tài liệu.
Answer Relevancy	Mức độ câu trả lời đi đúng trọng tâm câu hỏi, không lan man hay thừa thông tin.
Context Precision	Mức độ các chunk giá trị nhất được xếp ở vị trí ưu tiên trong ngữ cảnh, đối chiếu với đáp án tham chiếu.
Context Recall	Mức độ ngữ cảnh truy xuất bao phủ được thông tin cần thiết so với đáp án tham chiếu.

Bảng 4.8: Bốn độ đo RAGAS dùng để đánh giá generate node

Bảng 4.8 cho thấy bốn độ đo được chọn bao phủ cả hai mặt của generate node: chất lượng câu trả lời cuối cùng và chất lượng ngữ cảnh mà câu trả lời dựa vào. Trong đó, Faithfulness và Answer Relevancy phản ánh trực tiếp hành vi sinh câu trả lời, còn Context Precision và Context Recall phản ánh mức độ hỗ trợ của ngữ cảnh truy xuất đối với bước sinh.

Kết quả. Kết quả đánh giá trên 246 mẫu hỏi đáp được trình bày trong Bảng 4.9.

Chỉ số đánh giá	Giá trị trung bình
Faithfulness	0,9757
Answer Relevancy	0,8571
Context Precision	0,8555
Context Recall	0,9593
Thời gian retriever (giây)	4,579
Thời gian generator (giây)	1,979
Tổng thời gian pipeline (giây)	6,558

Bảng 4.9: Kết quả đánh giá generate node theo RAGAS trên 246 mẫu

Bảng 4.9 cho thấy Faithfulness đạt 0,9757 và Context Recall đạt 0,9593, trong khi Answer Relevancy đạt 0,8571 và Context Precision đạt 0,8555. Về thời gian, trung bình mỗi mẫu mất 4,579 giây cho bước truy xuất, 1,979 giây cho bước sinh câu trả lời và 6,558 giây cho toàn bộ pipeline.

Kết quả ở Bảng 4.9 cho thấy hai điểm đáng chú ý. Thứ nhất, Faithfulness và Context Recall đều ở mức cao, cho thấy khi retrieve node cung cấp được ngữ cảnh phù hợp, generate node có xu hướng bám tốt vào nguồn và ít bịa thông tin ngoài

tài liệu — đúng với mục tiêu giữ tính bám nguồn đã đặt ra ở Chương 3. Thứ hai, Answer Relevancy và Context Precision thấp hơn một bậc, phản ánh rằng câu trả lời đôi khi còn thừa thông tin và chất lượng câu trả lời vẫn phụ thuộc vào việc sắp xếp đúng các chunk giá trị nhất ở đầu danh sách ngữ cảnh. Nhận xét này nhất quán với kết quả ở Mục 4.9: bài học đúng thường đã được tìm thấy, nhưng thứ hạng giữa các chunk trong cùng bài vẫn còn dư địa cải thiện, và đây chính là phần mà reranker đóng góp.

4.9 Phân tích lỗi

Để hiểu rõ giới hạn của hệ thống, các lỗi quan sát được trong quá trình thực nghiệm được phân theo từng lớp của pipeline. Ở lớp dữ liệu và chunking, một số bài học có nội dung gần nhau về thuật ngữ nhưng khác bộ sách hoặc khác khối lớp, dễ gây nhầm lẫn cho retriever khi không có bộ lọc scope; ngoài ra, việc cắt chunk theo heading đôi khi tách rời một ý trải dài qua nhiều đoạn. Ở lớp retrieval, các truy vấn ngắn hoặc dùng từ chung chung khó được phân biệt đúng bài, dẫn tới việc bài học đúng không được đưa lên vị trí đầu; đây cũng là nhóm trường hợp mà reranker giúp cải thiện rõ nhất theo kết quả ở Mục 4.9. Ở lớp reranking, vẫn tồn tại những trường hợp cross-encoder đánh giá sai mức liên quan và đẩy chunk đúng xuống thấp, dù trên tổng thể reranker cải thiện thứ hạng đầu.

Ở lớp generation, các lỗi chính gồm câu trả lời thừa thông tin làm giảm Answer Relevancy, và một số ít trường hợp suy luận mở rộng ngoài ngữ cảnh khi tài liệu không đủ. Cuối cùng, ở lớp đánh giá, chuẩn *single-gold* ở mức bài học khiến các độ đo theo độ phủ như nDCG@10 và Precision@10 không phản ánh đúng chất lượng thứ hạng đầu, như đã phân tích ở Mục 4.9; tương tự, các độ đo RAGAS dựa trên đáp án tham chiếu sẽ chịu ảnh hưởng bởi chất lượng của đáp án tham chiếu. Các mẫu cụ thể có điểm số thấp được lưu trong raw output của cả hai luồng đánh giá, cho phép truy lại từng trường hợp để phân tích sâu hơn khi cần.

4.10 Nhận xét tổng hợp kết quả thực nghiệm

Từ các kết quả trên, có thể rút ra một số nhận xét chính. Thứ nhất, hướng tiếp cận RAG trên dữ liệu SGK Tin học THPT đã chứng minh được tính khả thi: retrieve node đạt Hit@1 bằng 0,8183 và MRR@10 bằng 0,8856 ở cấu hình tốt nhất, trong khi generate node đạt Faithfulness bằng 0,9757 trên tập đánh giá RAGAS. Thứ hai, reranker bằng cross-encoder đóng góp rõ rệt vào chất lượng thứ hạng đầu, cải thiện Hit@1 thêm khoảng 5,7 điểm phần trăm và mang lại lợi ích lớn nhất ở các nhóm bộ sách vốn yếu, tuy phải trả giá bằng chi phí thời gian tăng khoảng 31,8 lần. Thứ ba, kết quả của hai node nhất quán với nhau: việc retrieve node đưa đúng bài học vào ngữ cảnh là nền tảng để generate node đạt độ trung thực cao, và phần thứ hạng

còn cải thiện được ở retrieval cũng chính là phần ảnh hưởng tới Answer Relevancy và Context Precision của generation.

Bên cạnh các kết quả đạt được, chương cũng chỉ ra những giới hạn cần lưu ý khi diễn giải số liệu. Benchmark retrieval sử dụng chuẩn *single-gold* ở mức bài học, nên các độ đo theo độ phủ không nên được dùng làm kết luận chính; chi phí của reranker còn cao và cần được tối ưu khi triển khai; và việc đánh giá generation dựa trên đáp án tham chiếu phụ thuộc vào chất lượng của đáp án đó. Những giới hạn này là cơ sở cho các hướng cải thiện sẽ được trình bày ở Chương 5 và Chương 6.

Tóm lại, Chương 4 đã trình bày việc hiện thực hóa hệ thống từ dữ liệu SGK đến pipeline hỏi đáp hoàn chỉnh, mô tả chi tiết cách xây dựng và chứng thực hai bộ dữ liệu kiểm thử, đồng thời báo cáo kết quả đánh giá riêng cho retrieve node và generate node. Các kết luận trong chương đều dựa trên những số liệu đã thực sự được chạy và kiểm chứng, và mỗi bộ dữ liệu kiểm thử đều có thể truy ngược về nguồn để kiểm tra độc lập.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Các chương trước đã trình bày nền tảng lý thuyết, phương pháp đề xuất và kết quả thực nghiệm của hệ thống. Chương này tập trung phân tích các giải pháp và đóng góp nổi bật của đề án dưới góc nhìn thiết kế kỹ thuật. Trọng tâm không nằm ở việc mô tả lại từng module, mà làm rõ những quyết định thiết kế có ý nghĩa thực tiễn, các vấn đề khó phát sinh khi xây dựng trợ lý giáo dục dựa trên LLM và cách hệ thống giải quyết các vấn đề đó bằng một kiến trúc có thể kiểm soát, dễ mở rộng và phù hợp với dữ liệu sách giáo khoa.

Các đóng góp chính gồm xây dựng nền tảng tri thức cho SGK phân cấp, kết hợp hybrid retrieval với adaptive retrieval cho truy vấn giáo dục, thiết kế tầng điều phối hỗ trợ multi-intent và hội thoại đa lượt, tổ chức pipeline sinh nội dung đa tác tử có kiểm soát, và đưa ra một số quyết định thiết kế giúp hệ thống đạt được tính thực dụng trong triển khai.

5.1 Xây dựng nền tảng tri thức dựa trên chunk phân cấp và metadata

5.1.1 Vấn đề cần giải quyết

Khác với nhiều bài toán hỏi đáp trên văn bản số hóa sẵn, dữ liệu đầu vào của đề án là các tệp PDF sách giáo khoa. Đây không phải là nguồn văn bản sạch có thể đưa trực tiếp vào bước chunking và lập chỉ mục. Nếu quá trình chuyển đổi PDF sang text làm sai nội dung, sai heading hoặc mất cấu trúc bài học, chất lượng của toàn bộ các bước phía sau như chunking, retrieval và generation đều bị ảnh hưởng.

Khó khăn lớn ở bước này là đặc thù của SGK Tin học THPT. Tài liệu chứa văn bản lý thuyết, tiêu đề phân cấp, bảng, sơ đồ, câu hỏi cuối bài và nhiều đoạn mã nguồn minh họa. Trong quá trình thử nghiệm ban đầu, một số hướng OCR self-host cho tiếng Việt cho kết quả chưa ổn định: sai ký tự, sai cấu trúc heading, xử lý kém với bảng và sơ đồ, đồng thời khó chuyển các đoạn mã nguồn sang dạng văn bản phù hợp với pipeline truy xuất. Vì vậy, bài toán ở đây không chỉ là nhận dạng ký tự, mà là chuẩn hóa được một corpus có cấu trúc để phục vụ tầng tri thức.

5.1.2 Giải pháp

Đề án tổ chức bước tiền xử lý dữ liệu theo hai pha. Pha thứ nhất chuyển từng trang PDF sang Markdown thô bằng một mô hình thị giác-ngôn ngữ có năng lực OCR mạnh hơn trên tiếng Việt. Trong prompt OCR, đầu ra được ràng buộc theo hướng giữ nguyên nội dung văn bản, loại bỏ ảnh và yếu tố trang trí, chuẩn hóa heading ở mức cơ bản, đồng thời thay các đoạn mã nguồn bằng mô tả chức năng ngắn gọn. Cách làm này phù hợp với định hướng của đề án là truy xuất trên văn

bản, không truy xuất trực tiếp trên ảnh.

Pha thứ hai chuẩn hóa lại Markdown thô để thu được cấu trúc heading nhất quán cho toàn bộ SGK. Ở bước này, các phần như mục chính, mục con, tóm tắt bài học, luyện tập và vận dụng được đưa về các cấp heading xác định; đồng thời watermark, header/footer và các dấu vết OCR được loại bỏ. Sau khi có văn bản sạch, hệ thống mới thực hiện chunking theo tiêu đề, gắn metadata như bộ sách, lớp, chủ đề, bài học, level, breadcrumb và lesson_key, rồi mới chuyển sang bước embedding và indexing.

Việc dùng mô hình bên thứ ba ở bước OCR là một quyết định thực dụng hơn là một đóng góp thuật toán. Thành phần này mang tính black-box và không phải trọng tâm nghiên cứu của đề án. Tuy nhiên, nó giải quyết được yêu cầu quan trọng nhất của dữ liệu đầu vào là bảo toàn đủ nội dung và cấu trúc để các bước xử lý phía sau hoạt động ổn định. Do đó, phần đóng góp của đề án không nằm ở chính mô hình OCR, mà ở cách tổ chức pipeline chuyển đổi PDF sang corpus Markdown có thể đưa vào hệ tri thức một cách nhất quán.

5.1.3 Đóng góp đạt được

Đóng góp nổi bật của phần này là biến một nguồn dữ liệu SGK ban đầu ở dạng PDF không đồng nhất thành một corpus có cấu trúc đủ tốt để phục vụ toàn bộ pipeline RAG. Việc đầu tư cho bước PDF-to-Markdown giúp giảm lỗi lan truyền xuống các tầng sau, đặc biệt là lỗi sai phạm vi bài học, sai heading và sai quan hệ giữa nội dung với metadata.

Bên cạnh đó, cách kết hợp giữa văn bản đã chuẩn hóa, chunk phân cấp và metadata thống nhất tạo ra một nền tảng tri thức có thể mở rộng. Đây là tiền đề để cùng một cơ sở dữ liệu có thể phục vụ đồng thời retrieval cho hỏi đáp, sinh quiz, sinh slide và sinh giáo án, thay vì phải xây dựng nhiều nguồn dữ liệu tách rời cho từng chức năng.

5.2 Kết hợp hybrid retrieval, reranking và adaptive retrieval

5.2.1 Vấn đề cần giải quyết

Dữ liệu sách giáo khoa có đặc điểm khác với dữ liệu web mở. Nhiều khái niệm quan trọng được trình bày ngắn, có thuật ngữ chuyên ngành, tên lệnh, tên bài học hoặc cấu trúc chương mục rõ ràng. Nếu chỉ dùng tìm kiếm ngữ nghĩa, hệ thống có thể bỏ sót các đoạn có từ khóa chính xác. Nếu chỉ dùng tìm kiếm từ khóa, hệ thống khó xử lý câu hỏi diễn đạt tự nhiên khác với văn bản trong sách.

Ngoài ra, không phải câu hỏi nào cũng cần cùng một cách truy xuất. Câu hỏi “giải thích khái niệm cơ sở dữ liệu” cần đoạn nội dung chi tiết, trong khi câu hỏi

“lớp 12 có những chủ đề nào” cần tổng hợp cấu trúc chương trình. Một pipeline RAG cố định dễ hoặc lấy thiếu ngữ cảnh, hoặc tốn chi phí không cần thiết.

5.2.2 Giải pháp

Hệ thống sử dụng truy xuất lai gồm tìm kiếm từ khóa BM25-like, tìm kiếm ngữ nghĩa bằng embedding tiếng Việt, hợp nhất thứ hạng bằng RRF và xếp hạng lại bằng reranker. BM25 giúp giữ độ nhạy với thuật ngữ chính xác, còn embedding giúp bắt được các câu hỏi diễn đạt theo cách tự nhiên. RRF được dùng để hợp nhất hai danh sách kết quả mà không yêu cầu điểm số của chúng nằm trên cùng một thang đo.

Trên lớp truy xuất cơ sở, đồ án xây dựng AdaptiveRAGAgent. Thành phần này chọn strategy dựa trên QueryClassifier heuristic, không dùng LLM để ra quyết định. Bốn strategy chính gồm STANDARD, BROAD, CURRICULUM và HIERARCHICAL. STANDARD dùng cho truy vấn cụ thể nhưng thiếu phạm vi rõ. BROAD dùng cho câu hỏi tổng quan. CURRICULUM ưu tiên metadata chương trình học. HIERARCHICAL dùng khi có đủ tín hiệu lớp/chủ đề để truy xuất theo cấp bài học rồi mới đi vào chunk chi tiết.

Hệ thống còn bổ sung scope fallback. Khi phạm vi sách/lớp chỉ đến từ UI và không đủ chunk phù hợp, RAGService có thể mở rộng phạm vi tìm kiếm, đồng thời ghi lại actual_scope, scope_fallback_notice và debug step tương ứng. Nhờ vậy, hệ thống không trả lời rỗng quá sớm nhưng vẫn minh bạch về phạm vi dữ liệu đã dùng.

5.2.3 Đóng góp đạt được

Đóng góp ở đây không chỉ là ghép nhiều kỹ thuật retrieval, mà là đặt chúng trong một cơ chế chọn strategy theo loại câu hỏi. Cách làm này phù hợp với bài toán giáo dục vì câu hỏi của học sinh và giáo viên có mức độ cụ thể rất khác nhau. Adaptive RAG giúp hệ thống cân bằng giữa độ chính xác, độ bao phủ và chi phí truy xuất.

Việc QueryClassifier là heuristic cũng là một quyết định có chủ đích. Thay vì gọi LLM thêm một lượt để chọn chiến lược truy xuất, hệ thống giữ bước này nhanh, có thể kiểm thử và có thể giải thích. Đây là điểm quan trọng trong một hệ thống cần ổn định hơn là thể hiện khả năng suy luận tự do.

5.3 Thiết kế tầng điều phối cho multi-intent và multi-turn conversation

5.3.1 Vấn đề cần giải quyết

Trong giao diện hội thoại, người dùng không luôn đưa ra một yêu cầu đơn lẻ. Một câu có thể chứa nhiều ý định, ví dụ yêu cầu tạo slide rồi thêm câu hỏi luyện

tập, hoặc vừa giải thích kiến thức vừa sinh bài tập. Nếu hệ thống chỉ chọn một intent duy nhất, một phần yêu cầu sẽ bị bỏ qua. Ngược lại, nếu để LLM tự thực thi toàn bộ chuỗi hành động, hệ thống dễ sinh ra luồng không nhất quán với các service đang có.

5.3.2 Giải pháp

`IntentRouter.detect_multi()` phân tích tối đa ba intent trong một câu hỏi. Mỗi intent gồm nhóm ý định chính, loại nhiệm vụ, chủ đề, lớp, bộ sách và một số metadata cần thiết. Sau đó, `ActionPlanner.plan_all()` chuyển danh sách intent thành danh sách action cụ thể như sinh quiz, sinh slide, sinh giáo án, chấm câu trả lời, ôn câu sai hoặc giải thích khái niệm.

Việc giới hạn số intent là một quyết định thực tế. Nó đủ để xử lý các câu lệnh ghép thường gặp, nhưng tránh để một câu nhập mơ hồ sinh ra quá nhiều action và làm phình state. Sau bước lập kế hoạch, `ExecutionDispatcher` chịu trách nhiệm gọi đúng service hoặc handler theo action đã xác định.

5.3.3 Đóng góp đạt được

Cơ chế này giúp hệ thống xử lý yêu cầu tự nhiên tốt hơn mà vẫn giữ được tính quyết định của backend. LLM được dùng để hiểu câu nói của người dùng, nhưng kế hoạch thực thi cuối cùng được đưa về cấu trúc action rõ ràng. Điều này tạo ra một ranh giới cần thiết giữa năng lực ngôn ngữ của LLM và logic nghiệp vụ của phần mềm.

5.4 Triển khai kiến trúc supervisor–specialist trong sinh nội dung

5.4.1 Vấn đề cần giải quyết

Sinh câu hỏi luyện tập có thể xuất hiện trong hai bối cảnh khác nhau. Bối cảnh thứ nhất là quiz standalone, nơi người dùng tạo bộ câu hỏi, trả lời, được chấm điểm, xem lại câu sai và theo dõi tiến độ. Bối cảnh thứ hai là assessment nhúng trong slide hoặc giáo án, nơi câu hỏi chỉ là một artifact phụ để củng cố bài giảng.

Nếu gộp hai bối cảnh này vào cùng một agent hoặc cùng một pipeline, hệ thống sẽ khó quản lý state. Quiz standalone cần lưu từng lượt làm bài, từng câu hỏi, câu trả lời của người dùng, số lần thử và kết quả chấm. Assessment trong slide lại chủ yếu cần nội dung phù hợp với bài học và format trình bày.

5.4.2 Giải pháp

Đồ án giữ `QuizService` là một service độc lập cho quiz tương tác. Service này quản lý vòng đời sinh câu hỏi, kiểm tra chất lượng, lưu `QuizRound`, lưu `QuestionRecord`, chấm câu trả lời, review câu sai và thống kê tiến độ. Các handler câu hỏi như trắc nghiệm, tự luận, điền khuyết và đúng/sai được giữ trong

pipeline quiz riêng.

Trong content multi-agent pipeline, `ContentAssessmentAgent` chỉ sinh assessment nhúng cho slide hoặc giáo án. Agent này không thay thế `QuizService` và không quản lý state làm bài của học sinh. Cách tách này giúp tránh nhầm lẫn giữa một bài kiểm tra tương tác và một phần hoạt động trong tài liệu giảng dạy.

5.4.3 Đóng góp đạt được

Đây là một đóng góp kiến trúc quan trọng vì nó giữ đúng ranh giới nghiệp vụ. Quiz standalone có thể phát triển thêm các chức năng học tập cá nhân hóa mà không ảnh hưởng đến pipeline sinh slide. Ngược lại, pipeline sinh slide/giáo án có thể tối ưu cho artifact dài mà không phải mang theo toàn bộ logic chấm điểm và theo dõi attempt.

5.5 Đóng góp thiết kế trong việc nâng cao tính kiểm soát, khả năng mở rộng và tính thực dụng của hệ thống

5.5.1 Vấn đề cần giải quyết

Tạo slide và giáo án là tác vụ dài hơn nhiều so với hỏi đáp hoặc sinh một bộ câu hỏi ngắn. Hệ thống phải xác định mục tiêu học tập, lập dàn ý, viết nội dung chi tiết, gợi ý media, thêm assessment, ghép kết quả và kiểm tra chất lượng. Nếu giao toàn bộ tác vụ này cho một prompt duy nhất, đầu ra dễ thiếu cấu trúc, bỏ sót phần quan trọng hoặc khó sửa khi người dùng muốn thay đổi dàn ý. Đồng thời, nếu mọi năng lực đều được nhúng trực tiếp vào một agent trung tâm, hệ thống sẽ khó kiểm soát, khó mở rộng và khó theo dõi lỗi khi đưa vào vận hành thực tế.

5.5.2 Giải pháp

Đề án dùng `LangGraph` cho phạm vi content pipeline của slide và giáo án. Ở cấp hệ thống, `Orchestrator` vẫn là controller mã nguồn; `LangGraph` chỉ điều phối luồng sinh nội dung dài. `ContentSupervisor` gọi các tool adapter tương ứng với các specialist agent như `PedagogyPlannerAgent`, `ContentDraftingAgent`, `MediaResearchAgent`, `ContentAssessmentAgent` và `QualityReviewerAgent`.

Các agent không trao đổi tự do mà đi qua contract A2A-lite. Supervisor tạo `AgentTask`, specialist agent trả `AgentTaskResult`. Task chứa mục tiêu, đầu vào, ràng buộc, context refs và artifact mong đợi. Kết quả chứa trạng thái, artifact, confidence, warnings, used tools và latency. Contract này giúp mỗi kết quả trung gian có chủ sở hữu rõ ràng và dễ ghi debug. Sau khi các artifact được tạo, `merge_results` ghép outline, content, media và assessment thành kết quả thống nhất; kết quả sau đó được đưa qua `QualityReviewerAgent` và reflection loop

để quyết định approve, revise hoặc block.

Song song với đó, hệ thống đặt HITL tại bước duyệt dàn ý nhằm đưa người dùng vào đúng điểm quyết định có ảnh hưởng lớn nhất. Về mặt vận hành, Session-Store, TraceService và các quality path riêng cho quiz, slide, giáo án được bổ sung để lưu trạng thái đa lượt, quan sát tiến trình xử lý và hỗ trợ kiểm thử. Một điểm mở rộng quan trọng khác là khả năng theo dõi tiến trình học tập thông qua việc quản lý student profile như một phần của state. State của hệ thống có thể giữ các lượt làm quiz, câu trả lời sai và mức độ hoàn thành theo chủ đề cho các lượt tương tác tiếp theo. Ngoài ra, một tool layer in-process được tổ chức như điểm mở rộng có kiểm soát để tích hợp retrieval, content formatting, media search và export artifact.

5.5.3 Đóng góp đạt được

Đóng góp chính của phần này là tổ chức tác vụ sinh nội dung dài thành các bước có artifact rõ ràng, đồng thời giữ được tính kiểm soát ở nhiều mức: kiểm soát cấu trúc sinh nội dung, kiểm soát vòng đời hội thoại, kiểm soát trạng thái học tập của người dùng và kiểm soát điểm mở rộng của hệ thống. Hệ thống không chỉ sinh một đoạn văn lớn, mà có dàn ý, nội dung, media metadata, assessment và review; người dùng cũng có thể can thiệp tại bước outline thay vì chỉ nhận kết quả cuối.

A2A-lite, HITL, quality path, session, student profile, trace và tool layer không phải các thành phần tách rời, mà cùng tạo nên một kiến trúc có khả năng mở rộng và vận hành thực tế. Đây là điểm khác biệt quan trọng của đồ án: kiến trúc được tổ chức theo hướng có thể kiểm soát, quan sát, điều chỉnh và mở rộng khi số lượng tác vụ và độ phức tạp của workflow tăng lên. Việc theo dõi tiến trình học tập thông qua state cũng mở ra nền tảng cho các cơ chế hỗ trợ học tập cá nhân hóa ở các phiên bản tiếp theo.

5.6 Kết chương

Chương này đã hệ thống hóa các giải pháp và đóng góp chính của đồ án. Các đóng góp không nằm ở một thuật toán riêng lẻ, mà ở cách kết hợp LLM với các lớp điều phối, truy xuất, kiểm tra chất lượng, quản lý trạng thái và quan sát hệ thống. Thiết kế này giúp Educational Chatbot tránh phụ thuộc hoàn toàn vào khả năng sinh tự do của LLM, đồng thời giữ được tính mở rộng cho các chức năng giáo dục như quiz, slide và giáo án.

Những đóng góp trên cho thấy các khó khăn chính của đề tài đã được xử lý theo hướng thực dụng: truy xuất thích nghi theo loại câu hỏi, điều phối kiểm soát nhiều intent, quiz tương tác tách khỏi artifact generation, multi-agent có contract rõ ràng, và người dùng được đưa vào vòng duyệt ở các điểm quyết định.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết những kết quả chính mà đồ án đã đạt được, đồng thời nêu ra các hướng phát triển tiếp theo để hệ thống có thể tiến gần hơn tới khả năng triển khai trong thực tế. Nếu Chương 5 tập trung phân tích các đóng góp kỹ thuật dưới góc nhìn thiết kế, thì Chương 6 đặt trọng tâm vào việc nhìn lại toàn bộ quá trình thực hiện dưới góc nhìn kết quả nghiên cứu và triển vọng ứng dụng.

6.1 Kết luận

Đồ án đã giải quyết bài toán xây dựng một trợ lý giáo dục cho môn Tin học THPT dựa trên dữ liệu sách giáo khoa chính thống. Hướng tiếp cận được lựa chọn là kết hợp truy xuất tri thức từ SGK với khả năng sinh nội dung của mô hình ngôn ngữ lớn, nhằm bảo đảm đầu ra vừa bám sát nguồn học liệu vừa đủ linh hoạt để phục vụ nhiều tác vụ khác nhau. Trên cơ sở đó, hệ thống đã được tổ chức thành một kiến trúc gồm tầng điều phối yêu cầu, tầng tri thức và tầng sinh nội dung, trong đó các tác vụ như hỏi đáp, sinh slide và soạn giáo án được xử lý trên cùng một nền dữ liệu thống nhất.

Về mặt phương pháp, đồ án đã xây dựng được một pipeline tương đối hoàn chỉnh, bắt đầu từ bước chuẩn hóa dữ liệu SGK, chunking theo cấu trúc bài học, biểu diễn nội dung bằng embedding tiếng Việt, lập chỉ mục trên vector database, kết hợp tìm kiếm từ khóa với tìm kiếm ngữ nghĩa, rồi đưa ngữ cảnh truy xuất được vào bước sinh câu trả lời hoặc sinh học liệu. Bên cạnh đó, hệ thống còn tích hợp các cơ chế điều phối như phân tích ngữ cảnh hội thoại, định tuyến ý định, quản lý trạng thái phiên và pipeline đa tác tử cho các tác vụ sinh nội dung dài. Những thành phần này giúp hệ thống không chỉ hoạt động như một chatbot hỏi đáp đơn thuần mà còn có khả năng xử lý các yêu cầu học tập phức hợp hơn.

Về mặt thực nghiệm, đồ án đã tách riêng việc đánh giá retrieve node và generate node để làm rõ đóng góp của từng thành phần trong pipeline. Với benchmark retrieval gồm 600 câu hỏi bao phủ 185 bài học, cấu hình tốt nhất của hệ thống đạt Hit@1 bằng 0,8183, Hit@10 bằng 0,9833 và MRR@10 bằng 0,8856. Kết quả này cho thấy cơ chế truy xuất được đề xuất có khả năng đưa đúng bài học mục tiêu vào các vị trí đầu của danh sách kết quả. Đối với generate node, đánh giá trên 246 mẫu bằng RAGAS cho thấy hệ thống đạt Faithfulness 0,9757 và Context Recall 0,9593, phản ánh rằng câu trả lời sinh ra nhìn chung bám sát ngữ cảnh và ít sinh thông tin ngoài tài liệu. Các kết quả đó là cơ sở thực nghiệm để khẳng định hướng tiếp cận của đồ án là khả thi đối với bài toán xây dựng trợ lý học tập bám sát SGK Tin học THPT.

Tuy vậy, đồ án vẫn còn một số giới hạn. Thứ nhất, hệ thống hiện mới được phát triển và đánh giá chủ yếu như một nguyên mẫu nghiên cứu, chưa được hoàn thiện thành một ứng dụng web có khả năng vận hành ổn định cho số lượng lớn người dùng thực. Thứ hai, miền dữ liệu mới giới hạn trong môn Tin học THPT, nên khả năng khái quát sang các môn học khác mới dừng ở mức tiềm năng kiến trúc. Thứ ba, một số thành phần vẫn phụ thuộc vào dịch vụ mô hình bên ngoài, do đó cần tiếp tục được tối ưu thêm nếu muốn triển khai ở quy mô rộng trong môi trường giáo dục thực tế.

6.2 Hướng phát triển

Hướng phát triển đầu tiên là hoàn thiện hệ thống thành một ứng dụng web đầy đủ thay vì dừng ở mức nguyên mẫu nghiên cứu. Ở hướng này, kiến trúc hiện tại cần được mở rộng thêm một lớp ứng dụng dành cho người dùng cuối, với cơ chế xác thực, phân quyền, lưu lịch sử tương tác và quản lý tài khoản một cách nhất quán. Bên cạnh kho tri thức dùng cho retrieval, hệ thống cũng cần một cơ sở dữ liệu nghiệp vụ riêng để quản lý người dùng, phiên học tập, tiến độ làm quiz, lịch sử tạo slide hoặc giáo án, cũng như các thông tin cấu hình cá nhân. Việc tách rõ cơ sở dữ liệu phục vụ tri thức và cơ sở dữ liệu phục vụ nghiệp vụ sẽ giúp hệ thống dễ bảo trì hơn và phù hợp hơn với yêu cầu triển khai thực tế.

Hướng phát triển thứ hai là thiết kế lại hệ thống theo hướng hỗ trợ nhiều người dùng đồng thời. Trong môi trường trường học thực, một hệ thống như vậy không chỉ phục vụ một người dùng đơn lẻ mà có thể phải xử lý nhiều yêu cầu song song từ học sinh và giáo viên ở các lớp khác nhau. Điều này đòi hỏi bổ sung các cơ chế như quản lý session độc lập theo người dùng, điều phối hàng đợi tác vụ dài, tối ưu tài nguyên cho retrieval và reranking, cũng như theo dõi hiệu năng ở mức hệ thống. Khi đó, các thành phần hiện tại như session state, trace và workflow đa tác tử có thể trở thành nền tảng để mở rộng theo hướng đa người dùng và triển khai theo chiều ngang.

Hướng phát triển thứ ba là đưa hệ thống vào bối cảnh sử dụng thực tế tại các trường THPT ở Việt Nam. Muốn làm được điều này, ngoài việc hoàn thiện hạ tầng phần mềm, hệ thống cần được kiểm chứng thêm thông qua các đợt thử nghiệm với học sinh và giáo viên thật. Các thử nghiệm như vậy sẽ cho phép đánh giá không chỉ độ chính xác kỹ thuật của retrieval hay generation, mà còn cả mức độ hữu ích sư phạm, mức độ dễ sử dụng và tác động thực tế tới quá trình dạy và học. Đây là bước cần thiết để chuyển đồ án từ một kết quả nghiên cứu khả thi sang một giải pháp có thể ứng dụng trong môi trường giáo dục.

Hướng phát triển thứ tư là mở rộng miền tri thức sang nhiều môn học khác ngoài

Tin học. Về nguyên tắc, kiến trúc mà đề án đề xuất không phụ thuộc tuyệt đối vào nội dung môn Tin học, mà phụ thuộc chủ yếu vào khả năng xây dựng được một kho tri thức có cấu trúc đủ tốt từ sách giáo khoa của từng môn. Do đó, nếu bổ sung được dữ liệu của các môn như Toán, Vật lý hoặc Hóa học và điều chỉnh lại một số chiến lược retrieval cũng như prompt sinh nội dung, hệ thống có thể mở rộng thành một trợ lý học tập liên môn. Khi đó, giá trị của đề án sẽ không chỉ nằm ở một ứng dụng cho môn Tin học, mà còn ở một khung kiến trúc có thể tái sử dụng cho nhiều bài toán giáo dục khác nhau.

Cuối cùng, một hướng phát triển có ý nghĩa dài hạn là tăng cường mức độ cá nhân hóa cho người học. Hệ thống hiện đã có nền tảng quản lý trạng thái phiên và lưu vết các hoạt động học tập cơ bản. Trong tương lai, nếu mở rộng thêm phần hồ sơ người học, lịch sử câu hỏi, các lỗi thường gặp và tiến độ theo từng chủ đề, hệ thống có thể tiến tới việc gợi ý nội dung ôn tập, điều chỉnh mức độ khó của câu hỏi hoặc đề xuất học liệu phù hợp với từng học sinh. Đây là hướng phát triển phù hợp với mục tiêu lâu dài của các hệ thống giáo dục thông minh, đồng thời cũng là bước tiếp theo tự nhiên của kiến trúc hiện tại.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[1] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[4] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[5] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[7] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>.
- [7] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml>.